

File: aoi_controller.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/aoi_lib

```
import time
import os, cv2, time, math, json, logging
from .cnc_controller import GRBLCNCController
from .plc_axis_controller import PLCAxisController
from .camera_controller import CameraController
from .position_manager import InspectionPositionManager, InspectionPosition, InspectionSequence
```

```
class CNCAOIController:
```

```
    """
```

Controlador principal que integra o movimento CNC e captura de imagem para aplicações de Inspeção Óptica Automatizada (AOI).

```
    """
```

```
def __init__(self,
              serial_port=None,
              camera_interface=None,
              use_plc=True,
              plc_host='192.168.1.5',
              plc_port=502):
```

```
    """
```

Inicializa o controlador AOI.

Args:

serial_port: Porta serial para conexão com a CNC (opcional)

camera_interface: Interface para a câmera (opcional)

```
    """
```

Seleciona backend de movimento: GRBL ou CLP via Modbus TCP

if use_plc:

cria e conecta no CLP

self.cnc = PLCAxisController(host=plc_host, port=plc_port)

——— Carrega calibr. mecânica do config (igual ao AdhesiveApp) ———

from .config_manager import AOIConfigManager

cfg = AOIConfigManager()

ppr = float(cfg.get("calibration", "pulses_per_rev", default=1.0))

pitch = float(cfg.get("calibration", "fuso_pitch", default=1.0))

cálculo pulses/mm idêntico à das steps/mm:

self.cnc.pulses_per_mm = ppr / pitch if pitch != 0 else 1.0

else:

self.cnc = GRBLCNCController()

self.camera = CameraController(camera_interface)

self.position_manager = InspectionPositionManager()

self.is_running_sequence = False

self.current_sequence: InspectionSequence | None = None

```
def __getattr__(self, name):
```

```
    """
```

Delegates movement methods (step_move, jog_start, move_absolute, etc.) to the underlying CNC backend so the UI can call them directly:

```

        controller.step_move(...), controller.jog_start(...), etc.
    """

    # intercepta chamadas típicas de controle manual
    if hasattr(self.cnc, name):
        return getattr(self.cnc, name)
    # fallback padrão
    raise AttributeError(f"{self.__class__.__name__}!r} has no attribute {name!r}")

def connect_cnc(self, port=None, baudrate=115200):
    """Conecta à máquina CNC."""
    return self.cnc.connect(port, baudrate)

def run_gcode_file(self, filename, callback=None):
    """
    Executa um arquivo G-CODE diretamente

    Args:
        filename: Caminho do arquivo G-CODE
        callback: Função chamada após cada captura
    """
    from aoi_lib.gcode_manager import GCodeManager
    gcode_manager = GCodeManager()

    # Carrega o G-CODE para uma sequência temporária
    gcode_data = gcode_manager.read_gcode_file(filename)

    if not gcode_data:
        raise ValueError(f"Erro ao carregar arquivo G-CODE: {filename}")

    # Cria posições a partir dos dados do G-CODE
    positions = []
    for pos_data in gcode_data['positions']:
        position = InspectionPosition(
            pos_data['name'],
            pos_data.get('x', 0.0),
            pos_data.get('y', 0.0),
            pos_data.get('z', 0.0),
            pos_data.get('camera_params', {})
        )
        positions.append(position)

    # Cria sequência temporária
    temp_sequence = InspectionSequence(gcode_data['sequence_name'])
    for pos in positions:
        temp_sequence.add_position(pos)

    # Executa a sequência
    return self.run_sequence(temp_sequence.name, callback)

def disconnect_cnc(self):
    """Desconecta da máquina CNC."""

```

```

# CLP: fecha Modbus
if isinstance(self.cnc, PLCAxisController):
    self.cnc.close()
    return True
# GRBL: serial
if getattr(self.cnc, 'is_connected', False):
    return self.cnc.disconnect()
return True

def connect_camera(self, camera_id=0):
    """Conecta à câmera."""
    return self.camera.connect(camera_id)

def add_inspection_position(self, name, x, y, z=0.0, camera_params=None):
    """
    Adiciona uma posição de inspeção ao gerenciador.

    Args:
        name: Nome descritivo da posição
        x, y: Coordenadas
        camera_params: Parâmetros específicos da câmera (opcional)
    """
    position = InspectionPosition(name, x, y, z, camera_params)
    self.position_manager.add_position(position)
    return position

def add_current_position(self, name, camera_params=None):
    """Adiciona a posição atual como uma posição de inspeção."""
    position = self.cnc.get_current_position()
    return self.add_inspection_position(
        name,
        position['x'],
        position['y'],
        position['z'],
        camera_params
    )

def create_sequence(self, name, positions=None):
    """Cria uma nova sequência de inspeção."""
    sequence = InspectionSequence(name)
    if positions:
        for pos in positions:
            sequence.add_position(pos)
    self.position_manager.add_sequence(sequence)
    return sequence

def run_sequence(self, sequence_name, callback=None):
    """
    Executa uma sequência de inspeção completa.

    Args:

```

sequence_name: Nome da sequência a executar

callback: Função chamada após cada captura com imagem e metadados

"""

```
sequence = self.position_manager.get_sequence(sequence_name)
```

```
if not sequence:
```

```
    raise ValueError(f"Sequência '{sequence_name}' não encontrada")
```

```
self.is_running_sequence = True
```

```
self.current_sequence = sequence
```

```
results = []
```

```
for idx, position in enumerate(sequence.positions):
```

```
    if not self.is_running_sequence:        # stop_sequence() pode ter sido chamado
```

```
        break
```

```
# 1. Move para a posição (X, Y, Z) - despacha para CLP ou GRBL
```

```
feed_rate = getattr(self, '_current_feed_rate', 1000)
```

```
if isinstance(self.cnc, PLCAxisController):
```

```
    # Executa o movimento absoluto exato em mm, sem truncar
```

```
    # (move_to_absolute_position converte internamente usando pulses_per_mm)
```

```
    self.cnc.move_to_absolute_position(
```

```
        x=position.x,
```

```
        y=position.y,
```

```
        z=position.z,
```

```
        feed_rate=feed_rate
```

```
    )
```

```
    # Aguarda idle de todos os eixos
```

```
    self.cnc.wait_for_idle()
```

```
else:
```

```
    # comportamento original GRBL
```

```
    self.cnc.move_to_absolute_position(
```

```
        position.x, position.y, position.z,
```

```
        feed_rate=feed_rate
```

```
    )
```

```
    self.cnc.wait_for_idle()
```

```
# 2. Captura a imagem
```

```
image = self.camera.capture(position.camera_params)
```

```
# 3. Monta resultado
```

```
result = {
```

```
    "position": position,
```

```
    "image": image,
```

```
    "timestamp": time.time(),
```

```
    "sequence": sequence.name,
```

```
    "index": idx
```

```
}
```

```
results.append(result)
```

```
if callback:
```

```
    callback(result)
```

```

self.is_running_sequence = False
return results      # opcional, mantido para retro-compat.

def set_feed_rate(self, feed_rate):
    """Define a velocidade de movimentação para sequências"""
    self._current_feed_rate = feed_rate

def stop_sequence(self):
    """Para a execução da sequência atual."""
    self.is_running_sequence = False

def save_positions(self, filename):
    """Salva todas as posições e sequências em um arquivo."""
    return self.position_manager.save_to_file(filename)

def load_positions(self, filename):
    """Carrega posições e sequências de um arquivo."""
    return self.position_manager.load_from_file(filename)

def generate_map(self, origin, end, step_x, step_y, folder, program_name):
    """
    Gera um mapa de captura em grade entre dois cantos definidos.
    Usa math.ceil para garantir que não pule nenhum passo
    e captura exatamente 1 imagem por ponto.
    """

    logger = logging.getLogger("CNCAOIController.generate_map")

    # 1) validações básicas
    os.makedirs(folder, exist_ok=True)
    points = list(self._grid_points(origin, end, step_x, step_y))
    cols = max(p[1] for p in points) + 1
    rows = max(p[0] for p in points) + 1
    logger.info("Gerando mapa: %d colunas x %d linhas (%d pts)",
                cols, rows, len(points))

    # 4) vai para a origem
    self.cnc.move_to_absolute_position(origin['x'], origin['y'])
    self.cnc.wait_for_idle()

    # 5) percorre a grade
    capture_map = []      # para salvar JSON no final
    # Usar velocidade configurada se disponível
    feed_rate = getattr(self, '_current_feed_rate', 1000)
    for row_idx, col_idx, x, y in points:
        # move + espera
        self.cnc.move_to_absolute_position(x, y, feed_rate=feed_rate)
        self.cnc.wait_for_idle()

    # captura única

```

```

img = self.camera.capture()
if img is not None:
    fname = f"{program_name}_r{row_idx:03d}_c{col_idx:03d}.png"
    cv2.imwrite(os.path.join(folder, fname), img)
    capture_map.append(
        {"row": row_idx, "col": col_idx,
         "x": x, "y": y, "file": fname}
    )
else:
    logger.warning("Falha ao capturar em X=%.3f Y=%.3f", x, y)

# curto delay para estabilizar
time.sleep(0.05)

```

6) salva metadados -----

```

try:
    with open(os.path.join(folder, f"{program_name}_map.json"), "w") as fp:
        json.dump(capture_map, fp, indent=2)
except Exception as jerr:
    logger.error("Falha ao salvar JSON de mapa: %s", jerr)

```

```

logger.info("generate_map: concluído com sucesso.")

```

@staticmethod

```

def _grid_points(origin, end, sx, sy, snake=True):

```

```

    """

```

Gera tuplas (row, col, x, y) seguindo padrão "zig-zag" (snake) opcional.

```

    """

```

```

    dx = end['x'] - origin['x']
    dy = end['y'] - origin['y']
    if dx <= 0 or dy <= 0:
        raise ValueError("Cantos inválidos (dx/dy devem ser positivos)")
    if sx <= 0 or sy <= 0:
        raise ValueError("Passos sx/sy devem ser > 0")

```

```

    cols = int(math.ceil(dx / sx)) + 1
    rows = int(math.ceil(dy / sy)) + 1
    x_pos = [origin['x'] + min(i * sx, dx) for i in range(cols)]
    y_pos = [origin['y'] + min(j * sy, dy) for j in range(rows)]

```

```

    for r, y in enumerate(y_pos):
        scan = x_pos if (r % 2 == 0 or not snake) else list(reversed(x_pos))
        for c_idx, x in enumerate(scan):
            col = c_idx if (r % 2 == 0 or not snake) else cols - 1 - c_idx
            yield r, col, x, y

```

File: camera_controller.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/aoi_lib

```
class CameraController:
```

```
    """
```

```
    Classe para controlar uma câmera e capturar imagens.
```

```
    Pode ser adaptada para diferentes interfaces de câmera.
```

```
    """
```

```
def __init__(self, camera_interface=None):
```

```
    """
```

```
    Inicializa o controlador de câmera.
```

```
    Args:
```

```
        camera_interface: Interface de câmera personalizada (opcional)
```

```
    """
```

```
    self.camera = camera_interface
```

```
    self.is_connected = False
```

```
    self.last_error = ""
```

```
def connect(self, camera_id=0):
```

```
    # 0) Se já está conectado, primeiro desconecta para garantir
```

```
    # liberação correta do dispositivo.
```

```
    if self.is_connected:
```

```
        self.disconnect()
```

```
    # 1) Se existe um objeto pendente de conexão anterior, libera-o.
```

```
    if self.camera and hasattr(self.camera, "release"):
```

```
        try:
```

```
            self.camera.release()
```

```
        except Exception:
```

```
            pass
```

```
        self.camera = None
```

```
    # 2) Caso seja uma interface personalizada, usa-la.
```

```
    if self.camera and hasattr(self.camera, "connect"):
```

```
        try:
```

```
            self.is_connected = self.camera.connect()
```

```
            return self.is_connected
```

```
        except Exception as e:
```

```
            self.last_error = str(e)
```

```
            return False
```

```
    # 3) Caminho padrão OpenCV -----
```

```
    try:
```

```
        import cv2
```

```
        self.camera = cv2.VideoCapture(camera_id, cv2.CAP_DSHOW)
```

```
        self.is_connected = self.camera.isOpened()
```

```
        if not self.is_connected:
```

```
            self.last_error = f"cv2.VideoCapture({camera_id}) não abriu"
```

```
        return self.is_connected
```

```
except Exception as e:
    self.last_error = str(e)
    return False
```

```
def disconnect(self):
```

```
    if not self.is_connected and not self.camera:
        return
```

```
    try:
```

```
        if self.camera:
            if hasattr(self.camera, "release"):
                self.camera.release()
                # Aguarda driver liberar o handle USB
                import time; time.sleep(0.1)
            elif hasattr(self.camera, "disconnect"):
                self.camera.disconnect()
```

```
    finally:
```

```
        # Zera referências para forçar recriação no próximo connect()
        self.camera = None
        self.is_connected = False
```

```
def trigger_capture(self, params=None):
```

```
    """
```

Aciona o trigger da câmera para captura.

Args:

params: Parâmetros específicos da câmera para o trigger

Returns:

A imagem capturada ou None em caso de erro

```
    """
```

Primeiro realiza o trigger

```
    if not self._activate_trigger(params):
        return None
```

Em seguida, captura a imagem

```
    return self.capture(params)
```

```
def _activate_trigger(self, params=None):
```

```
    """
```

Ativa o hardware trigger da câmera, se disponível.

Args:

params: Parâmetros específicos para o trigger

Returns:

True se o trigger foi bem-sucedido, False caso contrário

```
    """
```

```
    if not self.is_connected:
```

```
        self.last_error = "Câmera não conectada"
        return False
```

try:

```
# Verifica se a interface da câmera tem um método de trigger específico
if hasattr(self.camera, 'trigger'):
    # Interface personalizada com método de trigger
    return self.camera.trigger(params)

# Para câmeras OpenCV padrão, criar um atraso simulando trigger por software
# e garantir que o frame seja descartado/limpo antes da captura real
import time
if hasattr(self.camera, 'grab'):
    # Limpa o buffer de frames
    self.camera.grab()
    time.sleep(0.1) # Pequeno atraso para simular trigger
    return True

return True # Se não houver trigger específico, continua com a captura normal
```

except Exception as e:

```
self.last_error = f"Erro ao ativar trigger: {str(e)}"
return False
```

def has_hardware_trigger(self):

"""

Verifica se a câmera possui suporte a hardware trigger.

Returns:

True se suporta trigger por hardware, False caso contrário

"""

if not self.is_connected:

return False

Verifica se a interface da câmera tem capacidade explícita de trigger

return hasattr(self.camera, 'trigger') or hasattr(self.camera, 'has_hardware_trigger')

def capture(self, params=None):

"""

Captura uma imagem da câmera.

Args:

params: Parâmetros específicos da câmera (exposição, etc.)

Returns:

A imagem capturada ou None em caso de erro

"""

if not self.is_connected:

self.last_error = "Câmera não conectada"

return None

try:

Para interface personalizada

```

if hasattr(self.camera, 'capture'):
    image = self.camera.capture(params)
    if image is None:
        self.last_error = "A captura de imagem retornou None"
        return None
    return image

# Para OpenCV
import cv2
ret, frame = self.camera.read()
if ret and frame is not None:
    # Aplicar parâmetros se fornecidos
    if params:
        # Exemplo: ajustar brilho/contraste
        if 'brightness' in params:
            frame = cv2.convertScaleAbs(frame, alpha=params['brightness'])

    # Verifica se a imagem possui dados válidos
    if frame.size == 0 or frame.shape[0] == 0 or frame.shape[1] == 0:
        self.last_error = "Imagem capturada tem dimensões inválidas"
        return None

    return frame
else:
    self.last_error = "Falha ao capturar imagem (sem dados ou retorno negativo)"
    return None

except Exception as e:
    self.last_error = f"Erro na captura de imagem: {str(e)}"
    return None

def set_parameters(self, **kwargs):
    """
    Define parâmetros da câmera.

    Args:
        **kwargs: Parâmetros específicos da câmera
    """
    if not self.is_connected:
        return False

    try:
        # Para interface personalizada
        if hasattr(self.camera, 'set_parameters'):
            return self.camera.set_parameters(**kwargs)

        # Para OpenCV
        import cv2
        for key, value in kwargs.items():
            if key.lower() == 'exposure':
                self.camera.set(cv2.CAP_PROP_EXPOSURE, value)

```

```

        elif key.lower() == 'focus':
            self.camera.set(cv2.CAP_PROP_FOCUS, value)
            # Adicione outros parâmetros conforme necessário

    return True

except Exception as e:
    self.last_error = str(e)
    return False

```

File: cnc_controller.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/aoi_lib

```

import time
import serial
import serial.tools.list_ports
import threading
from grbl_streamer import GrblStreamer # Importa a biblioteca grbl-streamer
from pathlib import Path
import json

import logging
# Se ainda não existir, defina um logger para esta classe:
logger = logging.getLogger("GRBLCNCController")
logger.setLevel(logging.DEBUG)

class EventEmitter:
    """
    Sistema de eventos para comunicação assíncrona.
    Permite registrar múltiplos callbacks para diferentes eventos.
    """
    def __init__(self):
        self.callbacks = {}
        self.default_callbacks = []

    def connect(self, callback, event_name=None):
        """
        Conecta um callback a um evento específico ou como callback padrão.

        Args:
            callback: Função a ser chamada quando o evento ocorrer
            event_name: Nome do evento (se None, será adicionado aos callbacks padrão)
        """
        if event_name is None:
            self.default_callbacks.append(callback)
        else:
            if event_name not in self.callbacks:
                self.callbacks[event_name] = []
            self.callbacks[event_name].append(callback)

```

```
# Retorna uma função que pode ser usada para desconectar este callback
return lambda: self.disconnect(callback, event_name)
```

```
def disconnect(self, callback, event_name=None):
```

```
    """
```

```
    Desconecta um callback específico.
```

```
    Args:
```

```
        callback: Callback a ser removido
```

```
        event_name: Nome do evento do qual remover (se None, remove dos callbacks padrão)
```

```
    """
```

```
    if event_name is None:
```

```
        if callback in self.default_callbacks:
```

```
            self.default_callbacks.remove(callback)
```

```
    else:
```

```
        if event_name in self.callbacks and callback in self.callbacks[event_name]:
```

```
            self.callbacks[event_name].remove(callback)
```

```
def disconnect_all(self, event_name=None):
```

```
    """
```

```
    Desconecta todos os callbacks de um evento específico ou todos os eventos.
```

```
    Args:
```

```
        event_name: Nome do evento para limpar (se None, limpa todos os eventos)
```

```
    """
```

```
    if event_name is None:
```

```
        self.callbacks.clear()
```

```
        self.default_callbacks.clear()
```

```
    else:
```

```
        if event_name in self.callbacks:
```

```
            self.callbacks[event_name].clear()
```

```
def emit(self, *args, event_name=None, **kwargs):
```

```
    """
```

```
    Emite um evento, chamando todos os callbacks registrados.
```

```
    Args:
```

```
        *args: Argumentos posicionais para passar aos callbacks
```

```
        event_name: Nome do evento a ser emitido (se None, apenas callbacks padrão são chamados)
```

```
        **kwargs: Argumentos nomeados para passar aos callbacks
```

```
    """
```

```
    # Chama todos os callbacks padrão
```

```
    for callback in self.default_callbacks:
```

```
        try:
```

```
            callback(*args, **kwargs)
```

```
        except Exception as e:
```

```
            print(f"Erro em callback para evento: {e}")
```

```
    # Se um nome de evento for especificado, chama esses callbacks também
```

```
    if event_name is not None and event_name in self.callbacks:
```

```
        for callback in self.callbacks[event_name]:
```

```

try:
    callback(*args, **kwargs)
except Exception as e:
    print(f"Erro em callback para evento '{event_name}': {e}")

```

```

def has_listeners(self, event_name=None):

```

```

    """

```

Verifica se há callbacks registrados para um evento específico ou para qualquer evento.

Args:

event_name: Nome do evento a verificar (se None, verifica todos)

Returns:

bool: True se houver callbacks registrados, False caso contrário

```

    """

```

if event_name is None:

```

    return bool(self.default_callbacks) or any(bool(callbacks) for callbacks in self.callbacks.values())

```

else:

```

    return event_name in self.callbacks and bool(self.callbacks[event_name])

```

```

class GRBLCNCController:

```

```

    """

```

Classe para controlar uma máquina CNC usando GRBL.

Implementada usando a biblioteca grbl-streamer.

```

    """

```

```

def __init__(self):

```

```

    """Inicializa o controlador CNC."""

```

```

    self.grbl = None

```

```

    self.lock = threading.Lock()

```

```

    self.is_connected = False

```

```

    self.current_position = {'x': 0, 'y': 0, 'z': 0}

```

```

    self.machine_status = "Disconnected"

```

```

    self.last_response = ""

```

```

    # se True, inverte lógica dos eixos Y / Z (para a visão do operador)

```

```

    self.invert_y = False

```

```

    self.invert_z = False

```

```

    self.last_error = ""

```

```

    # Eventos para comunicação assíncrona

```

```

    self.response_received = EventEmitter()

```

```

    self.status_update = EventEmitter()

```

```

    self.position_update = EventEmitter()

```

```

    self.error_message = EventEmitter()

```

```

    # Thread para consulta periódica de status

```

```

    self.status_thread = None

```

```

    self.running = False

```

```

    # Controle de jog contínuo

```

```

self.jogging = False
self.current_jog_command = None
self.steps_to_mm_factor = 1.0

# Limites máximos de feed e aceleração (mm/min | mm/s²)
self.max_feed = {'x': 1000.0, 'y': 1000.0, 'z': 800.0}
self.max_acc = {'x': 50.0, 'y': 50.0, 'z': 30.0}

# evita mostrar várias vezes o mesmo aviso de clamp
self._feed_clamp_warned = False

# ----- NOVO BLOCO -----
# Lê a configuração para saber a cinemática desejada
cfg_path = Path(__file__).resolve().parent.parent / "aoi_config.json"
if cfg_path.exists():
    try:
        cfg_data = json.loads(cfg_path.read_text(encoding="utf-8"))
    except Exception:
        cfg_data = {}
else:
    cfg_data = {}
cnc_cfg = cfg_data.get("cnc", {})
self.kinematics_mode = cnc_cfg.get("system_type", "cartesian").lower()
self.corexy_cfg = cnc_cfg.get("corexy_config", {})

# LOGA a escolha para facilitar depuração
logger.info("Cinemática inicial: %s | corexy_cfg=%s",
            self.kinematics_mode, self.corexy_cfg)

# ----- seleção dinâmica de cinemática -----
def set_kinematics_mode(self, mode: str):
    """
    Altera o modo de cinemática em tempo-real.
    Aceita 'cartesian' ou 'corexy'.
    """
    mode = str(mode).lower()
    assert mode in ("cartesian", "corexy"), "Modo inválido"
    self.kinematics_mode = mode

# ----- helpers CoreXY -----
def _convert_xy_to_ab(self, x: float, y: float) -> tuple[float, float]:
    """
    Converte deslocamentos/cartesianas (X,Y) para pulsos dos motores
    A e B de um sistema CoreXY.

    Motor A = (+X) + (+Y)
    Motor B = (+X) - (+Y)
    """

```

O utilizador pode inverter o sentido de cada motor nas Preferências. Não aplicamos steps-per-unit aqui; o firmware já usa mm (ou steps) directos conforme a sua configuração.

```
"""
```

```
cfg = getattr(self, "corexy_cfg", {}) or {}
inv_a = -1 if cfg.get("motor_a_invert", False) else 1
inv_b = -1 if cfg.get("motor_b_invert", False) else 1

a = (x + y) * inv_a
b = (x - y) * inv_b
return a, b
```

A/B → X/Y (para leitura de posição)

```
def _convert_ab_to_xy(self, a: float, b: float) -> tuple[float, float]:
```

```
"""
```

Inverte a cinemática CoreXY

A = +X + Y

B = +X - Y

⇒ X = (A + B)/2

Y = (A - B)/2

Considera inversão individual dos motores, se configurado.

```
"""
```

```
cfg = getattr(self, "corexy_cfg", {}) or {}
inv_a = -1 if cfg.get("motor_a_invert", False) else 1
inv_b = -1 if cfg.get("motor_b_invert", False) else 1
a *= inv_a
b *= inv_b
x = (a + b) / 2.0
y = (a - b) / 2.0
return x, y
```

```
def set_invert_y(self, invert: bool = True):
```

```
    """Define se o eixo Y deve ser invertido (+Y vai para a frente do usuário)."""
```

```
    self.invert_y = bool(invert)
```

```
def set_invert_z(self, invert: bool = True):
```

```
    """Define se o eixo Z deve ser invertido (+Z = cima)."""
```

```
    self.invert_z = bool(invert)
```

```
def set_grbl_y_direction(self, forward_positive: bool = True):
```

```
"""
```

Altera o parâmetro \$3 (Step Dir Invert Mask) para inverter
(ou não) o eixo Y diretamente no firmware GRBL.

forward_positive=True → bit1 ligado (valor 2 adicionado)

forward_positive=False → bit1 desligado

```
"""
```

```
if not self.is_connected or not self.grbl:
```

```
    return False
```

```
try:
```

```
    # pede os settings com $#
```

```
    self.grbl.send_immediately("$")
```

```
    time.sleep(0.3) # pequena pausa para GRBL responder
```

```
    # A biblioteca expõe o dicionário de settings em grbl.settings
```

```

current_mask = int(self.grbl.settings.get("$3", 0))
want_bit = 0x02 if forward_positive else 0
new_mask = (current_mask | 0x02) if forward_positive else (current_mask & ~0x02)
if new_mask != current_mask:
    self.grbl.send_immediately(f"$3={new_mask}")
return True
except Exception:
    return False

```

```
def connect(self, port=None, baudrate=115200):
```

```
    """
```

Conecta à máquina CNC.

Args:

port: Porta serial (None para autodetecção)

baudrate: Taxa de transmissão

Returns:

bool: True se conectado com sucesso

```
    """
```

```
try:
```

```
    # Autodetectar porta se não especificada
```

```
    if port is None:
```

```
        ports = [p.device for p in serial.tools.list_ports.comports()]
```

```
        if not ports:
```

```
            self.last_error = "Nenhuma porta serial encontrada"
```

```
            return False
```

```
        port = ports[0]
```

```
    # Inicializa e configura o GrblStreamer
```

```
    self.grbl = GrblStreamer(port=port, baud=baudrate)
```

```
    # Configura callbacks para eventos do GrblStreamer
```

```
    self._setup_callbacks()
```

```
    # Desbloqueia a máquina
```

```
    self.send_command("$X", priority=True)
```

```
    # Configuração para manter motores energizados
```

```
    self._configure_motor_hold()
```

```
    # Garante que o Y esteja invertido fisicamente (positivo = frente)
```

```
    self.set_grbl_y_direction(forward_positive=True)
```

```
    # ----- Lê $$ para descobrir limits -----
```

```
    # Solicitamos os settings logo após o unlock para conhecer os
```

```
    # limites máximos de feed ($110 / $111) e armazená-los em
```

```
    # self.max_feed.
```

```
    logger.debug("CONEXÃO: Solicitando $$ para obter limites de velocidade")
```

```
    self.grbl.send_immediately("$$")
```

```
    time.sleep(0.5)          # pequeno atraso p/ GRBL responder
```

```

self._cache_settings_from_grbl()

# Inicia thread de consulta de status
self.running = True
self.status_thread = threading.Thread(target=self._status_polling_thread, daemon=True)
self.status_thread.start()

self.is_connected = True
self.machine_status = "Idle" # Estado inicial presumido
return True

except Exception as e:
    self.last_error = str(e)
    return False

def _configure_motor_hold(self):
    """
    Configura o GRBL para manter os motores energizados quando parados.
    Isso evita que os motores fiquem 'soltos' e possam ser girados manualmente.
    """
    try:
        # $1=255: Mantém motores sempre energizados
        self.send_command("$1=255", priority=True)
        time.sleep(0.1)

        logger.info("MOTOR HOLD: Configurado $1=255 para manter motores energizados")

    except Exception as e:
        logger.error(f"Erro ao configurar motor hold: {e}")

def set_motor_hold_enabled(self, enabled: bool):
    """
    Habilita ou desabilita o travamento dos motores quando parados.

    Args:
        enabled: True para manter motores energizados, False para desligar após movimento
    """
    if not self.is_connected:
        return False

    try:
        if enabled:
            # Mantém motores sempre energizados
            self.send_command("$1=255", priority=True)
            logger.info("MOTOR HOLD: Motores configurados para permanecer energizados")
        else:
            # Desliga motores após 25ms de inatividade (padrão)
            self.send_command("$1=25", priority=True)
            logger.info("MOTOR HOLD: Motores configurados para desligar após inatividade")

    return True

```

```
except Exception as e:
    logger.error(f"Erro ao configurar motor hold: {e}")
    return False
```

```
def _setup_callbacks(self):
    """
    Configura callbacks para eventos do GrblStreamer.
    Esta função deve ser adaptada com base na API real da biblioteca.
    """
    # Configuração para receber atualizações de status da máquina
    self.grbl.register_status_callback(self._on_status_update)

    # Configuração para receber respostas de comandos
    self.grbl.register_response_callback(self._on_response_received)

    # Configuração para receber notificações de erro
    self.grbl.register_error_callback(self._on_error_message)
```

```
def _status_polling_thread(self):
    """Thread para consulta periódica de status."""
    while self.running and self.is_connected:
        try:
            # Requisição de status ao GRBL a cada 200ms
            if self.grbl:
                self.grbl.get_status()
                time.sleep(0.2)
        except Exception:
            time.sleep(0.5) # Maior intervalo em caso de erro
```

```
def disconnect(self):
    """Desconecta da máquina CNC."""
    # Garante que o jog é parado antes de desconectar
    if self.jogging:
        self.stop_continuous_jog()

    self.running = False
    if self.status_thread:
        self.status_thread.join(1.0) # Espera até 1 segundo pela thread

    if self.grbl:
        try:
            self.grbl.close()
            self.grbl = None
        except Exception as e:
            self.last_error = str(e)

    self.is_connected = False
    self.machine_status = "Disconnected"
```

```
def is_moving(self):
```

```
"""Verifica se a máquina está em movimento."""
```

```
return self.machine_status == "Run" or self.machine_status == "Jog"
```

```
def wait_for_idle(self,  
    timeout: float = 5.0,  
    poll_interval: float = 0.1,  
    idle_grace: float = 0.4) -> bool:  
    """
```

Aguarda o término de qualquer movimento.

A lógica:

- 1) força consulta de status (?) a cada 'poll_interval'
- 2) aguarda ver a máquina entrar em Run ou Jog
- 3) depois aguarda voltar a Idle

Assim evitamos o "salto" observado quando a variável machine_status ainda está 'Idle' logo após o envio do comando.

```
    """
```

```
if not self.is_connected or not self.grbl:  
    return False
```

```
start = time.time()  
saw_motion = False  
idle_since = None
```

```
while True:
```

```
    # força GRBL a reportar um status  
    try:  
        self.grbl.send_immediately("?")  
    except Exception:  
        pass
```

```
    state = self.machine_status # atualizado pelo callback _on_status_update
```

```
    if state in ("Run", "Jog"):  
        saw_motion = True  
        idle_since = None # zera caso volte a ver Run  
    elif state == "Idle":  
        if saw_motion: # cenário normal  
            return True  
        # nunca vimos Run → conta um "grace time" em Idle  
        idle_since = idle_since or time.time()  
        if (time.time() - idle_since) >= idle_grace:  
            logger.debug("wait_for_idle: Idle estável sem movimento; liberando cedo.")  
            return True
```

```
    if time.time() - start > timeout:  
        logger.warning("wait_for_idle: timeout depois de %.1fs (state=%s)", timeout, state)  
        return False
```

```
    time.sleep(poll_interval)
```

```
def get_current_position(self):
```

```
''''
```

Retorna a posição atual do CNC.

Certifica-se de retornar uma cópia do dicionário para evitar alteração acidental.

```
''''
```

```
logger.debug(f"get_current_position: Retornando posição atual: {self.current_position if hasattr(self, 'current_position')}
```

```
if hasattr(self, 'current_position'):
```

```
    # current_position é um dicionário com chaves 'x', 'y' e 'z'
```

```
    # Retorna uma cópia para evitar alterações acidentais
```

```
    return self.current_position.copy()
```

```
return {'x': 0, 'y': 0, 'z': 0}
```

```
def move_to_absolute_position(self, x=None, y=None, z=None, feed_rate=1000):
```

```
''''
```

Move para uma posição absoluta.

Args:

x, y: Coordenadas de destino (None para não mover esse eixo)

feed_rate: Velocidade de avanço em mm/min

```
''''
```

```
if not self.is_connected:
```

```
    return False
```

```
# Muda para modo absoluto
```

```
self.send_command("G90", priority=True)
```

```
# Ajusta feed se exceder limite configurado
```

```
axis_limit = max(
```

```
    self.max_feed['x'] if x is not None else 0,
```

```
    self.max_feed['y'] if y is not None else 0,
```

```
    self.max_feed['z'] if z is not None else 0
```

```
) or min(self.max_feed.values()) # fallback
```

```
if feed_rate > axis_limit:
```

```
    if not self._feed_clamp_warned:
```

```
        logger.warning("Feed solicitado (%s) > limite (%s). "
```

```
            "Será enviado como %s",
```

```
            feed_rate, axis_limit, axis_limit)
```

```
        self._feed_clamp_warned = True
```

```
        feed_rate = axis_limit
```

```
# -----
```

```
# Constrói o comando considerando o modo cinemático.
```

```
# – cartesian : envia X/Y diretos
```

```
# – corexy : converte (X,Y) → (A,B) e envia nos eixos X/Y
```

```
# -----
```

```
command = "G1"
```

```
if self.kinematics_mode == "corexy":
```

```
    # Y lógico pode ser invertido antes da conversão
```

```
    x_cart = 0.0 if x is None else float(x)
```

```
    y_cart = 0.0 if y is None else float((-y) if self.invert_y else y)
```

```
    a, b = self._convert_xy_to_ab(x_cart, y_cart)
```

```
    command += f" X{a:.3f} Y{b:.3f}"
```

```

else: # cartesiano
    if x is not None:
        command += f" X{float(x):.3f}"
    if y is not None:
        y_send = -y if self.invert_y else y
        command += f" Y{float(y_send):.3f}"

```

```

# Z é igual em ambos os modos
if z is not None:
    z_send = -z if self.invert_z else z
    command += f" Z{float(z_send):.3f}"

```

```

command += f" F{feed_rate}"

```

```

# Envia o comando
self.send_command(command, priority=True)
return True

```

```

def step_move(self, axis: str, distance: float, feed_rate: float = 1000):

```

```

    """

```

Move um único passo (distância em mm, sinal define direção) no eixo selecionado.
Encapsula o uso de move_relative para manter a UI livre de detalhes G-code.

```

    """

```

```

if axis.upper() == "X":
    return self.move_relative(x=distance, y=0, feed_rate=feed_rate)
elif axis.upper() == "Y":
    return self.move_relative(x=0, y=distance, feed_rate=feed_rate)
elif axis.upper() == "Z":
    return self.move_relative(x=0, y=0, z=distance, feed_rate=feed_rate)
else:
    logger.error(f"step_move: eixo inválido '{axis}'")
    return False

```

"Aliases" para manter nomenclatura intuitiva na UI

```

def jog_start(self, axis, direction, feed_rate=1000):
    """Wrapper p/ iniciar jog contínuo."""
    return self.start_continuous_jog(axis, direction, feed_rate)

```

```

def jog_stop(self):
    """Wrapper p/ parar jog contínuo."""
    return self.stop_continuous_jog()

```

```

def move_relative(self, x=0, y=0, z=0, feed_rate=1000):
    """

```

Move em relação à posição atual.

Args:

x, y: Distância a mover em cada eixo
feed_rate: Velocidade de avanço em mm/min

```

    """

```

```

if not self.is_connected:

```

```

return False

# -----
# SANITIZAÇÃO: evita None → TypeError em abs()
# -----
x = 0.0 if x is None else float(x)
y = 0.0 if y is None else float(y)
z = 0.0 if z is None else float(z)

# Ajuste de feed
axis_limit = max(
    self.max_feed['x'] if abs(x) > 0 else 0,
    self.max_feed['y'] if abs(y) > 0 else 0,
    self.max_feed['z'] if abs(z) > 0 else 0
) or min(self.max_feed.values())
if feed_rate > axis_limit:
    if not self._feed_clamp_warned:
        logger.warning("Feed solicitado (%s) > limite (%s). "
            "Será enviado como %s",
            feed_rate, axis_limit, axis_limit)
        self._feed_clamp_warned = True
    feed_rate = axis_limit

# Muda para modo relativo
self.send_command("G91", priority=True)

# -----
# Constrói o comando (com conversão CoreXY, se activada)
# -----
if self.kinematics_mode == "corexy":
    tgt_x = x if x is not None else 0.0
    tgt_y = y if y is not None else 0.0
    a, b = self._convert_xy_to_ab(tgt_x, tgt_y)
    command = f"G1 X{a:.3f} Y{b:.3f}"
    if z is not None:
        z_send = -z if self.invert_z else z
        command += f" Z{z_send}"
        command += f" F{feed_rate}"
else: # cartesiano
    y_send = -y if self.invert_y else y
    command = "G1"
    command += f" X{x} Y{y_send}"
    if z != 0:
        z_send = -z if self.invert_z else z
        command += f" Z{z_send}"
        command += f" F{feed_rate}"

# Envia o comando
self.send_command(command, priority=True)
return True

```

```
def start_continuous_jog(self, axis, direction, feed_rate=1000):
```

```
    """
```

Inicia um movimento jog contínuo em um eixo específico.

O movimento continua até que stop_continuous_jog seja chamado.

Args:

axis: Eixo a mover ('X' ou 'Y')

direction: Direção do movimento (1 ou -1)

feed_rate: Velocidade em mm/min

Returns:

bool: True se o comando foi iniciado com sucesso

```
    """
```

```
if not self.is_connected or self.jogging:
```

```
    return False
```

```
# Para qualquer movimento anterior e garante modo relativo
```

```
self.emergency_stop()
```

```
time.sleep(0.1) # Pequena pausa para garantir que o GRBL processou o comando de parada
```

```
self.send_command("~", priority=True) # Retoma após parada
```

```
self.send_command("G91", priority=True) # Modo relativo
```

```
# -----
```

```
# 1. Distância "longa" para JOG (só para ficar em movimento)
```

```
# -----
```

```
distance = 1000 * direction
```

```
# -----
```

```
# 2. Gera o comando $J=... com total segurança
```

```
# -----
```

```
jog_command = None # ← sempre inicializado
```

```
axis_uc = axis.upper()
```

```
if self.kinematics_mode == "corexy":
```

```
# ■- CoreXY: converter XY→AB mas enviar nos eixos X/Y
```

```
if axis_uc == "X":
```

```
    a, b = self._convert_xy_to_ab(distance, 0)
```

```
    jog_command = f"$J=G91 X{a:.3f} Y{b:.3f} F{feed_rate}"
```

```
elif axis_uc == "Y":
```

```
    a, b = self._convert_xy_to_ab(0, distance)
```

```
    jog_command = f"$J=G91 X{a:.3f} Y{b:.3f} F{feed_rate}"
```

```
elif axis_uc == "Z":
```

```
    dist_z = -distance if self.invert_z else distance
```

```
    jog_command = f"$J=G91 Z{dist_z:.3f} F{feed_rate}"
```

```
else: # ■- Cartesiano puro
```

```
if axis_uc == "X":
```

```
    jog_command = f"$J=G91 X{distance:.3f} F{feed_rate}"
```

```
elif axis_uc == "Y":
```

```
    dist_y = -distance if self.invert_y else distance
```

```
    jog_command = f"$J=G91 Y{dist_y:.3f} F{feed_rate}"
```

```

elif axis_uc == "Z":
    dist_z = -distance if self.invert_z else distance
    jog_command = f"$J=G91 Z{dist_z:.3f} F{feed_rate}"

# -----
# 3. Segurança extra: aborta se algo ficou sem tratar
# -----
if jog_command is None:
    logger.error("start_continuous_jog: comando não gerado "
                "(axis=%s mode=%s)", axis_uc, self.kinematics_mode)
    return False

# -----
# 4. Atualiza estado e envia comando
# -----
self.jogging = True
self.current_jog_command = jog_command
return self.send_command(jog_command, priority=True)

def _cache_settings_from_grbl(self):
    """
    Copia $settings do objeto GrblStreamer (se disponível) e
    atualiza self.max_feed.
    """
    if not self.grbl or not hasattr(self.grbl, "settings"):
        logger.warning("_cache_settings_from_grbl: settings não disponíveis")
        return
    try:
        s = self.grbl.settings
        self.max_feed['x'] = float(s.get("$110", self.max_feed['x']))
        self.max_feed['y'] = float(s.get("$111", self.max_feed['y']))
        self.max_acc['x'] = float(s.get("$120", self.max_acc['x']))
        self.max_acc['y'] = float(s.get("$121", self.max_acc['y']))
        self.max_feed['z'] = float(s.get("$112", self.max_feed['z']))
        self.max_acc['z'] = float(s.get("$122", self.max_acc['z']))
        logger.info("Limites de feed carregados – X:%s Y:%s (mm/min)",
                    self.max_feed['x'], self.max_acc['x'],
                    self.max_feed['y'], self.max_acc['y'])
    except Exception as e:
        logger.error("Falha ao parsear $settings: %s", e)

def stop_continuous_jog(self):
    """
    Para o movimento jog contínuo atual.

    Returns:
        bool: True se o comando foi enviado com sucesso
    """
    if not self.is_connected or not self.jogging:
        return False

```

```
# Cancela o jog enviando Feed Hold (!).
# NÃO enviamos '~' para não reiniciar o jog.
success = self.send_command("!", priority=True)
```

```
# Atualiza o estado interno: jog deve estar parado
self.jogging = False
self.current_jog_command = None
return success
```

```
def home(self):
    """Envia a máquina para a posição home."""
    if not self.is_connected:
        return False

    self.send_command("$H")
    return True
```

```
def set_zero(self):
    """Define a posição atual como zero."""
    if not self.is_connected:
        return False

    self.send_command("G92 X0 Y0")
    return True
```

```
def unlock(self):
    """Envia o comando de desbloqueio ($X) para o GRBL."""
    if not self.is_connected or not self.grbl:
        logger.warning("UNLOCK: CNC não conectada.")
        return False
    try:
        logger.info("UNLOCK: Enviando comando de desbloqueio ($X)")

        # A biblioteca tem um método dedicado para killalarm
        self.grbl.killalarm()

        logger.info("UNLOCK: Comando de desbloqueio enviado.")
        return True
    except Exception as e:
        logger.error(f"UNLOCK: Erro ao enviar comando de desbloqueio: {e}", exc_info=True)
        return False
```

```
def emergency_stop(self):
    """Para todos os movimentos imediatamente (Feed Hold)."""
    if not self.is_connected or not self.grbl:
        logger.warning("EMERGENCY STOP: CNC não conectada.")
        return False
    try:
        logger.info("EMERGENCY STOP: Enviando comando Feed Hold (!)")

        # CORREÇÃO: Usar send_immediately() em vez de send_realtime_command()
```

```
self.grbl.send_immediately("!")
```

```
self.jogging = False # Atualiza o estado de jog, pois o movimento parou
```

```
self.machine_status = "Hold" # Estado esperado após '!' é Hold
```

```
logger.info("EMERGENCY STOP: Comando Feed Hold (!) enviado. Estado esperado: Hold.")
```

```
return True
```

```
except Exception as e:
```

```
    logger.error(f"EMERGENCY STOP: Erro ao enviar comando Feed Hold (!): {e}", exc_info=True)
```

```
    return False
```

```
def send_soft_reset(self):
```

```
    """Envia um comando de Soft Reset (Ctrl+X) para o GRBL."""
```

```
    if not self.is_connected or not self.grbl:
```

```
        logger.warning("SOFT RESET: CNC não conectada.")
```

```
        return False
```

```
    try:
```

```
        # O Soft Reset no GRBL é o caractere Ctrl+X (código ASCII 0x18 ou 24 em decimal)
```

```
        logger.info("SOFT RESET: Enviando comando Soft Reset (Ctrl+X)")
```

```
        # Utiliza o método send_immediately() com o caractere adequado
```

```
        self.grbl.send_immediately("\x18")
```

```
        self.jogging = False # Assume que o reset também para o jog
```

```
        self.machine_status = "Alarm" # O estado esperado após reset é Alarme
```

```
        logger.info("SOFT RESET: Comando enviado. Estado esperado: Alarm.")
```

```
        return True
```

```
    except Exception as e:
```

```
        # Loga o erro se ocorrer
```

```
        logger.error(f"SOFT RESET: Erro ao enviar comando Soft Reset: {e}", exc_info=True)
```

```
        return False
```

```
def send_command(self, command, priority=False):
```

```
    """
```

```
    Envia um comando personalizado ao GRBL.
```

```
    Args:
```

```
        command: Comando G-code
```

```
        priority: True para enviar com prioridade
```

```
    """
```

```
    if not self.is_connected or not self.grbl:
```

```
        return False
```

```
    try:
```

```
        # Comandos de controle realtime (!, ~, ?) são tratados especialmente
```

```
        if command in ["!", "~", "?"]:
```

```
            self.grbl.send_immediately(command)
```

```
        # Comandos prioritários são enviados com prioridade
```

```
        elif priority:
```

```
            self.grbl.send_immediately(command)
```

```
        # Comandos "normais" vão para fila padrão
```

```

else:
    self.grbl.send(command) # método normal do grbl-streamer
    return True
except Exception as e:
    self.last_error = str(e)
    self.error_message.emit(str(e))
    return False

def _on_response_received(self, response):
    """Callback quando uma resposta é recebida."""
    self.last_response = response
    self.response_received.emit(response)

def _on_status_update(self, status_data):
    """Callback quando o status da máquina é atualizado."""
    logger.debug(">> _on_status_update chamado com status_data: %s", status_data)

    # Extraí o estado da máquina
    state = status_data.get('state', 'Unknown')
    self.machine_status = state
    logger.debug("Estado da máquina extraído: %s", state)

    # Extraí a posição (espera-se um dicionário com chaves 'x', 'y' e 'z')
    position = status_data.get('position', None)
    if position:
        # -----
        # Ajuste da CINEMÁTICA para exibir valores CARTESIANOS reais
        # -----
        if self.kinematics_mode == "corexy":
            # 1) valores de motores A/B vindos do GRBL
            a_phys = float(position.get('x', 0)) # eixo X físico = motor A
            b_phys = float(position.get('y', 0)) # eixo Y físico = motor B
            # 2) converte para XY cartesianas
            x_cart, y_cart = self._convert_ab_to_xy(a_phys, b_phys)
            # 3) aplica inversão lógica de Y (visão do operador) se habilitada
            y_cart_log = -y_cart if self.invert_y else y_cart
            new_position = {
                'x': x_cart,
                'y': y_cart_log,
                'z': position.get('z', 0)
            }
        else:
            # Modo cartesiano puro (mantém lógica anterior)
            new_position = {
                'x': position.get('x', 0),
                'y': -position.get('y', 0) if self.invert_y else position.get('y', 0),
                'z': position.get('z', 0)
            }
        logger.debug("Posição (XY ajustada): %s", new_position)
        if new_position['x'] == 0 and new_position['y'] == 0:
            logger.warning("As coordenadas X e Y continuam 0 mesmo após movimento. status_data recebido: %s", status_data)

```

```

        self.current_position = new_position
        self.position_update.emit(self.current_position)
    else:
        logger.warning("status_data não contém a chave 'position': %s", status_data)

    status_string = f"<{state}>|MPos:{self.current_position['x']:.3f},{self.current_position['y']:.3f},{self.current_position['z']:.3f}>"
    logger.debug("String de status formada: %s", status_string)
    self.status_update.emit(status_string)

def _on_error_message(self, error):
    """Callback quando ocorre um erro."""
    self.last_error = error
    self.error_message.emit(error)

```

File: config_manager.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/aoi_lib

#config_manager.py

```

import os, json, logging
from pathlib import Path

# -----
# AOIConfigManager – gerencia arquivo JSON de preferências
# -----

class AOIConfigManager:
    _DEFAULT_CFG = {
        "cnc": {
            "system_type": "cartesian",      # cartesian | corexy
            "max_feed": {"x": 2500.0, "y": 2500.0, "z": 800.0}, # + $112
            "max_acc": {"x": 120.0, "y": 120.0, "z": 60.0}, # + $122
            "invert_y": True,                 # sentido lógico (+Y frente)
            "invert_z": False,                # novo – (+Z = cima)
            "motor_hold_enabled": True, # Manter motores energizados quando parados
            # apenas se corexy estiver selecionado
            "corexy_config": {
                "motor_a_invert": False,
                "motor_b_invert": False,
                "steps_per_unit": 80.0
            }
        },
        "connections": {
            "last_cnc_port": "",
            "plc_host": "192.168.0.5", # IP padrão do PLC
            "plc_port": 502, # Porta padrão Modbus TCP
            "auto_connect_cnc": True,
            "last_camera_id": 0,
            "auto_connect_camera": True
        },
        "ui": {

```

```

        "theme": "light"
    },
    # ----- NOVOS GRUPOS -----
    "movement": {                # controles manuais
        "step_size": 10.0,        # mm
        "feed_rate": 1000.0       # mm/min
    },
    "calibration": {             # parâmetros mecânicos
        "pulses_per_rev": 400.0,
        "fuso_pitch": 5.0         # mm / volta
    }
}

# ---- atalhos para gravação rápida -----
def remember_cnc_port(self, port: str):
    self.set("connections", "last_cnc_port", value=port)
def remember_camera_id(self, cam_id: int | str):
    self.set("connections", "last_camera_id", value=int(cam_id))
def remember_step_feed(self, step: float, feed: float):
    self.set("movement", "step_size", value=step)
    self.set("movement", "feed_rate", value=feed)
def remember_calibration(self, pulses: float, fuso: float):
    self.set("calibration", "pulses_per_rev", value=pulses)
    self.set("calibration", "fuso_pitch", value=fuso)

# ----- novo atalho para lembrar o sistema de eixos -----
def remember_system_type(self, sys_type: str):
    """Grava cartesian|corexy escolhido pelo usuário."""
    self.set("cnc", "system_type", value=sys_type.lower())

def __init__(self, cfg_path: str | None = None):
    self.log = logging.getLogger("AOIConfig")
    # Se o caminho não for informado, grava ao lado do executável
    default_path = Path(__file__).resolve().parent.parent / "aoi_config.json"
    self.cfg_path = Path(cfg_path) if cfg_path else default_path
    self.data = {}
    self.load()

# ----- json -----
def load(self):
    if self.cfg_path.exists():
        try:
            with open(self.cfg_path, "r", encoding="utf-8") as fp:
                self.data = json.load(fp)
            self.log.info("Config carregada de %s", self.cfg_path)
        except Exception as e:
            self.log.error("Falha ao ler config, usando padrão: %s", e)
            self.data = self._DEFAULT_CFG.copy()
    else:
        self.log.info("Arquivo de config inexistente – criando padrão em %s", self.cfg_path)
        self.data = self._DEFAULT_CFG.copy()

```

```
self.save()
```

```
def save(self):
```

```
    try:
```

```
        with open(self.cfg_path, "w", encoding="utf-8") as fp:
```

```
            json.dump(self.data, fp, indent=2)
```

```
            self.log.debug("Config salva em %s", self.cfg_path)
```

```
    except Exception as e:
```

```
        self.log.error("Erro salvando config: %s", e)
```

```
# ----- API simples -----
```

```
def get(self, *path, default=None):
```

```
    ref = self.data
```

```
    for p in path:
```

```
        if p not in ref:
```

```
            return default
```

```
        ref = ref[p]
```

```
    return ref
```

```
def set(self, *path, value):
```

```
    ref = self.data
```

```
    for p in path[:-1]:
```

```
        ref = ref.setdefault(p, {})
```

```
    ref[path[-1]] = value
```

```
    self.save()
```

```
# ----- aplica limites ao GRBL -----
```

```
def apply_to_cnc(self, cnc):
```

```
    """
```

```
    Envia $110,$111,$120,$121 e sentido de Y logo  
    após a conexão da CNC.
```

```
    """
```

```
# modo cartesiano x corexy -----
```

```
try:
```

```
    sys_type = self.get("cnc", "system_type", default="cartesian")
```

```
    cnc.set_kinematics_mode(sys_type)
```

```
    # passa configurações específicas do CoreXY
```

```
    cnc.corexy_cfg = self.get("cnc", "corexy_config", default={})
```

```
    self.log.info("Modo de cinemática aplicado: %s", sys_type)
```

```
except Exception as e:
```

```
    self.log.error("Falha ao aplicar modo de cinemática: %s", e)
```

```
if not cnc or not cnc.is_connected:
```

```
    return
```

```
maxf = self.get("cnc", "max_feed", default={})
```

```
acc = self.get("cnc", "max_acc", default={})
```

```
# ----- GARANTE EIXO Z -----
```

```
# Se o usuário ainda não possui as novas chaves no JSON,
```

```
# criamos valores seguros para evitar KeyError.
```

```
if "z" not in maxf:
```

```
    maxf["z"] = 800.0      # mm/min (ajuste depois em Preferências)
```

```

        self.set("cnc", "max_feed", "z", value=maxf["z"])
    if "z" not in acc:
        acc["z"] = 60.0          # mm/s²
        self.set("cnc", "max_acc", "z", value=acc["z"])
    invert_y = self.get("cnc", "invert_y", default=True)
    invert_z = self.get("cnc", "invert_z", default=False)
    cmds = [
        f"$110={maxf['x']}", f"$111={maxf['y']}", f"$112={maxf['z']}",
        f"$120={acc['x']}", f"$121={acc['y']}", f"$122={acc['z']}",
    ]
    for c in cmds:
        cnc.send_command(c, priority=True)
    cnc.set_invert_y(invert=invert_y)
    cnc.set_invert_z(invert=invert_z)
    # Configuração de motor hold
    motor_hold = self.get("cnc", "motor_hold_enabled", default=True)
    cnc.set_motor_hold_enabled(motor_hold)
    self.log.info("Limites aplicados ao GRBL: feed %s acc %s invert_y=%s",
                  maxf, acc, invert_y, invert_z)

```

```

# =====

```

```

# SettingsDialog – UI PyQt6 para editar as preferências

```

```

# =====

```

```

from PyQt6.QtWidgets import (QDialog, QFormLayout, QDoubleSpinBox, QCheckBox,
                             QPushButton, QHBoxLayout, QGroupBox, QComboBox)

```

```

class SettingsDialog(QDialog):

```

```

    def __init__(self, cfg: AOIConfigManager, parent=None):

```

```

        super().__init__(parent)

```

```

        self.setWindowTitle("Preferências da AOI / CNC")

```

```

        self.cfg = cfg

```

```

        form = QFormLayout(self)

```

```

# ----- widgets para feed / acel -----

```

```

self.spin_f_x = QDoubleSpinBox(); self.spin_f_x.setRange(1, 30000)

```

```

self.spin_f_y = QDoubleSpinBox(); self.spin_f_y.setRange(1, 30000)

```

```

self.spin_f_z = QDoubleSpinBox(); self.spin_f_z.setRange(1, 30000)

```

```

self.spin_a_x = QDoubleSpinBox(); self.spin_a_x.setRange(1, 1000)

```

```

self.spin_a_y = QDoubleSpinBox(); self.spin_a_y.setRange(1, 1000)

```

```

self.spin_a_z = QDoubleSpinBox(); self.spin_a_z.setRange(1, 1000)

```

```

self.chk_invert_y = QCheckBox("Inverter lógica do eixo Y (+Y frente)")

```

```

self.chk_invert_z = QCheckBox("Inverter lógica do eixo Z (+Z cima)")

```

```

# Configuração de motor hold

```

```

self.chk_motor_hold = QCheckBox("Manter motores energizados quando parados")

```

```

self.chk_motor_hold.setToolTip("Evita que motores fiquem 'soltos' e possam ser girados manualmente")

```

```

# valores atuais

```

```

self.spin_f_x.setValue(cfg.get("cnc", "max_feed", "x"))

```

```

self.spin_f_y.setValue(cfg.get("cnc", "max_feed", "y"))

```

```

self.spin_a_x.setValue(cfg.get("cnc", "max_acc", "x"))

```

```

self.spin_a_y.setValue(cfg.get("cnc", "max_acc", "y"))
# Usa False como valor-padrão caso a chave ainda não exista no JSON
self.chk_invert_y.setChecked(cfg.get("cnc", "invert_y", default=False))
self.chk_invert_z.setChecked(cfg.get("cnc", "invert_z", default=False))
self.spin_f_z.setValue(cfg.get("cnc", "max_feed", "z"))
self.spin_a_z.setValue(cfg.get("cnc", "max_acc", "z"))
self.chk_motor_hold.setChecked(cfg.get("cnc", "motor_hold_enabled", default=True))

```

```

form.addRow("Feed máx X (mm/min):", self.spin_f_x)
form.addRow("Feed máx Y (mm/min):", self.spin_f_y)
form.addRow("Feed máx Z (mm/min):", self.spin_f_z)
form.addRow("Acel máx X (mm/s²):", self.spin_a_x)
form.addRow("Acel máx Y (mm/s²):", self.spin_a_y)
form.addRow("Acel máx Z (mm/s²):", self.spin_a_z)
form.addRow(self.chk_invert_y)
form.addRow(self.chk_invert_z)
form.addRow(self.chk_motor_hold)

```

```

# --- NOVO BLOCO: seleção do tipo de sistema -----
self.combo_sys = QComboBox()
self.combo_sys.addItem(["cartesian", "corexy"])
self.combo_sys.setCurrentText(cfg.get("cnc", "system_type",
                                     default="cartesian"))
form.addRow("Tipo de sistema:", self.combo_sys)

```

```

# Grupo CoreXY (visível apenas quando selecionado)
self.grp_corexy = QGroupBox("Opções CoreXY")
g_core = QFormLayout(self.grp_corexy)
self.chk_inv_a = QCheckBox("Inverter Motor A")
self.chk_inv_b = QCheckBox("Inverter Motor B")
core_cfg = cfg.get("cnc", "corexy_config", default={})
self.chk_inv_a.setChecked(core_cfg.get("motor_a_invert", False))
self.chk_inv_b.setChecked(core_cfg.get("motor_b_invert", False))
g_core.addRow(self.chk_inv_a)
g_core.addRow(self.chk_inv_b)
form.addRow(self.grp_corexy)

```

```

# Mostrar/esconder grupo conforme seleção inicial
self.grp_corexy.setVisible(self.combo_sys.currentText() == "corexy")
self.combo_sys.currentTextChanged.connect(
    lambda txt: self.grp_corexy.setVisible(txt == "corexy")
)

```

```

# ----- botões -----
btn_box = QHBoxLayout()
btn_ok = QPushButton("Salvar")
btn_can = QPushButton("Cancelar")
btn_ok.clicked.connect(self._on_save)
btn_can.clicked.connect(self.reject)
btn_box.addWidget(btn_ok); btn_box.addWidget(btn_can)
form.addRow(btn_box)

```

```

def _on_save(self):
    self.cfg.set("cnc", "max_feed", "x", value=self.spin_f_x.value())
    self.cfg.set("cnc", "max_feed", "y", value=self.spin_f_y.value())
    self.cfg.set("cnc", "max_acc", "x", value=self.spin_a_x.value())
    self.cfg.set("cnc", "max_acc", "y", value=self.spin_a_y.value())
    self.cfg.set("cnc", "max_feed", "z", value=self.spin_f_z.value())
    self.cfg.set("cnc", "max_acc", "z", value=self.spin_a_z.value())
    self.cfg.set("cnc", "invert_y", value=self.chk_invert_y.isChecked())
    self.cfg.set("cnc", "invert_z", value=self.chk_invert_z.isChecked())
    self.cfg.set("cnc", "motor_hold_enabled", value=self.chk_motor_hold.isChecked())

    # ----- grava modo cartesiano/corexy -----
    sys_type = self.combo_sys.currentText()
    self.cfg.remember_system_type(sys_type)

    # ----- grava parâmetros CoreXY se aplicável -----
    if sys_type == "corexy":
        self.cfg.set("cnc", "corexy_config", "motor_a_invert",
                    value=self.chk_inv_a.isChecked())
        self.cfg.set("cnc", "corexy_config", "motor_b_invert",
                    value=self.chk_inv_b.isChecked())
    self.accept()

```

File: gcode_manager.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/aoi_lib

"""

Módulo para gerenciamento de G-CODE na aplicação de inspeção AOI.
 Permite criar, ler e manipular arquivos G-CODE para controle de percursos de inspeção.

"""

```

import os
import re
import time
import logging
from datetime import datetime

```

```

logger = logging.getLogger("GCodeManager")

```

```

class GCodeManager:

```

"""

Gerencia operações de G-CODE para o sistema AOI, incluindo:

- Geração de G-CODE a partir de posições de inspeção
- Leitura de arquivos G-CODE para criar sequências de inspeção
- Inclusão de comandos especiais para captura de imagens

"""

Definição de comandos especiais como comentários em G-CODE

```
CAPTURE_IMAGE_CMD = ";AOI_CAPTURE_IMAGE"
POSITION_NAME_CMD = ";AOI_POSITION_NAME:"
SEQUENCE_NAME_CMD = ";AOI_SEQUENCE_NAME:"
CAMERA_PARAMS_CMD = ";AOI_CAMERA_PARAMS:"
```

```
def __init__(self):
```

```
    """Inicializa o gerenciador de G-CODE"""
```

```
    self.feed_rate = 1000 # Taxa de avanço padrão (mm/min)
```

```
    self.safe_z_height = 5 # Altura Z para movimentos seguros
```

```
    self.use_z_axis = False # Se verdadeiro, inclui movimentos no eixo Z
```

```
def generate_gcode_from_sequence(self, sequence, include_header=True):
```

```
    """
```

```
    Gera G-CODE a partir de uma sequência de inspeção
```

```
    Args:
```

```
        sequence: Objeto InspectionSequence contendo as posições
```

```
        include_header: Se True, inclui cabeçalho com informações da sequência
```

```
    Returns:
```

```
        Uma string contendo o G-CODE completo
```

```
    """
```

```
    gcode_lines = []
```

```
    # Adiciona cabeçalho com informações
```

```
    if include_header:
```

```
        gcode_lines.extend([
            f"; AOI Inspection Sequence Generated G-CODE",
            f"; Date: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}",
            f"{self.SEQUENCE_NAME_CMD} {sequence.name}",
            f"; Total positions: {len(sequence.positions)}",
            f"; Feed rate: {self.feed_rate} mm/min",
            ";",
        ])
```

```
    # Configurações iniciais
```

```
    gcode_lines.extend([
        "G21 ; Set units to mm",
        "G90 ; Set absolute positioning",
        f"F{self.feed_rate} ; Set feed rate"
    ])
```

```
    # Para cada posição na sequência
```

```
    for i, position in enumerate(sequence.positions):
```

```
        # Adiciona comentário com o nome da posição
```

```
        gcode_lines.append(f"\n; Position {i+1}: {position.name}")
```

```
        gcode_lines.append(f"{self.POSITION_NAME_CMD} {position.name}")
```

```
    # Adiciona parâmetros da câmera se existirem
```

```
    if position.camera_params:
```

```
        params_str = ",".join([f"{k}:{v}" for k, v in position.camera_params.items()])
```

```
gcode_lines.append(f'{self.CAMERA_PARAMS_CMD} {params_str}')
```

```
# Adiciona movimento para a posição
```

```
gcode_lines.append(f'G0 X{position.x:.3f} Y{position.y:.3f} ; Move to position')
```

```
# Adiciona comando para captura de imagem
```

```
gcode_lines.append(f'{self.CAPTURE_IMAGE_CMD} ; Capture image at this position')
```

```
# Finaliza o programa
```

```
gcode_lines.append("\nM2 ; End program")
```

```
return "\n".join(gcode_lines)
```

```
def read_gcode_file(self, filename):
```

```
    """
```

Lê um arquivo G-CODE e cria uma sequência de posições de inspeção

Args:

filename: Caminho do arquivo G-CODE

Returns:

Um dicionário contendo:

- 'sequence_name': Nome da sequência extraída do arquivo

- 'positions': Lista de dicionários com informações das posições

```
    """
```

```
if not os.path.exists(filename):
```

```
    logger.error(f'Arquivo G-CODE não encontrado: {filename}')
```

```
    return None
```

```
try:
```

```
    with open(filename, 'r') as f:
```

```
        gcode_content = f.read()
```

```
    return self.parse_gcode(gcode_content)
```

```
except Exception as e:
```

```
    logger.error(f'Erro ao ler arquivo G-CODE: {str(e)}')
```

```
    return None
```

```
def parse_gcode(self, gcode_content):
```

```
    """
```

Analisa o conteúdo G-CODE e extrai sequência e posições

Args:

gcode_content: String contendo o G-CODE a ser analisado

Returns:

Dicionário com dados da sequência e posições

```
    """
```

```
    lines = gcode_content.split('\n')
```

```
    sequence_name = "Sequência Importada"
```

```

positions = []

current_position = None

# Extraí nome da sequência
for line in lines:
    if self.SEQUENCE_NAME_CMD in line:
        sequence_name = line.split(self.SEQUENCE_NAME_CMD, 1)[1].strip()
        break

# Extraí as posições
for line in lines:
    # Nova posição
    if self.POSITION_NAME_CMD in line:
        # Finaliza posição anterior se existir
        if current_position:
            positions.append(current_position)

        # Inicia nova posição
        position_name = line.split(self.POSITION_NAME_CMD, 1)[1].strip()
        current_position = {
            'name': position_name,
            'x': 0.0,
            'y': 0.0,
            'z': 0.0,
            'camera_params': {}
        }

# Parâmetros da câmera
elif self.CAMERA_PARAMS_CMD in line and current_position:
    params_str = line.split(self.CAMERA_PARAMS_CMD, 1)[1].strip()
    params_pairs = params_str.split(',')

    for pair in params_pairs:
        if ':' in pair:
            key, value = pair.split(':', 1)
            # Tenta converter para número se possível
            try:
                if '.' in value:
                    value = float(value)
                else:
                    value = int(value)
            except ValueError:
                pass # Mantém como string

            current_position['camera_params'][key] = value

# Movimento G0/G1
elif (line.strip().startswith('G0') or line.strip().startswith('G1')) and current_position:
    # Extraí coordenadas X e Y
    x_match = re.search(r'X([-\.d.]+)', line)

```

```

y_match = re.search(r'Y([-\\d.]+)', line)

if x_match:
    current_position['x'] = float(x_match.group(1))
if y_match:
    current_position['y'] = float(y_match.group(1))
# Z (opcional)
z_match = re.search(r'Z([-\\d.]+)', line)
if z_match:
    current_position['z'] = float(z_match.group(1))

# Adiciona a última posição se existir
if current_position:
    positions.append(current_position)

return {
    'sequence_name': sequence_name,
    'positions': positions
}

def save_gcode_to_file(self, sequence, filename):
    """
    Gera G-CODE a partir de uma sequência e salva em um arquivo

    Args:
        sequence: Objeto InspectionSequence contendo as posições
        filename: Caminho do arquivo onde o G-CODE será salvo

    Returns:
        Boolean indicando sucesso da operação
    """
    try:
        gcode_content = self.generate_gcode_from_sequence(sequence)

        with open(filename, 'w') as f:
            f.write(gcode_content)

        return True

    except Exception as e:
        logger.error(f'Erro ao salvar arquivo G-CODE: {str(e)}')
        return False

```

File: grbl_patched.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/aoi_lib

```

from grbl_streamer import GrblStreamer
import re, logging

```

```

log = logging.getLogger("GrblPatched")

```

```

class GrblPatched(GrblStreamer):
    """
    Subclasse que corrige o bug do _update_hash_state
    na presença da linha [HLP:...] emitida por Grbl 1.1g.
    """

    _RE_NUMERIC_TUPLE = re.compile(r'^-?\d+(\.\d+)?$')

    # Construtor: garante hash_state e referencia da serial
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        # Evita AttributeError no primeiro _onread()
        if not hasattr(self, "hash_state"):
            self.hash_state: dict[str, object] = {}
        # Nome real do atributo onde a lib guarda a porta serial
        self._serial_attrs = (
            "handle_serial", "ser", "_serial", "serial",
            "_GrblStreamer__serial"    # ← possível pelo name-mangling
        )
        # guarda handle descoberto (opcional, para acelerar próximas buscas)
        self._serial_cached = None

    # Procura dinamicamente o objeto pySerial dentro da instância
    def _find_serial_handle(self):
        # a) já temos cache válido?
        if self._serial_cached and getattr(self._serial_cached, "is_open", False):
            return self._serial_cached

        # b) busca pelos nomes "conhecidos"
        for attr in self._serial_attrs:
            ser = getattr(self, attr, None)
            if ser and getattr(ser, "write", None):
                self._serial_cached = ser
                return ser

        # c) procura qualquer atributo que pareça um Serial
        import serial as _pyserial
        for name, val in self.__dict__.items():
            if isinstance(val, _pyserial.Serial):
                self._serial_cached = val
                # anexa nome novo para acelerar da próxima vez
                if name not in self._serial_attrs:
                    self._serial_attrs += (name,)
                    log.debug("Novo atributo serial descoberto: %s", name)
                return val

        # d) procura dentro do _iface da grbl-streamer -----
        iface = getattr(self, "_iface", None)
        if iface:

```

```

# tenta iface.ser / iface.serial primeiro
for sub in ("ser", "serial", "_serial"):
    ssub = getattr(iface, sub, None)
    if isinstance(ssub, _pyserial.Serial):
        self._serial_cached = ssub
        log.debug("Serial encontrado em _iface.%s", sub)
        return ssub
# varre demais atributos do _iface
for subname, subval in iface.__dict__.items():
    if isinstance(subval, _pyserial.Serial):
        self._serial_cached = subval
        log.debug("Serial encontrado em _iface.%s", subname)
        return subval

```

```

# e) procura de forma genérica em objetos com atributo .ser
for objname, obj in self.__dict__.items():
    maybe_ser = getattr(obj, "ser", None)
    if isinstance(maybe_ser, _pyserial.Serial):
        self._serial_cached = maybe_ser
        log.debug("Serial descoberto em %s.ser", objname)
        return maybe_ser

```

```

# nenhum encontrado
return None

```

```

def _update_hash_state(self, line):
    """

```

Copia e corrige o método original. Linhas cujo prefixo não é numérico (ex.: HLP) são apenas armazenadas em self.hash_state_raw e ****não**** convertem para float().

```

    """

```

```

if not (line.startswith("[") and line.endswith("]")):
    return # não é hash-state

```

```

# Remove colchetes e separa chave / valores

```

```

inner = line[1:-1]      # "HLP:$ $ $ ..."

```

```

if ':' not in inner:

```

```

    return

```

```

key, value = inner.split(':', 1)

```

```

# Se o valor tem vírgulas tenta converter; caso contrário pula

```

```

if ',' in value:

```

```

    tpl_str = value.split(',')

```

```

    # CONVERTE APENAS SE TODOS OS ITENS SÃO NUMÉRICOS -----

```

```

    if all(self._RE_NUMERIC_TUPLE.fullmatch(v.strip()) for v in tpl_str):

```

```

        tpl = tuple(float(x) for x in tpl_str)

```

```

        self.hash_state[key] = tpl

```

```

    else:

```

```

        # mantém como string raw sem estourar excepción

```

```

        self.hash_state[key] = value

```

```

        log.debug("Ignorando conversão numérica de [%s]", inner)

```

else:

Mantém valor simples (ex.: PRB ou HLP)

self.hash_state[key] = value

Sobrescreve métodos de envio para preservar os ESPAÇOS

----- baixo nível -----

def _raw_write(self, cmd: str):

"""

Envia `cmd` exatamente como recebido, mantendo todos os espaços (necessário para eixos múltiplos: "Y-30.0 A-30.0").

"""

ser = self._find_serial_handle()

if ser is None:

Log detalhado para futura investigação

log.error("Handle serial não encontrado. Atributos disponíveis: %s",

" ".join(self.__dict__.keys()))

raise RuntimeError("Handle serial não encontrado em GrblStreamer")

if not cmd.endswith("\n"):

cmd += "\n"

ser.write(cmd.encode("ascii"))

ser.flush()

----- comandos em tempo-real / prioridade -----

def send_immediately(self, cmd: str):

"""

Override que envia o texto cru. Se falhar, volta para a implementação original.

"""

try:

self._raw_write(cmd)

self.last_sent = cmd

except Exception as exc:

log.warning("send_immediately RAW falhou (%s); fallback...", exc)

super().send_immediately(cmd)

----- fila normal -----

def send(self, cmd: str, *args, **kwargs):

"""

Mesmo comportamento de preservação de espaços para a fila padrão de comandos.

"""

try:

self._raw_write(cmd)

except Exception as exc:

log.warning("send RAW falhou (%s); fallback...", exc)

super().send(cmd, *args, **kwargs)

File: plc_axis_controller.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/aoi_lib

```
# plc_axis_controller.py
```

```
"""
```

Módulo independente para controle de eixos X, Y e Z via CLP (Modbus TCP).

Fornece movimentos absolutos, relativos, jog e homing para cada eixo.

```
"""
```

```
import time
```

```
import logging
```

```
from pymodbus.client import ModbusTcpClient
```

```
class PLCAxisController:
```

```
    """
```

Controller para 3 eixos (X, Y, Z) usando Modbus TCP.

Métodos principais:

connect()/close()

set_zero(axis)

move_absolute(axis, position, speed=None)

move_relative(axis, offset, speed=None)

jog_start(axis, direction, speed=None)

jog_stop(axis)

wait_for_idle(axis, tolerance=1, timeout=10)

read_position(axis)

home_axis(axis)

home_all()

```
    """
```

Mapeamento de memórias (coils) e registradores (holding)

ADDRESSES = {

'X': {

'zero': 1000, # M1000_X

'move_abs': 1050, # M1050_X

'pos_input': 1100, # D1100_X

'speed': 21000, # D21000_X

'jog_plus': 1070, # M1070_X

'jog_minus': 1080, # M1080_X

'jog_stop_plus': 1010, # M1010_X

'jog_stop_minus': 1011, # M1011_X

'pos_reg': 3000 # D3000_X

},

'Y': {

'zero': 500, # M500_Y

'move_abs': 550, # M550_Y

'pos_input': 600, # D600_Y

'speed': 20500, # D20500_Y

'jog_plus': 570, # M570_Y

'jog_minus': 580, # M580_Y

'jog_stop_plus': 510, # M510_Y

'jog_stop_minus': 511, # M511_Y

'pos_reg': 3200 # D3200_Y

},

```

'Z': {
    'zero':      1500, # M1500_Z
    'move_abs':  1550, # M1550_Z
    'pos_input': 1600, # D1600_Z
    'speed':     21500, # D21500_Z
    'jog_plus':  1570, # M1570_Z
    'jog_minus': 1580, # M1580_Z
    'jog_stop_plus': 1510, # M1510_Z
    'jog_stop_minus': 1511, # M1511_Z
    'pos_reg':   3400 # D3400_Z
}
}

```

```

def __init__(self, host: str='192.168.1.5', port: int=502):
    """Conecta ao CLP via Modbus TCP."""
    # guarda os parâmetros de conexão
    self.host = host
    self.port = port
    # flag para indicar se está online
    self.is_connected = False
    # estado inicial da máquina
    self.machine_status = "Disconnected"
    # cria o cliente Modbus e tenta conectar imediatamente
    self.client = ModbusTcpClient(host, port=port)
    # limites de feed (compatível com MovementControlWidget)
    self.max_feed = {'x': float('inf'),
                     'y': float('inf'),
                     'z': float('inf')}
    # fator de conversão pulses → mm
    self.pulses_per_mm = 1.0
    # tenta abrir a conexão; levanta erro se falhar
    connected = self.client.connect()
    if not connected:
        raise ConnectionError(f"Falha ao conectar ao CLP em {host}:{port}")
    self.is_connected = True
    self.machine_status = "Idle"

```

```

def close(self):
    """Fecha a conexão Modbus."""
    if self.client:
        self.client.close()
    # sinaliza para a aplicação que não está mais conectado
    self.is_connected = False

```

```

def disconnect(self):
    """
    Alias para fechar a conexão (compatibilidade com AOIControllerApp).
    """
    self.close()

```

```

def _write_dword(self, address: int, value: int):
    """
    Escreve valor INT32 (little-endian) em dois registradores
    de retenção (holding registers).
    """
    if not self.client:
        raise IOError("Cliente Modbus não inicializado")
    u32 = value & 0xFFFFFFFF
    lo = u32 & 0xFFFF
    hi = (u32 >> 16) & 0xFFFF
    return self.client.write_registers(address, [lo, hi])

def _read_dword(self, address: int) -> int:
    """
    Lê INT32 assinado de dois registradores de retenção.
    """
    if not self.client:
        raise IOError("Cliente Modbus não inicializado")
    res = self.client.read_holding_registers(address, count=2)
    if res.isError():
        raise IOError(f"Falha na leitura DWORD em {address}")
    lo, hi = res.registers
    u32 = (hi << 16) | lo
    return u32 if u32 < 0x80000000 else u32 - 0x100000000

def _pulse_coil(self, coil: int, duration_ms: int=20):
    """
    Aciona um coil por `duration_ms` milissegundos (borda de subida).
    """
    if not self.client:
        raise IOError("Cliente Modbus não inicializado")
    self.client.write_coil(coil, True)
    time.sleep(duration_ms/1000.0)
    self.client.write_coil(coil, False)

def set_zero(self, axis: str):
    """Zera a posição atual do eixo (memória de zero)."""
    mem = self.ADDRESSES[axis]['zero']
    self._pulse_coil(mem)

def move_absolute(self, axis: str, position: int, speed: int=None):
    """
    Move o eixo `axis` para posição absoluta (pulsos).
    Se `speed` for fornecido, grava no registrador de velocidade.
    """
    cfg = self.ADDRESSES[axis]
    self._write_dword(cfg['pos_input'], int(position))
    if speed is not None:
        self._write_dword(cfg['speed'], int(speed))
    self._pulse_coil(cfg['move_abs'])

```

```

def move_relative(self, x=None, y=None, z=None, feed_rate=1000):
    """
    Move de forma relativa em múltiplos eixos.
    Compatível com GRBLCNCController para uso em threads de medição.

    Args:
        x, y, z: Deslocamento relativo em mm (None para não mover o eixo)
        feed_rate: Velocidade em mm/min
    """
    # Converte feed_rate (mm/min) para pulsos/min
    speed_pulses = int(round(feed_rate * self.pulses_per_mm)) if feed_rate else None

    # Move cada eixo especificado de forma relativa
    if x is not None:
        x_pulses = int(round(x * self.pulses_per_mm))
        current_x = self._read_dword(self.ADDRESSES['X']['pos_reg'])
        self.move_absolute('X', current_x + x_pulses, speed_pulses)

    if y is not None:
        y_pulses = int(round(y * self.pulses_per_mm))
        current_y = self._read_dword(self.ADDRESSES['Y']['pos_reg'])
        self.move_absolute('Y', current_y + y_pulses, speed_pulses)

    if z is not None:
        z_pulses = int(round(z * self.pulses_per_mm))
        current_z = self._read_dword(self.ADDRESSES['Z']['pos_reg'])
        self.move_absolute('Z', current_z + z_pulses, speed_pulses)

    return True

def wait_for_idle(self, axis: str, tolerance: int=1, timeout: int=10) -> bool:
    """
    Aguarda até o eixo atingir o alvo  $\pm$  `tolerance` pulsos,
    retornando True se dentro de `timeout` segundos.
    """
    target = self._read_dword(self.ADDRESSES[axis]['pos_input'])
    t0 = time.time()
    while time.time() - t0 < timeout:
        cur = self._read_dword(self.ADDRESSES[axis]['pos_reg'])
        if abs(cur - target) <= tolerance:
            return True
        time.sleep(0.05)
    return False

def step_move(self, axis: str, distance_mm: float, feed_rate: float = None) -> bool:
    """
    Move um passo no eixo especificado:
    - distance_mm: deslocamento em mm (positivo ou negativo)
    - feed_rate: velocidade em mm/min (opcional)
    Converte mm → pulsos usando pulses_per_mm, faz move_relative + wait_for_idle.
    """

```

Retorna True se o eixo atingir o destino, False em timeout.

"""

1) converte milímetros em pulsos

pulses = int(round(distance_mm * self.pulses_per_mm))

2) converte feed_rate (mm/min) em pulsos/min, se fornecido

speed = int(round(feed_rate * self.pulses_per_mm)) if feed_rate is not None else None

3) executa movimento relativo usando o método antigo interno

self._move_relative_single_axis(axis, pulses, speed)

4) aguarda até o eixo estar idle

return self._wait_for_idle_axis(axis)

def _move_relative_single_axis(self, axis: str, offset: int, speed: int=None):

"""

Move um único eixo de forma relativa (método interno).

"""

current = self._read_dword(self.ADDRESSES[axis]['pos_reg'])

self.move_absolute(axis, current + offset, speed)

def jog_start(self, axis: str, direction: int, feed_rate: float = None):

"""

Inicia jog contínuo no eixo.

Args:

axis: Nome do eixo ('X', 'Y' ou 'Z')

direction: Direção do movimento (+1 ou -1)

feed_rate: Velocidade em mm/min (será convertida para pulsos/min)

"""

Normaliza nome do eixo para maiúscula

axis = axis.upper()

if axis not in self.ADDRESSES:

raise ValueError(f"Eixo inválido: {axis}")

cfg = self.ADDRESSES[axis]

Converte feed_rate (mm/min) para pulsos/min se fornecido

if feed_rate is not None:

speed_pulses = int(round(feed_rate * self.pulses_per_mm))

self._write_dword(cfg['speed'], speed_pulses)

Aciona o coil apropriado baseado na direção

coil = cfg['jog_plus'] if direction > 0 else cfg['jog_minus']

self.client.write_coil(coil, True)

def jog_stop(self, axis: str = None):

"""

Para o jog contínuo de todos os eixos.

O parâmetro axis é ignorado para manter compatibilidade com a UI.

"""

Para todos os eixos (a UI não especifica qual)

alvos = [axis] if axis else list(self.ADDRESSES.keys())

for ax in alvos:

cfg = self.ADDRESSES[ax]

self.client.write_coil(cfg['jog_plus'], False)

self.client.write_coil(cfg['jog_minus'], False)

```

def move_to_absolute_position(self, x=None, y=None, z=None, feed_rate=1000):
    """
    Move para uma posição absoluta em coordenadas.
    Compatível com GRBLCNCController para uso em threads de medição.

    Args:
        x, y, z: Coordenadas de destino em mm (None para não mover o eixo)
        feed_rate: Velocidade em mm/min
    """
    # Converte feed_rate (mm/min) para pulsos/min
    speed_pulses = int(round(feed_rate * self.pulses_per_mm)) if feed_rate else None

    # Move cada eixo que foi especificado
    if x is not None:
        x_pulses = int(round(x * self.pulses_per_mm))
        self.move_absolute('X', x_pulses, speed_pulses)

    if y is not None:
        y_pulses = int(round(y * self.pulses_per_mm))
        self.move_absolute('Y', y_pulses, speed_pulses)

    if z is not None:
        z_pulses = int(round(z * self.pulses_per_mm))
        self.move_absolute('Z', z_pulses, speed_pulses)

    return True

def wait_for_idle(self, axis=None, tolerance: int=1, timeout: int=10):
    """
    Aguarda até que os eixos fiquem idle.
    - Se axis for uma string ('X','Y','Z'), aguarda apenas esse eixo.
    - Se axis for None, aguarda todos os eixos (X, Y, Z) sequencialmente.
    tolerance: tolerância em pulsos.
    timeout: tempo máximo de espera em segundos.
    """
    # Se pediram eixo específico, despacha direto
    if isinstance(axis, str):
        return self._wait_for_idle_axis(axis, tolerance, timeout)
    # Senão espera todos os eixos em série
    for ax in ('X', 'Y', 'Z'):
        if not self._wait_for_idle_axis(ax, tolerance, timeout):
            return False
    return True

def _wait_for_idle_axis(self, axis: str, tolerance: int=1, timeout: int=10) -> bool:
    """
    Aguarda até um eixo específico atingir o alvo.
    (Renomeado do antigo wait_for_idle para evitar conflito)
    """
    target = self._read_dword(self.ADDRESSES[axis]['pos_input'])

```

```

t0 = time.time()
while time.time() - t0 < timeout:
    cur = self._read_dword(self.ADDRESSES[axis]['pos_reg'])
    if abs(cur - target) <= tolerance:
        return True
    time.sleep(0.05)
return False

```

```

def read_position(self, axis: str) -> int:
    """Lê a posição atual do eixo em pulsos."""
    return self._read_dword(self.ADDRESSES[axis]['pos_reg'])

```

```

def get_current_position(self) -> dict:
    """
    Retorna a posição atual de X, Y e Z em milímetros.
    Assume 1 pulso = 1 mm por padrão; se necessário ajuste em pulses_per_mm.
    """
    try:
        xp = self.read_position('X')
        yp = self.read_position('Y')
        zp = self.read_position('Z')
        return {
            'x': xp / self.pulses_per_mm,
            'y': yp / self.pulses_per_mm,
            'z': zp / self.pulses_per_mm
        }
    except Exception as e:
        raise IOError(f"Falha ao obter posição atual via PLC: {e}")

```

```

def home_axis(self, axis: str):
    """
    Executa homing do eixo individual (usa mesma memória de zero).
    """
    self._pulse_coil(self.ADDRESSES[axis]['zero'])

```

```

def home_all(self):
    """
    Sequência de homing completa: Z primeiro, depois X e Y.
    """
    # homing Z
    self._pulse_coil(self.ADDRESSES['Z']['zero'])
    time.sleep(3.0)
    # homing X e Y em paralelo
    self._pulse_coil(self.ADDRESSES['X']['zero'])
    self._pulse_coil(self.ADDRESSES['Y']['zero'])

```

```

if __name__ == "__main__":
    # Exemplo de uso rápido
    plc = PLCAxisController("192.168.1.5", 502)
    try:

```

```

plc.home_all()
plc.move_absolute("X", 10000)
if plc.wait_for_idle("X"):
    print("X chegou ao destino")
plc.jog_start("Y", +1, speed=1500)
time.sleep(1.0)
plc.jog_stop("Y")
finally:
    plc.close()

```

File: position_manager.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/aoi_lib

import time

class InspectionPosition:

"""Representa uma posição de inspeção."""

def __init__(self, name, x, y, z=0.0, camera_params=None):

self.name = name

self.x = x

self.y = y

self.z = z

self.camera_params = camera_params or {}

self.creation_time = time.time()

def to_dict(self):

"""Converte para dicionário para serialização."""

return {

 'name': self.name,

 'x': self.x,

 'y': self.y,

 'z': self.z,

 'camera_params': self.camera_params,

 'creation_time': self.creation_time

}

@classmethod

def from_dict(cls, data):

"""Cria a partir de um dicionário."""

position = cls(

 data['name'],

 data.get('x', 0.0),

 data.get('y', 0.0),

 data.get('z', 0.0),

 data.get('camera_params', {})

)

position.creation_time = data.get('creation_time', time.time())

return position

```

class InspectionSequence:
    """Representa uma sequência de posições de inspeção."""

    def __init__(self, name):
        self.name = name
        self.positions = []
        self.creation_time = time.time()
        self.last_run_time = None

    def add_position(self, position):
        """Adiciona uma posição à sequência."""
        self.positions.append(position)

    def remove_position(self, index):
        """Remove uma posição pelo índice."""
        if 0 <= index < len(self.positions):
            del self.positions[index]
            return True
        return False

    def move_position(self, old_index, new_index):
        """Move uma posição na sequência."""
        if 0 <= old_index < len(self.positions) and 0 <= new_index < len(self.positions):
            position = self.positions.pop(old_index)
            self.positions.insert(new_index, position)
            return True
        return False

    def to_dict(self):
        """Converte para dicionário para serialização."""
        return {
            'name': self.name,
            'positions': [p.to_dict() for p in self.positions],
            'creation_time': self.creation_time,
            'last_run_time': self.last_run_time
        }

    @classmethod
    def from_dict(cls, data, positions_dict=None):
        """Cria a partir de um dicionário."""
        sequence = cls(data['name'])
        sequence.creation_time = data.get('creation_time', time.time())
        sequence.last_run_time = data.get('last_run_time')

        # Reconstruir posições
        for pos_data in data['positions']:
            if positions_dict and pos_data['name'] in positions_dict:
                # Usa posição existente
                sequence.positions.append(positions_dict[pos_data['name']])

```

```
else:
    # Cria nova posição
    position = InspectionPosition.from_dict(pos_data)
    sequence.positions.append(position)
```

```
return sequence
```

```
class InspectionPositionManager:
```

```
    """Gerencia posições e sequências de inspeção."""
```

```
    def __init__(self):
```

```
        self.positions = {} # name -> position
```

```
        self.sequences = {} # name -> sequence
```

```
    def add_position(self, position):
```

```
        """Adiciona uma posição."""
```

```
        self.positions[position.name] = position
```

```
    def remove_position(self, position_name):
```

```
        """Remove uma posição pelo nome."""
```

```
        if position_name in self.positions:
```

```
            del self.positions[position_name]
```

```
        # Atualiza sequências que usam esta posição
```

```
        for sequence in self.sequences.values():
```

```
            sequence.positions = [p for p in sequence.positions if p.name != position_name]
```

```
        return True
```

```
        return False
```

```
    def get_position(self, position_name):
```

```
        """Obtém uma posição pelo nome."""
```

```
        return self.positions.get(position_name)
```

```
    def add_sequence(self, sequence):
```

```
        """Adiciona uma sequência."""
```

```
        self.sequences[sequence.name] = sequence
```

```
    def remove_sequence(self, sequence_name):
```

```
        """Remove uma sequência pelo nome."""
```

```
        if sequence_name in self.sequences:
```

```
            del self.sequences[sequence_name]
```

```
            return True
```

```
        return False
```

```
    def get_sequence(self, sequence_name):
```

```
        """Obtém uma sequência pelo nome."""
```

```
        return self.sequences.get(sequence_name)
```

```
    def save_to_file(self, filename):
```

```

"""Salva todas as posições e sequências em um arquivo JSON."""
data = {
    'positions': {name: pos.to_dict() for name, pos in self.positions.items()},
    'sequences': {name: seq.to_dict() for name, seq in self.sequences.items()}
}

try:
    with open(filename, 'w') as f:
        import json
        json.dump(data, f, indent=2)
    return True
except Exception as e:
    print(f"Erro ao salvar dados: {e}")
    return False

def load_from_file(self, filename):
    """Carrega posições e sequências de um arquivo JSON."""
    try:
        with open(filename, 'r') as f:
            import json
            data = json.load(f)

        # Limpa dados existentes
        self.positions.clear()
        self.sequences.clear()

        # Carrega posições
        for name, pos_data in data.get('positions', {}).items():
            position = InspectionPosition.from_dict(pos_data)
            self.positions[name] = position

        # Carrega sequências
        for name, seq_data in data.get('sequences', {}).items():
            sequence = InspectionSequence.from_dict(seq_data, self.positions)
            self.sequences[name] = sequence

    return True

except Exception as e:
    print(f"Erro ao carregar dados: {e}")
    return False

```

File: stencil_tension.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/aoi_lib

"""

Módulo: stencil_tension.py

Descrição: mede a tensão do stencil em um grid NxN e salva em JSON.

"""

```

import json
import serial
import serial.tools.list_ports
import numpy as np
import logging
from tkinter import Toplevel, Label, Entry, Button, messagebox
from PyQt6.QtWidgets import (
    QDialog, QGridLayout, QLabel, QLineEdit, QPushButton, QMessageBox,
    QComboBox, QGroupBox, QFrame, QHBoxLayout, QProgressBar
)
from PyQt6.QtCore import QThread, pyqtSignal
import time, logging

# Configure logging
log = logging.getLogger(__name__)

# ===== Gerenciador de Conexão Serial para Tensiômetro =====
class TensiometerSerialManager:
    """
    Gerencia conexão serial dedicada para o tensiômetro.
    Permite configurar porta, baudrate e parâmetros de comunicação.
    """

    def __init__(self):
        self.serial_connection = None
        self.is_connected = False
        self.port = None
        self.baudrate = 2400 # Padrão para tensiômetros AS-120N
        self.timeout = 1.0
        self.last_error = ""
        # Parâmetros específicos do AS-120N
        self.REQ_COMMAND = b'\x20' # Comando de requisição
        self.FRAME_LEN = 9 # Tamanho do frame de resposta
        self.UNIT_MAP = {0x05: "N/cm²", 0x04: "kg/cm²", 0x06: "lb/cm²"}

    def _real_dig(self, b: int) -> int:
        """Converte nibble baixo deslocado (+0x0A) em dígito 0–9."""
        return ((b & 0x0F) + 10) % 10

    def _decode_frame(self, frame: bytes) -> float | None:
        """
        Decodifica um frame de 9 bytes do AS-120N.
        Retorna o valor em float (N/cm²).
        """
        if len(frame) < self.FRAME_LEN or frame[0] != 0x10 or frame[2] != 0x19:
            return None
        casas = (frame[3] >> 4) & 0x07
        d3 = self._real_dig(frame[6])
        d2 = self._real_dig(frame[7])
        d1 = self._real_dig(frame[8])
        raw = d3 * 100 + d2 * 10 + d1

```

```
return raw / (10 ** casas)
```

```
def get_available_ports(self):
```

```
    """Retorna lista de portas seriais disponíveis"""
```

```
    try:
```

```
        ports = [port.device for port in serial.tools.list_ports.comports()]
```

```
        return ports
```

```
    except Exception as e:
```

```
        log.error(f"Erro ao listar portas: {e}")
```

```
    return []
```

```
def connect(self, port, baudrate=2400, timeout=1.0):
```

```
    """
```

```
    Conecta ao tensiômetro em uma porta específica
```

```
    Args:
```

```
        port: Porta COM (ex: 'COM3', '/dev/ttyUSB0')
```

```
        baudrate: Taxa de transmissão (padrão: 2400)
```

```
        timeout: Timeout para leitura em segundos
```

```
    """
```

```
    try:
```

```
        # Desconecta se já estiver conectado
```

```
        if self.is_connected:
```

```
            self.disconnect()
```

```
        # Cria nova conexão
```

```
        self.serial_connection = serial.Serial(
```

```
            port=port,
```

```
            baudrate=baudrate,
```

```
            bytesize=serial.EIGHTBITS,
```

```
            parity=serial.PARITY_NONE,
```

```
            stopbits=serial.STOPBITS_ONE,
```

```
            timeout=timeout
```

```
        )
```

```
        # Configuração específica para AS-120N
```

```
        self.serial_connection.dtr = False
```

```
        self.serial_connection.rts = False
```

```
        self.port = port
```

```
        self.baudrate = baudrate
```

```
        self.timeout = timeout
```

```
        self.is_connected = True
```

```
        self.last_error = ""
```

```
        log.info(f"Tensiômetro conectado em {port} @ {baudrate} baud")
```

```
        return True
```

```
    except Exception as e:
```

```
        self.last_error = str(e)
```

```
        log.error(f"Erro ao conectar tensiômetro: {e}")
```

```
return False
```

```
def disconnect(self):
```

```
    """Desconecta do tensiômetro"""
```

```
    try:
```

```
        if self.serial_connection and self.serial_connection.is_open:
```

```
            self.serial_connection.close()
```

```
            log.info("Tensiômetro desconectado")
```

```
    except Exception as e:
```

```
        log.error(f"Erro ao desconectar tensiômetro: {e}")
```

```
    finally:
```

```
        self.serial_connection = None
```

```
        self.is_connected = False
```

```
        self.port = None
```

```
def read_tension_value(self):
```

```
    """
```

```
    Lê valor do tensiômetro AS-120N usando protocolo binário
```

```
    Returns:
```

```
        str: Valor formatado ou "0" em caso de erro
```

```
    """
```

```
    if not self.is_connected or not self.serial_connection:
```

```
        self.last_error = "Tensiômetro não conectado"
```

```
        return "0"
```

```
    try:
```

```
        # Limpa buffer de entrada
```

```
        self.serial_connection.reset_input_buffer()
```

```
        # Envia comando de requisição
```

```
        self.serial_connection.write(self.REQ_COMMAND)
```

```
        self.serial_connection.flush()
```

```
        # Aguarda resposta
```

```
        import time
```

```
        time.sleep(0.05) # 50ms conforme exemplo
```

```
        # Lê frame de resposta (9 bytes exatos)
```

```
        raw_data = self.serial_connection.read(self.FRAME_LEN)
```

```
    if not raw_data:
```

```
        self.last_error = "Sem resposta do tensiômetro"
```

```
        return "0"
```

```
    if len(raw_data) != self.FRAME_LEN:
```

```
        self.last_error = f"Frame incompleto: {len(raw_data)} bytes (esperado: {self.FRAME_LEN})"
```

```
        return "0"
```

```
        # Decodifica o frame binário
```

```
        value = self._decode_frame(raw_data)
```

```

if value is None:
    self.last_error = f"Frame inválido: {raw_data.hex(' ')}"
    return "0"

```

```

# Retorna valor formatado
return f"{value:.2f}"

```

```

except Exception as e:
    self.last_error = str(e)
    log.error(f"Erro ao ler tensiômetro: {e}")
    return "0"

```

```

def send_command(self, command):

```

```

    """

```

Envia comando para o tensiômetro (se suportado)

Args:

command: Comando a ser enviado

```

    """

```

```

if not self.is_connected or not self.serial_connection:
    return False

```

try:

```

    cmd_bytes = (command + '\r\n').encode('ascii')
    self.serial_connection.write(cmd_bytes)
    self.serial_connection.flush()
    return True

```

```

except Exception as e:

```

```

    self.last_error = str(e)
    log.error(f"Erro ao enviar comando: {e}")
    return False

```

===== Thread para Medição de Tensão =====

```

class TensionMeasurementThread(QThread):

```

```

    """

```

Thread responsável por executar a medição de tensão em background, permitindo que a interface continue responsiva.

```

    """

```

Sinais para comunicação com a interface

progress_updated = pyqtSignal(int, int, str) # ponto_atual, total_pontos, status_msg

measurement_completed = pyqtSignal(dict) # ponto medido com dados

finished = pyqtSignal(list) # lista completa de medições

error_occurred = pyqtSignal(str) # mensagem de erro

```

def __init__(self, cnc_controller, tensiometer, points, z_down,
              user_feed, stabilization_time, parameters):

```

```

    super().__init__()

```

```

    self.cnc = cnc_controller

```

```

    self.tensiometer = tensiometer

```

```
self.points = points
self.z_down = z_down
self.user_feed = user_feed
self.stabilization_time = stabilization_time
self.parameters = parameters
self.measurements = []
self._stop_requested = False
```

```
def request_stop(self):
    """Solicita parada da medição"""
    self._stop_requested = True
```

```
def run(self):
    """Executa o processo de medição"""
    try:
        log = logging.getLogger("TensionMeasurementThread")
        log.info("Iniciando medição de tensão em thread separada")

        # Garante modo absoluto
        if hasattr(self.cnc, "set_absolute_mode"):
            self.cnc.set_absolute_mode()
        else:
            if hasattr(self.cnc, "send_raw_gcode"):
                self.cnc.send_raw_gcode("G90")

        # Move para altura segura
        self._move_abs(z=0, feed=self.user_feed)

        total_points = len(self.points)

        for idx, (x, y) in enumerate(self.points, 1):
            # Verifica se foi solicitada a parada
            if self._stop_requested:
                log.info("Medição interrompida pelo usuário")
                self.error_occurred.emit("Medição interrompida pelo usuário")
                return

            log.debug("Ponto %d de %d -> X%.3f Y%.3f", idx, total_points, x, y)

            # Emite progresso
            self.progress_updated.emit(idx, total_points, f"Medindo ponto {idx}/{total_points}")

            # 1) Move XY
            self._move_abs(x=x, y=y, feed=self.user_feed)

            # 2) Desce Z
            self._move_rel(z=self.z_down, feed=self.user_feed)

            # 3) Aguarda estabilização
            stabilization_sec = self.stabilization_time / 1000.0
            log.debug("Aguardando estabilização por %.1fs...", stabilization_sec)
```

```
time.sleep(stabilization_sec)
```

```
# 4) Lê tensão
```

```
tension = self.tensiometer.read_tension_value()
```

```
log.debug("Tensão medida no ponto %d: %s", idx, tension)
```

```
# 5) Salva medição
```

```
measurement = {"x": x, "y": y, "z": self.z_down, "tension": tension}
```

```
self.measurements.append(measurement)
```

```
# Emite medição individual
```

```
self.measurement_completed.emit(measurement)
```

```
# 6) Sobe Z
```

```
self._move_rel(z=-self.z_down, feed=self.user_feed)
```

```
# 7) Pequena pausa entre pontos
```

```
time.sleep(0.1)
```

```
# Emite resultado final
```

```
log.info("Medição de tensão concluída com sucesso")
```

```
self.finished.emit(self.measurements)
```

```
except Exception as e:
```

```
log.error(f"Erro durante medição: {e}", exc_info=True)
```

```
self.error_occurred.emit(f"Erro durante medição: {str(e)}")
```

```
def _move_abs(self, *, x=None, y=None, z=None, feed=None):
```

```
    """Move em coordenadas absolutas"""
```

```
    if feed is not None:
```

```
        self.cnc.move_to_absolute_position(x=x, y=y, z=z, feed_rate=feed)
```

```
    else:
```

```
        self.cnc.move_to_absolute_position(x=x, y=y, z=z)
```

```
    self.cnc.wait_for_idle()
```

```
def _move_rel(self, *, x=None, y=None, z=None, feed=None):
```

```
    """Move em coordenadas relativas"""
```

```
    kwargs = {}
```

```
    if x is not None: kwargs['x'] = x
```

```
    if y is not None: kwargs['y'] = y
```

```
    if z is not None: kwargs['z'] = z
```

```
    if feed is not None: kwargs['feed_rate'] = feed
```

```
    self.cnc.move_relative(**kwargs)
```

```
    self.cnc.wait_for_idle()
```

```
# ===== Classe principal =====
```

```
class StencilTensionMeasurement:
```

```
    """
```

```
    Abre uma janela para o usuário inserir:
```

- ponto inicial (X,Y)

- ponto final (X,Y)
- quantidade N (grid NxN)
- altura Z

Movimenta o CNC, lê o tensiômetro e grava JSON.

.....

```
def __init__(self, master, serial_handler):
    self.master = master
    self.serial_handler = serial_handler    # já existente na aplicação
    self.window = None

# ----- GUI -----
def show_window(self):
    self.window = Toplevel(self.master)
    self.window.title("Tensão do Stencil")
    self.window.resizable(False, False)

# ---- Entradas ----
Label(self.window, text="Ponto Inicial").grid(row=0, column=0, columnspan=4, pady=(5, 0))
Label(self.window, text="X:").grid(row=1, column=0)
self.start_x = Entry(self.window, width=8); self.start_x.grid(row=1, column=1)
Label(self.window, text="Y:").grid(row=1, column=2)
self.start_y = Entry(self.window, width=8); self.start_y.grid(row=1, column=3)

Label(self.window, text="Ponto Final").grid(row=2, column=0, columnspan=4, pady=(10, 0))
Label(self.window, text="X:").grid(row=3, column=0)
self.end_x = Entry(self.window, width=8); self.end_x.grid(row=3, column=1)
Label(self.window, text="Y:").grid(row=3, column=2)
self.end_y = Entry(self.window, width=8); self.end_y.grid(row=3, column=3)

Label(self.window, text="Quantidade (N):").grid(row=4, column=0, pady=(10, 0))
self.quantity = Entry(self.window, width=8); self.quantity.grid(row=4, column=1)

Label(self.window, text="Altura Z:").grid(row=4, column=2, pady=(10, 0))
self.height = Entry(self.window, width=8); self.height.grid(row=4, column=3)

Button(self.window, text="Iniciar Medição", command=self.start_measurement)\
    .grid(row=5, column=0, columnspan=4, pady=15)

# ----- Cálculo do grid -----
def calculate_grid_points(self):
    try:
        sx, sy = float(self.start_x.get()), float(self.start_y.get())
        ex, ey = float(self.end_x.get()), float(self.end_y.get())
        n = int(self.quantity.get())

        if n < 2:
            raise ValueError

        x_points = np.linspace(sx, ex, n)
        y_points = np.linspace(sy, ey, n)
```

```

        return [(float(x), float(y)) for x in x_points for y in y_points]
    except ValueError:
        messagebox.showerror("Erro", "Preencha todos os campos corretamente.\n"
                               "Quantidade deve ser um inteiro >= 2.")
    return None

# ----- Loop de medição -----
def start_measurement(self):
    points = self.calculate_grid_points()
    if points is None:
        return

    try:
        z = float(self.height.get())
    except ValueError:
        messagebox.showerror("Erro", "Altura Z inválida.")
        return

    measurements = []
    for x, y in points:
        # Move XY
        self.serial_handler.send_command(f"G0 X{x:.3f} Y{y:.3f}")
        # Baixa Z
        self.serial_handler.send_command(f"G1 Z{z:.3f}")

        # ---- Lê tensiômetro ----
        tension = self.read_tensiometer_value()
        measurements.append({"x": x, "y": y, "z": z, "tension": tension})

        # Sobe Z (evita arrastar o sensor)
        self.serial_handler.send_command("G1 Z0")

    self.save_json(measurements)
    messagebox.showinfo("Concluído", "Medições finalizadas e arquivo salvo.")
    self.window.destroy()

# ----- Leitura do tensiômetro -----
def read_tensiometer_value(self):
    """
    Exemplo de leitura (AS-120N a 2400 baud, 8N1).
    Adapte se seu serial_handler já contiver método apropriado.
    """
    ser: serial.Serial = self.serial_handler.serial # porta já aberta pelo app
    raw = ser.readline().decode(errors="ignore").strip()
    return raw or "0"

# ----- Salva resultado -----
def save_json(self, measurements):
    data = {
        "type": "stencil_tension",
        "parameters": {

```

```

        "start": {"x": float(self.start_x.get()), "y": float(self.start_y.get())},
        "end": {"x": float(self.end_x.get()), "y": float(self.end_y.get())},
        "quantity": int(self.quantity.get()),
        "height": float(self.height.get())
    },
    "measurements": measurements
}
with open("stencil_tension_measurements.json", "w", encoding="utf-8") as f:
    json.dump(data, f, indent=4, ensure_ascii=False)

```

```

# -----
# NOVA VERSÃO PyQt6 – compatível com AOIControllerApp (consumo_lib.py)
# -----

```

```

class StencilTensionDialog(QDialog):

```

```

    """

```

```

    Diálogo PyQt6 para medir tensão do stencil em um grid NxN.
    Usa cntrl_cnc (GRBLCNCController) para movimentação.
    """

```

```

    """

```

```

    def __init__(self, parent, cntrl_cnc):
        super().__init__(parent)
        self.setWindowTitle("Tensão do Stencil")
        self.cnc = cntrl_cnc
        # Inicializa gerenciador do tensiômetro
        self.tensiometer = TensiometerSerialManager()
        self.measurement_thread = None
        self._build_ui()
        # Popula lista de portas disponíveis
        self._refresh_ports()

```

```

# ----- UI -----

```

```

    def _build_ui(self):
        lay = QGridLayout(self)
        r = 0
        # Grupo de Conexão do Tensiômetro
        conn_group = QGroupBox("Conexão do Tensiômetro")
        conn_layout = QGridLayout(conn_group)

        # Porta Serial
        conn_layout.addWidget(QLabel("Porta:"), 0, 0)
        self.port_combo = QComboBox()
        conn_layout.addWidget(self.port_combo, 0, 1)

        # Baudrate
        conn_layout.addWidget(QLabel("Baudrate:"), 0, 2)
        self.baudrate_combo = QComboBox()
        self.baudrate_combo.addItem("2400", "9600", "19200", "38400", "57600", "115200")
        self.baudrate_combo.setCurrentText("2400")
        conn_layout.addWidget(self.baudrate_combo, 0, 3)

```

```

# Botões de conexão
self.refresh_ports_btn = QPushButton("Atualizar")
self.refresh_ports_btn.clicked.connect(self._refresh_ports)
conn_layout.addWidget(self.refresh_ports_btn, 1, 0)

self.connect_btn = QPushButton("Conectar")
self.connect_btn.clicked.connect(self._toggle_connection)
conn_layout.addWidget(self.connect_btn, 1, 1)

self.test_btn = QPushButton("Testar Leitura")
self.test_btn.clicked.connect(self._test_reading)
self.test_btn.setEnabled(False)
conn_layout.addWidget(self.test_btn, 1, 2)

# Status da conexão
self.connection_status = QLabel("Desconectado")
self.connection_status.setStyleSheet("color: red; font-weight: bold;")
conn_layout.addWidget(self.connection_status, 1, 3)

lay.addWidget(conn_group, r, 0, 1, 5); r += 1

# Separador
line = QFrame()
line.setFrameShape(QFrame.Shape.HLine)
line.setFrameShadow(QFrame.Shadow.Sunken)
lay.addWidget(line, r, 0, 1, 5); r += 1

# Grupo de Configurações Avançadas
adv_group = QGroupBox("Configurações de Medição")
adv_layout = QGridLayout(adv_group)

adv_layout.addWidget(QLabel("Tempo de estabilização (ms):"), 0, 0)
self.stabilization_time = QLineEdit("500")
self.stabilization_time.setToolTip("Tempo de espera após descida do Z antes da leitura")
adv_layout.addWidget(self.stabilization_time, 0, 1)

# Velocidade de movimentação configurável
adv_layout.addWidget(QLabel("Velocidade de movimentação (mm/min):"), 1, 0)
self.movement_feed = QLineEdit("30")
self.movement_feed.setToolTip("Velocidade para todas as movimentações durante a medição")
adv_layout.addWidget(self.movement_feed, 1, 1)

adv_group.setLayout(adv_layout)
lay.addWidget(adv_group, r, 0, 1, 5); r += 1

lay.addWidget(QLabel("<b>Ponto Inicial</b>"), r, 0, 1, 5); r += 1

lay.addWidget(QLabel("X:"), r, 0)
self.ed_sx = QLineEdit(); lay.addWidget(self.ed_sx, r, 1)
lay.addWidget(QLabel("Y:"), r, 2)
self.ed_sy = QLineEdit(); lay.addWidget(self.ed_sy, r, 3)

```

```
btn_cap_start = QPushButton("Capturar"); lay.addWidget(btn_cap_start, r, 4); r += 1
```

```
lay.addWidget(QLabel("<b>Ponto Final</b>"), r, 0, 1, 5); r += 1
```

```
lay.addWidget(QLabel("X:"), r, 0)
```

```
self.ed_ex = QLineEdit(); lay.addWidget(self.ed_ex, r, 1)
```

```
lay.addWidget(QLabel("Y:"), r, 2)
```

```
self.ed_ey = QLineEdit(); lay.addWidget(self.ed_ey, r, 3)
```

```
btn_cap_end = QPushButton("Capturar"); lay.addWidget(btn_cap_end, r, 4); r += 1
```

```
lay.addWidget(QLabel("Quantidade (N):"), r, 0)
```

```
self.ed_n = QLineEdit("3"); lay.addWidget(self.ed_n, r, 1)
```

```
lay.addWidget(QLabel("Altura Z (mm):"), r, 2)
```

```
self.ed_z = QLineEdit("2"); lay.addWidget(self.ed_z, r, 3)
```

```
btn_cap_z = QPushButton("Capturar"); lay.addWidget(btn_cap_z, r, 4); r += 1
```

```
# Botões de controle da medição
```

```
measurement_buttons_layout = QHBoxLayout()
```

```
self.start_measurement_btn = QPushButton("Iniciar Medição")
```

```
self.stop_measurement_btn = QPushButton("Parar Medição")
```

```
self.stop_measurement_btn.setEnabled(False)
```

```
measurement_buttons_layout.addWidget(self.start_measurement_btn)
```

```
measurement_buttons_layout.addWidget(self.stop_measurement_btn)
```

```
lay.addLayout(measurement_buttons_layout, r, 0, 1, 5); r += 1
```

```
# Barra de progresso
```

```
self.progress_bar = QProgressBar()
```

```
self.progress_bar.setVisible(False)
```

```
lay.addWidget(self.progress_bar, r, 0, 1, 5); r += 1
```

```
# Labels de status
```

```
self.status_label = QLabel("")
```

```
lay.addWidget(self.status_label, r, 0, 1, 5); r += 1
```

```
# Conexões -----
```

```
self.start_measurement_btn.clicked.connect(self._on_start)
```

```
self.stop_measurement_btn.clicked.connect(self._on_stop)
```

```
btn_cap_start.clicked.connect(self._capture_start_xy)
```

```
btn_cap_end.clicked.connect(self._capture_end_xy)
```

```
btn_cap_z.clicked.connect(self._capture_z_height)
```

```
# ----- Conexão do Tensiômetro -----
```

```
def _refresh_ports(self):
```

```
    """Atualiza lista de portas disponíveis"""
```

```
    self.port_combo.clear()
```

```
    ports = self.tensiometer.get_available_ports()
```

```
    if ports:
```

```
        self.port_combo.addItem(ports)
```

```
    else:
```

```

self.port_combo.addItem("Nenhuma porta encontrada")

def _toggle_connection(self):
    """Conecta/desconecta do tensiômetro"""
    if self.tensiometer.is_connected:
        # Desconectar
        self.tensiometer.disconnect()
        self.connect_btn.setText("Conectar")
        self.test_btn.setEnabled(False)
        self.connection_status.setText("Desconectado")
        self.connection_status.setStyleSheet("color: red; font-weight: bold;")

    else:
        # Conectar
        port = self.port_combo.currentText()
        if port == "Nenhuma porta encontrada":
            QMessageBox.warning(self, "Erro", "Nenhuma porta disponível")
            return

        baudrate = int(self.baudrate_combo.currentText())

        if self.tensiometer.connect(port, baudrate):
            self.connect_btn.setText("Desconectar")
            self.test_btn.setEnabled(True)
            self.connection_status.setText(f"Conectado - {port}")
            self.connection_status.setStyleSheet("color: green; font-weight: bold;")
        else:
            QMessageBox.critical(
                self, "Erro de Conexão",
                f"Falha ao conectar:\n{self.tensiometer.last_error}"
            )

def _test_reading(self):
    """Testa leitura do tensiômetro"""
    if not self.tensiometer.is_connected:
        QMessageBox.warning(self, "Erro", "Tensiômetro não conectado")
        return

    value = self.tensiometer.read_tension_value()

    if value != "0" or not self.tensiometer.last_error:
        QMessageBox.information(
            self, "Teste de Leitura",
            f"Valor lido: {value}"
        )
    else:
        QMessageBox.warning(
            self, "Erro de Leitura",
            f"Falha na leitura:\n{self.tensiometer.last_error}"
        )

```

```

def _get_user_feed_rate(self):
    """
    Obtém a velocidade de movimentação configurada pelo usuário.
    Retorna valor padrão se inválido.
    """
    try:
        feed = float(self.movement_feed.text())
        if feed <= 0:
            return 1000.0 # fallback
        return feed
    except ValueError:
        return 1000.0 # fallback se valor inválido

# ----- helpers -----
# ----- wrappers de movimentação com espera -----
def _move_abs(self, *, x=None, y=None, z=None, feed=None):
    """
    Move em coordenadas absolutas.
    feed → mapeado para feed_rate exigido pelo GRBLCNCController.
    """
    if feed is None:
        feed = self._get_user_feed_rate()
    if feed is not None:
        self.cnc.move_to_absolute_position(
            x=x, y=y, z=z, feed_rate=feed
        )
    else:
        self.cnc.move_to_absolute_position(
            x=x, y=y, z=z
        )
    self.cnc.wait_for_idle()

def _move_rel(self, *, x=None, y=None, z=None, feed=None):
    """
    Move em coordenadas relativas.
    feed → feed_rate.
    """
    if feed is None:
        feed = self._get_user_feed_rate()
    kwargs = {}
    if x is not None: kwargs['x'] = x
    if y is not None: kwargs['y'] = y
    if z is not None: kwargs['z'] = z
    if feed is not None: kwargs['feed_rate'] = feed
    self.cnc.move_relative(**kwargs)
    self.cnc.wait_for_idle()

def _grid_points(self):
    try:
        sx, sy = float(self.ed_sx.text()), float(self.ed_sy.text())
        ex, ey = float(self.ed_ex.text()), float(self.ed_ey.text())

```

```

n = int(self.ed_n.text())
if n < 2:
    raise ValueError
xs = np.linspace(sx, ex, n)
ys = np.linspace(sy, ey, n)
points = []
for idx_row, y in enumerate(ys):
    row_xs = xs if idx_row % 2 == 0 else xs[::-1] # zig-zag
    for x in row_xs:
        points.append((float(x), float(y)))
return points
except ValueError:
    QMessageBox.warning(self, "Erro", "Preencha todos os valores corretamente (N ≥ 2).")
    return None

# ----- captura automática de posições -----
def _require_connection(self) -> bool:
    if not self.cnc.is_connected:
        QMessageBox.warning(self, "Erro", "CNC não conectada")
        return False
    return True

def _capture_start_xy(self):
    if not self._require_connection():
        return
    pos = self.cnc.get_current_position()
    self.ed_sx.setText(f'{pos["x"]:.3f}')
    self.ed_sy.setText(f'{pos["y"]:.3f}')

def _capture_end_xy(self):
    if not self._require_connection():
        return
    pos = self.cnc.get_current_position()
    self.ed_ex.setText(f'{pos["x"]:.3f}')
    self.ed_ey.setText(f'{pos["y"]:.3f}')

def _capture_z_height(self):
    if not self._require_connection():
        return
    pos = self.cnc.get_current_position()
    # Usa valor absoluto para sempre baixar – usuário pode ajustar depois
    self.ed_z.setText(f'{abs(pos["z"]):.3f}')

def _read_tension(self):
    """
    Lê valor do tensiômetro usando a conexão dedicada
    """
    if not self.tensiometer.is_connected:
        log.warning("Tentativa de leitura com tensiômetro desconectado")
        return "0"
    return self.tensiometer.read_tension_value()

```

```

# ----- ciclo principal -----
def _on_start(self):
    """Inicia o processo de medição em thread separada"""
    pts = self._grid_points()
    if pts is None:
        return

    # Agora o zero de Z fica no topo -> valor POSITIVO desce,
    # valor NEGATIVO sobe. Basta usar o valor digitado para descer.
    try:
        z_val = float(self.ed_z.text())
        z_down = z_val
    except ValueError:
        QMessageBox.warning(self, "Erro", "Altura Z inválida.")
        return

    # Verifica se tensiômetro está conectado
    if not self.tensiometer.is_connected:
        QMessageBox.warning(self, "Erro", "Conecte o tensiômetro antes de iniciar")
        return

    if not self.cnc.is_connected:
        QMessageBox.warning(self, "Erro", "CNC não conectada")
        return

    # Obtém parâmetros de configuração
    user_feed = self._get_user_feed_rate()
    try:
        stabilization_ms = int(self.stabilization_time.text())
    except ValueError:
        stabilization_ms = 500 # fallback

    # Parâmetros para salvar no JSON
    parameters = {
        "start": {"x": float(self.ed_sx.text()), "y": float(self.ed_sy.text())},
        "end": {"x": float(self.ed_ex.text()), "y": float(self.ed_ey.text())},
        "quantity": int(self.ed_n.text()),
        "height": float(self.ed_z.text())
    }

    # Configura interface para medição
    self._setup_measurement_ui(len(pts))

    # Cria e inicia thread de medição
    self.measurement_thread = TensionMeasurementThread(
        self.cnc, self.tensiometer, pts, z_down,
        user_feed, stabilization_ms, parameters
    )

    # Conecta sinais
    self.measurement_thread.progress_updated.connect(self._on_progress_updated)

```

```

self.measurement_thread.measurement_completed.connect(self._on_measurement_completed)
self.measurement_thread.finished.connect(self._on_measurement_finished)
self.measurement_thread.error_occurred.connect(self._on_measurement_error)

# Inicia medição
self.measurement_thread.start()

def _on_stop(self):
    """Para o processo de medição"""
    if self.measurement_thread and self.measurement_thread.isRunning():
        self.measurement_thread.request_stop()
        self.status_label.setText("Parando medição...")
        self.stop_measurement_btn.setEnabled(False)

def _setup_measurement_ui(self, total_points):
    """Configura interface para medição"""
    self.start_measurement_btn.setEnabled(False)
    self.stop_measurement_btn.setEnabled(True)
    self.progress_bar.setVisible(True)
    self.progress_bar.setRange(0, total_points)
    self.progress_bar.setValue(0)
    self.status_label.setText("Iniciando medição...")

def _reset_measurement_ui(self):
    """Reseta interface após medição"""
    self.start_measurement_btn.setEnabled(True)
    self.stop_measurement_btn.setEnabled(False)
    self.progress_bar.setVisible(False)
    self.status_label.setText("")

def _on_progress_updated(self, current, total, status_msg):
    """Atualiza progresso da medição"""
    self.progress_bar.setValue(current)
    self.status_label.setText(status_msg)

    # Atualiza título da janela com progresso
    self.setWindowTitle(f"Tensão do Stencil - {current}/{total}")

def _on_measurement_completed(self, measurement):
    """Chamado quando uma medição individual é completada"""
    # Aqui pode adicionar lógica para processar cada medição individual
    # Por exemplo, mostrar em uma tabela em tempo real
    log.debug(f"Medição completada: {measurement}")

def _on_measurement_finished(self, measurements):
    """Chamado quando todas as medições são completadas"""
    self._reset_measurement_ui()
    self.setWindowTitle("Tensão do Stencil")

    # Salva resultados
    try:

```

```

parameters = {
    "start": {"x": float(self.ed_sx.text()), "y": float(self.ed_sy.text())},
    "end": {"x": float(self.ed_ex.text()), "y": float(self.ed_ey.text())},
    "quantity": int(self.ed_n.text()),
    "height": float(self.ed_z.text())
}

```

```

data = {
    "type": "stencil_tension",
    "parameters": parameters,
    "measurements": measurements
}

```

```

with open("stencil_tension_measurements.json", "w", encoding="utf-8") as fp:
    json.dump(data, fp, indent=4, ensure_ascii=False)

```

```

QMessageBox.information(
    self, "Concluído",
    f"Medição concluída!\n"
    f"Total de pontos: {len(measurements)}\n"
    f"Arquivo salvo: stencil_tension_measurements.json"
)
self.accept()

```

except Exception as e:

```

QMessageBox.critical(self, "Erro", f"Falha ao salvar resultados: {e}")

```

def _on_measurement_error(self, error_msg):

```

    """Chamado quando ocorre erro na medição"""
    self._reset_measurement_ui()
    self.setWindowTitle("Tensão do Stencil")
    QMessageBox.critical(self, "Erro na Medição", error_msg)

```

def closeEvent(self, event):

```

    """Garante desconexão e parada da thread ao fechar o diálogo"""
    # Para thread se estiver rodando
    if self.measurement_thread and self.measurement_thread.isRunning():
        self.measurement_thread.request_stop()
        self.measurement_thread.wait(3000) # Aguarda até 3 segundos

```

Desconecta tensiômetro

```

if self.tensiometer.is_connected:
    self.tensiometer.disconnect()

```

```

super().closeEvent(event)

```

File: __init__.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/aoi_lib

```
# Exporta as classes principais para facilitar importação
from .aoi_controller import CNCAOIController
from .cnc_controller import GRBLCNCController
from .camera_controller import CameraController
from .position_manager import InspectionPosition, InspectionSequence

__version__ = '0.1.0'
```

File: move_task.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/aoi_libutils

```
from PyQt6.QtCore import QThread, pyqtSignal

class MoveTaskThread(QThread):
    """
    Executa um deslocamento absoluto sem bloquear a interface:
    – envia o G-code
    – aguarda a máquina ficar Idle
    """
    finished = pyqtSignal(float, float, float)
    error = pyqtSignal(str)

    def __init__(self, cnc, x=None, y=None, z=None, feed=1000):
        super().__init__()
        self._cnc = cnc
        self._x = x
        self._y = y
        self._z = z
        self._feed = feed

    def run(self):
        try:
            ok = self._cnc.move_to_absolute_position(
                x=self._x, y=self._y, z=self._z, feed_rate=self._feed
            )
            if not ok:
                self.error.emit("Falha ao enviar comando de movimento")
                return
            self._cnc.wait_for_idle()
            self.finished.emit(self._x or 0.0, self._y or 0.0, self._z or 0.0)
        except Exception as exc:
            self.error.emit(str(exc))
```

File: __init__.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/aoi_lib\utils

File: consumo_lib.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro

```
import sys
import cv2
import os
import time
import numpy as np
from PyQt6.QtWidgets import (QApplication, QMainWindow, QWidget, QVBoxLayout, QHBoxLayout,
                             QPushButton, QLabel, QGroupBox, QGridLayout, QLineEdit,
                             QComboBox, QListWidget, QCheckBox, QListWidgetItem,
                             QFileDialog, QMessageBox, QTabWidget, QSizePolicy,
                             QSplitter, QFrame, QTableWidgetItem, QTableWidgetItem, QHeaderView,
                             QDialog, QDialog, QInputDialog,
                             QProgressDialog)
from PyQt6.QtCore import Qt, QThread, pyqtSignal, QTimer, QEvent, QRectF, QPointF
from PyQt6.QtGui import (QPixmap, QImage, QFont, QAction, QDoubleValidator,
                          QPainter, QColor, QPen, QBrush)

from aoi_lib import CNCAOIController, InspectionPosition
from aoi_lib.plc_axis_controller import PLCAxisController
from grbl_streamer import GrblStreamer
from aoi_lib.utils.move_task import MoveTaskThread
from aoi_lib.config_manager import AOIConfigManager, SettingsDialog
from dataclasses import dataclass
from aoi_lib.stencil_tension import StencilTensionDialog
import logging
import json

logger = logging.getLogger("consumo_lib")
logger.setLevel(logging.DEBUG)
# Se necessário, adicione um handler:
if not logger.handlers:
    ch = logging.StreamHandler()
    ch.setLevel(logging.DEBUG)
    formatter = logging.Formatter("%(asctime)s - %(name)s - %(levelname)s - %(message)s")
    ch.setFormatter(formatter)
    logger.addHandler(ch)

class TensionVisualizationWidget(QWidget):
    """Widget para visualizar os resultados de medição de tensão do stencil"""

    def __init__(self, parent=None):
        super().__init__(parent)
        self.setup_ui()
```

```

self.measurements_data = None
self.canvas_margin = 50
self.point_radius = 15

def setup_ui(self):
    layout = QVBoxLayout(self)

    # Título e controles
    title_layout = QHBoxLayout()

    title_label = QLabel("Visualização de Tensão do Stencil")
    title_font = QFont()
    title_font.setBold(True)
    title_font.setPointSize(14)
    title_label.setFont(title_font)

    # Botão para carregar arquivo
    self.load_file_btn = QPushButton("Carregar Arquivo JSON")
    self.load_file_btn.clicked.connect(self.load_tension_file)

    # Botão para recarregar último arquivo
    self.reload_btn = QPushButton("Recarregar")
    self.reload_btn.clicked.connect(self.reload_last_file)
    self.reload_btn.setEnabled(False)

    title_layout.addWidget(title_label)
    title_layout.addStretch()
    title_layout.addWidget(self.load_file_btn)
    title_layout.addWidget(self.reload_btn)

    layout.addLayout(title_layout)

    # Informações do arquivo carregado
    self.info_label = QLabel("Nenhum arquivo carregado")
    self.info_label.setStyleSheet("color: #666; font-style: italic;")
    layout.addWidget(self.info_label)

    # Canvas de visualização
    self.canvas = TensionCanvas()
    layout.addWidget(self.canvas, 1) # Proporção 1 para expandir

    # Legenda
    self.legend_label = QLabel("")
    layout.addWidget(self.legend_label)

    self.last_file_path = None

def load_tension_file(self):
    """Carrega arquivo JSON com dados de tensão"""
    file_path, _ = QFileDialog.getOpenFileName(
        self,

```

```

"Carregar Dados de Tensão",
    """
    "Arquivos JSON (*.json);;Todos os arquivos (*)"
)

```

```

if file_path:
    self.load_file(file_path)

```

```

def load_file(self, file_path):
    """Carrega arquivo específico"""
    try:
        with open(file_path, 'r', encoding='utf-8') as f:
            data = json.load(f)

        # Valida se é um arquivo de tensão válido
        if data.get('type') != 'stencil_tension':
            QMessageBox.warning(
                self, "Arquivo Inválido",
                "Este não é um arquivo de medição de tensão válido."
            )
            return

        self.measurements_data = data
        self.last_file_path = file_path
        self.reload_btn.setEnabled(True)

        # Atualiza informações
        self.update_info_display()

        # Atualiza canvas
        self.canvas.set_measurements(data)

        # Atualiza legenda
        self.update_legend()

    except Exception as e:
        QMessageBox.critical(
            self, "Erro",
            f"Erro ao carregar arquivo:\n{str(e)}"
        )

```

```

def reload_last_file(self):
    """Recarrega o último arquivo carregado"""
    if self.last_file_path:
        self.load_file(self.last_file_path)

```

```

def update_info_display(self):
    """Atualiza informações do arquivo carregado"""
    if not self.measurements_data:
        return

```

```
params = self.measurements_data.get('parameters', {})  
measurements = self.measurements_data.get('measurements', [])
```

```
start = params.get('start', {})  
end = params.get('end', {})  
quantity = params.get('quantity', 0)
```

```
info_text = (  
    f"Arquivo carregado: {len(measurements)} pontos medidos | "  
    f"Grid: {quantity}x{quantity} | "  
    f"Área: X({start.get('x', 0):.1f} a {end.get('x', 0):.1f}) "  
    f"Y({start.get('y', 0):.1f} a {end.get('y', 0):.1f})"  
)
```

```
self.info_label.setText(info_text)  
self.info_label.setStyleSheet("color: #333; font-weight: bold;")
```

```
def update_legend(self):  
    """Atualiza legenda com informações dos valores"""  
    if not self.measurements_data:  
        return
```

```
measurements = self.measurements_data.get('measurements', [])  
if not measurements:  
    return
```

```
# Calcula estatísticas  
tensions = [float(m.get('tension', 0)) for m in measurements]  
min_tension = min(tensions)  
max_tension = max(tensions)  
avg_tension = sum(tensions) / len(tensions)
```

```
legend_text = (  
    f"Tensão: Mín: {min_tension:.2f} N/cm² | "  
    f"Máx: {max_tension:.2f} N/cm² | "  
    f"Média: {avg_tension:.2f} N/cm² | "  
    f"■ Baixa | ■ Média | ■ Alta"  
)
```

```
self.legend_label.setText(legend_text)
```

```
class TensionCanvas(QWidget):
```

```
    """Canvas personalizado para desenhar os pontos de tensão"""
```

```
def __init__(self, parent=None):  
    super().__init__(parent)  
    self.measurements = None  
    self.setMinimumSize(400, 400)
```

```
def set_measurements(self, data):  
    """Define os dados de medição"""  
    self.measurements = data
```

```
self.update() # Força redesenho
```

```
def paintEvent(self, event):
```

```
    """Desenha o canvas com os pontos de tensão"""
```

```
    painter = QPainter(self)
```

```
    painter.setRenderHint(QPainter.RenderHint.Antialiasing)
```

```
    # Fundo branco
```

```
    painter.fillRect(self.rect(), QColor(255, 255, 255))
```

```
    if not self.measurements:
```

```
        # Desenha mensagem quando não há dados
```

```
        painter.setPen(QColor(128, 128, 128))
```

```
        painter.drawText(
```

```
            self.rect(),
```

```
            Qt.AlignmentFlag.AlignCenter,
```

```
            "Carregue um arquivo JSON para visualizar os dados"
```

```
        )
```

```
    return
```

```
self._draw_measurements(painter)
```

```
def _draw_measurements(self, painter):
```

```
    """Desenha as medições no canvas"""
```

```
    measurements_list = self.measurements.get('measurements', [])
```

```
    params = self.measurements.get('parameters', {})
```

```
    if not measurements_list:
```

```
        return
```

```
    # Calcula limites da área
```

```
    start = params.get('start', {'x': 0, 'y': 0})
```

```
    end = params.get('end', {'x': 100, 'y': 100})
```

```
    # Dimensões da área de trabalho
```

```
    work_width = abs(end['x'] - start['x'])
```

```
    work_height = abs(end['y'] - start['y'])
```

```
    # Dimensões do canvas (com margem)
```

```
    margin = 50
```

```
    canvas_width = self.width() - 2 * margin
```

```
    canvas_height = self.height() - 2 * margin
```

```
    # Usa o menor lado para manter proporção quadrada
```

```
    canvas_size = min(canvas_width, canvas_height)
```

```
    # Calcula escala
```

```
    scale_x = canvas_size / work_width if work_width > 0 else 1
```

```
    scale_y = canvas_size / work_height if work_height > 0 else 1
```

```
    scale = min(scale_x, scale_y)
```

```

# Centro do canvas
center_x = self.width() / 2
center_y = self.height() / 2

# Desenha bordas da área de trabalho
self._draw_work_area_border(painter, center_x, center_y, work_width, work_height, scale)

# Calcula estatísticas para coloração
tensions = [float(m.get('tension', 0)) for m in measurements_list]
min_tension = min(tensions) if tensions else 0
max_tension = max(tensions) if tensions else 100
tension_range = max_tension - min_tension if max_tension != min_tension else 1

# Desenha cada ponto
for measurement in measurements_list:
    self._draw_measurement_point(
        painter, measurement, start, center_x, center_y,
        scale, min_tension, tension_range
    )

def _draw_work_area_border(self, painter, center_x, center_y, work_width, work_height, scale):
    """Desenha a borda da área de trabalho"""
    # Calcula posição do retângulo da área de trabalho
    rect_width = work_width * scale
    rect_height = work_height * scale

    rect_x = center_x - rect_width / 2
    rect_y = center_y - rect_height / 2

    # Desenha borda
    painter.setPen(QPen(QColor(200, 200, 200), 2))
    painter.setBrush(QBrush()) # Sem preenchimento
    painter.drawRect(QRectF(rect_x, rect_y, rect_width, rect_height))

    # Desenha grid de referência (opcional)
    painter.setPen(QPen(QColor(240, 240, 240), 1))

    # Linhas verticais
    for i in range(1, 3): # Assume grid 3x3
        x = rect_x + (rect_width * i / 3)
        painter.drawLine(QPointF(x, rect_y), QPointF(x, rect_y + rect_height))

    # Linhas horizontais
    for i in range(1, 3):
        y = rect_y + (rect_height * i / 3)
        painter.drawLine(QPointF(rect_x, y), QPointF(rect_x + rect_width, y))

def _draw_measurement_point(self, painter, measurement, start, center_x, center_y,
                           scale, min_tension, tension_range):
    """Desenha um ponto de medição individual"""
    x = measurement.get('x', 0)

```

```

y = measurement.get('y', 0)
tension = float(measurement.get('tension', 0))

# Calcula dimensões da área de trabalho
params = self.measurements.get('parameters', {})
end = params.get('end', {'x': 100, 'y': 100})

work_width = abs(end['x'] - start['x'])
work_height = abs(end['y'] - start['y'])

# Dimensões do retângulo de trabalho no canvas
margin = 50
canvas_size = min(self.width() - 2 * margin, self.height() - 2 * margin)
scale = min(canvas_size / work_width, canvas_size / work_height) if work_width > 0 and work_height > 0 else 1

rect_width = work_width * scale
rect_height = work_height * scale

# Posição do retângulo da área de trabalho (centralizado)
rect_x = center_x - rect_width / 2
rect_y = center_y - rect_height / 2

# Normaliza a posição do ponto dentro da área de trabalho (0 a 1)
norm_x = (x - start['x']) / work_width if work_width > 0 else 0
norm_y = (y - start['y']) / work_height if work_height > 0 else 0

# Mapeia para coordenadas do canvas
# Nota: no canvas, Y cresce para baixo, então invertamos norm_y
canvas_x = rect_x + (norm_x * rect_width)
canvas_y = rect_y + ((1 - norm_y) * rect_height) # Inverte Y para visualização correta
# Determina cor baseada na tensão
color = self._get_tension_color(tension, min_tension, tension_range)

# Desenha círculo
point_radius = 20
painter.setPen(QPen(QColor(100, 100, 100), 2))
painter.setBrush(QBrush(color))

painter.drawEllipse(
    QPointF(canvas_x, canvas_y),
    point_radius, point_radius
)

# Desenha texto com valor
painter.setPen(QColor(0, 0, 0))
painter.setFont(QFont("Arial", 8, QFont.Weight.Bold))

# Texto centralizado no círculo
text = f'{tension:.1f}'
text_rect = painter.fontMetrics().boundingRect(text)
text_x = canvas_x - text_rect.width() / 2

```

```
text_y = canvas_y + text_rect.height() / 4
```

```
painter.drawText(QPointF(text_x, text_y), text)
```

```
# Desenha coordenadas menores abaixo
```

```
coord_text = f"({x:.1f},{y:.1f})"
```

```
painter.setFont(QFont("Arial", 6))
```

```
painter.setPen(QColor(80, 80, 80))
```

```
coord_rect = painter.fontMetrics().boundingRect(coord_text)
```

```
coord_x = canvas_x - coord_rect.width() / 2
```

```
coord_y = canvas_y + point_radius + 15
```

```
painter.drawText(QPointF(coord_x, coord_y), coord_text)
```

```
def _get_tension_color(self, tension, min_tension, tension_range):
```

```
    """Retorna cor baseada no valor da tensão"""
```

```
    if tension_range == 0:
```

```
        return QColor(100, 200, 100) # Verde padrão
```

```
    # Normaliza tensão (0-1)
```

```
    normalized = (tension - min_tension) / tension_range
```

```
    # Mapeia para cores: Verde (baixo) -> Amarelo (médio) -> Vermelho (alto)
```

```
    if normalized < 0.33:
```

```
        # Verde para amarelo
```

```
        ratio = normalized * 3
```

```
        return QColor(int(100 + 155 * ratio), 200, int(100 * (1 - ratio)))
```

```
    elif normalized < 0.66:
```

```
        # Amarelo para laranja
```

```
        ratio = (normalized - 0.33) * 3
```

```
        return QColor(255, int(200 - 50 * ratio), 0)
```

```
    else:
```

```
        # Laranja para vermelho
```

```
        ratio = (normalized - 0.66) * 3
```

```
        return QColor(255, int(150 * (1 - ratio)), 0)
```

```
class ImageViewerWidget(QWidget):
```

```
    """Widget para exibir imagens capturadas pela câmera"""
```

```
    def __init__(self, parent=None):
```

```
        super().__init__(parent)
```

```
        self.layout = QVBoxLayout(self)
```

```
    # Label para exibir a imagemgit
```

```
    self.image_label = QLabel()
```

```
    self.image_label.setAlignment(Qt.AlignmentFlag.AlignCenter)
```

```
    self.image_label.setText("Nenhuma imagem capturada")
```

```
    self.image_label.setStyleSheet("border: 1px solid gray; background-color: #f0f0f0;")
```

```
    self.image_label.setMinimumSize(400, 300)
```

```
    # Informações da imagem
```

```
self.info_label = QLabel("Informações da imagem:")
```

```
self.layout.addWidget(self.image_label)
```

```
self.layout.addWidget(self.info_label)
```

```
def display_image(self, image, info_text=None):
```

```
    """Mostra uma imagem no widget"""
```

```
    if image is None:
```

```
        self.image_label.setText("Imagem inválida")
```

```
        return
```

```
    # Converte imagem OpenCV para QPixmap
```

```
    h, w, c = image.shape
```

```
    bytes_per_line = 3 * w
```

```
    q_img = QImage(image.data, w, h, bytes_per_line, QImage.Format.Format_RGB888).rgbSwapped()
```

```
    pixmap = QPixmap.fromImage(q_img)
```

```
    # Redimensiona se for muito grande
```

```
    if pixmap.width() > 800 or pixmap.height() > 600:
```

```
        pixmap = pixmap.scaled(800, 600, Qt.AspectRatioMode.KeepAspectRatio)
```

```
    self.image_label.setPixmap(pixmap)
```

```
    # Atualiza informações
```

```
    if info_text:
```

```
        self.info_label.setText(info_text)
```

```
    else:
```

```
        self.info_label.setText(f"Imagem: {w}x{h}px")
```

```
class PositionListWidget(QWidget):
```

```
    """Widget para gerenciar lista de posições"""
```

```
    position_selected = pyqtSignal(object) # Emite a posição selecionada
```

```
    def __init__(self, parent=None):
```

```
        super().__init__(parent)
```

```
        self.layout = QVBoxLayout(self)
```

```
    # Título
```

```
    title_label = QLabel("Posições de Inspeção")
```

```
    title_font = QFont()
```

```
    title_font.setBold(True)
```

```
    title_label.setFont(title_font)
```

```
    # Lista de posições
```

```
    self.positions_list = QListWidget()
```

```
    self.positions_list.currentItemChanged.connect(self.on_position_selected)
```

```
    # Botões
```

```
    buttons_layout = QHBoxLayout()
```

```
    self.add_position_btn = QPushButton("Adicionar Posição Atual")
```

```
    self.remove_position_btn = QPushButton("Remover")
```

```
buttons_layout.addWidget(self.add_position_btn)
buttons_layout.addWidget(self.remove_position_btn)
```

```
self.layout.addWidget(title_label)
self.layout.addWidget(self.positions_list)
self.layout.addLayout(buttons_layout)
```

```
# Mapeia id(QListWidgetItem) → InspectionPosition.
# QListWidgetItem NÃO é hashable, portanto usamos id(item).
self.positions: dict[int, InspectionPosition] = {}
```

```
def add_position(self, position: InspectionPosition):
    """Adiciona uma posição à lista"""
    item_text = f'{position.name} ({position.x:.2f}, {position.y:.2f}, {getattr(position, 'z', 0.0):.2f})'
    item = QListWidgetItem(item_text)
    self.positions_list.addItem(item)
    self.positions[id(item)] = position
```

```
def remove_selected_position(self):
    """Remove a posição selecionada"""
    current_item = self.positions_list.currentItem()
    if current_item:
        position = self.positions.pop(id(current_item))
        row = self.positions_list.row(current_item)
        self.positions_list.takeItem(row)
        return position
    return None
```

```
def clear_positions(self):
    """Limpa todas as posições"""
    self.positions_list.clear()
    self.positions = {}
```

```
def get_all_positions(self):
    """Retorna todas as posições"""
    return list(self.positions.values())
```

```
def on_position_selected(self, current, previous):
    """Manipula evento de seleção de posição"""
    if current:
        pos = self.positions.get(id(current))
        if pos:
            self.position_selected.emit(pos)
```

```
class SequenceControlWidget(QWidget):
    """Widget para controlar a execução da sequência"""
    def __init__(self, parent=None):
        super().__init__(parent)
        self.layout = QVBoxLayout(self)

    # Grupo de sequência
```

```

sequence_group = QGroupBox("Controle de Sequência")
sequence_layout = QGridLayout()

# Nome da sequência
sequence_layout.addWidget(QLabel("Nome:"), 0, 0)
self.sequence_name = QLineEdit("Sequência PCB")
sequence_layout.addWidget(self.sequence_name, 0, 1)

# Botões de controle
self.create_sequence_btn = QPushButton("Criar Sequência")
self.run_sequence_btn = QPushButton("Executar Sequência")
self.stop_sequence_btn = QPushButton("Parar")
self.stop_sequence_btn.setEnabled(False)

sequence_layout.addWidget(self.create_sequence_btn, 1, 0)
sequence_layout.addWidget(self.run_sequence_btn, 1, 1)
sequence_layout.addWidget(self.stop_sequence_btn, 2, 0, 1, 2)

# Status
sequence_layout.addWidget(QLabel("Status:"), 3, 0)
self.sequence_status = QLabel("Pronto")
sequence_layout.addWidget(self.sequence_status, 3, 1)

sequence_group.setLayout(sequence_layout)

# Grupo de arquivo
file_group = QGroupBox("Salvar/Carregar")
file_layout = QVBoxLayout()
self.save_btn = QPushButton("Salvar Programa (JSON)")
self.load_btn = QPushButton("Carregar Programa (JSON)")
self.save_gcode_btn = QPushButton("Exportar para G-CODE")
self.load_gcode_btn = QPushButton("Importar de G-CODE")
file_layout.addWidget(self.save_btn)
file_layout.addWidget(self.load_btn)
file_layout.addWidget(self.save_gcode_btn)
file_layout.addWidget(self.load_gcode_btn)

file_group.setLayout(file_layout)

self.layout.addWidget(sequence_group)
self.layout.addWidget(file_group)
self.layout.addStretch()

```

```

class PositionRegistryWidget(QWidget):

```

```

    """Widget for showing registered positions"""

```

```

    def __init__(self, controller, cfg: AOIConfigManager, parent=None):

```

```

        super().__init__(parent)

```

```

        self.controller = controller

```

```

        self.cfg = cfg

```

```

        self.cfg = cfg

```

```

        self.positions = []

```

```

self.setup_ui()

def setup_ui(self):
    layout = QVBoxLayout(self)

    # Title
    title_label = QLabel("Registered Positions")
    title_font = QFont()
    title_font.setBold(True)
    title_label.setFont(title_font)
    layout.addWidget(title_label)

    # Registered positions table
    self.positions_table = QTableWidgetItem(0, 3)
    self.positions_table.setHorizontalHeaderLabels(["Name", "X (mm)", "Y (mm)"])
    self.positions_table.horizontalHeader().setSectionResizeMode(QHeaderView.ResizeMode.Stretch)
    layout.addWidget(self.positions_table)

    # Buttons
    buttons_layout = QHBoxLayout()

    self.delete_button = QPushButton("Delete Selected")
    self.delete_button.clicked.connect(self.delete_position)

    self.create_sequence_btn = QPushButton("Create Sequence")

    buttons_layout.addWidget(self.delete_button)
    buttons_layout.addWidget(self.create_sequence_btn)

    layout.addLayout(buttons_layout)

def add_position(self, name, x, y, z=0.0, image=None):
    """Add a position to the registry"""
    # Create position object
    position = {
        'name': name,
        'x': x,
        'y': y,
        'z': z,
        'image': image,
    }

    # Add to internal list
    self.positions.append(position)

    # Add to table
    row = self.positions_table.rowCount()
    self.positions_table.insertRow(row)
    self.positions_table.setItem(row, 0, QTableWidgetItem(name))
    self.positions_table.setItem(row, 1, QTableWidgetItem(f"{x:.3f}")
    self.positions_table.setItem(row, 2, QTableWidgetItem(f"{y:.3f}")

```

```

def delete_position(self):
    """Delete selected position"""
    selected_rows = self.positions_table.selectedItems()
    if not selected_rows:
        return

    row = selected_rows[0].row()
    if row >= 0 and row < len(self.positions):
        del self.positions[row]
        self.positions_table.removeRow(row)

def clear_positions(self):
    """Clear all positions"""
    self.positions.clear()
    while self.positions_table.rowCount() > 0:
        self.positions_table.removeRow(0)

class CameraPreviewWidget(QWidget):
    """Widget for displaying camera preview and capturing images"""
    image_captured = pyqtSignal(object, str) # Emits the captured image and position name

    def __init__(self, controller, cfg: AOIConfigManager, parent=None):
        super().__init__(parent)
        self.controller = controller
        self.current_image = None
        self.preview_timer = QTimer(self)
        self.preview_timer.timeout.connect(self.update_preview)
        self.setup_ui()

    def setup_ui(self):
        layout = QVBoxLayout(self)
        # GroupBox para preview da câmera (inclui botões de controle)
        preview_group = QGroupBox("Camera Preview")
        preview_group.setMinimumHeight(450)
        pg_layout = QVBoxLayout(preview_group)

        # Área de visualização
        self.image_label = QLabel()
        self.image_label.setAlignment(Qt.AlignmentFlag.AlignCenter)
        self.image_label.setText("Camera Preview")
        self.image_label.setStyleSheet("border: 1px solid gray; background-color: #f0f0f0;")
        self.image_label.setMinimumSize(600, 450)
        pg_layout.addWidget(self.image_label)

        # Botões de preview dentro da mesma groupbox
        btn_layout = QHBoxLayout()
        self.start_preview_btn = QPushButton("Start Preview")
        self.start_preview_btn.clicked.connect(self.start_preview)
        self.stop_preview_btn = QPushButton("Stop Preview")
        self.stop_preview_btn.clicked.connect(self.stop_preview)

```

```
self.stop_preview_btn.setEnabled(False)
btn_layout.addWidget(self.start_preview_btn)
btn_layout.addWidget(self.stop_preview_btn)
pg_layout.addLayout(btn_layout)
```

```
layout.addWidget(preview_group, 1)
```

```
def start_preview(self):
    """Start camera preview"""
    if not hasattr(self.controller.camera, 'is_connected') or not self.controller.camera.is_connected:
        QMessageBox.warning(self, "Error", "Camera not connected")
        return
    self.preview_timer.start(100) # Update every 100ms
    self.start_preview_btn.setEnabled(False)
    self.stop_preview_btn.setEnabled(True)
```

```
def stop_preview(self):
    """Stop camera preview"""
    self.preview_timer.stop()
    self.start_preview_btn.setEnabled(True)
    self.stop_preview_btn.setEnabled(False)
```

```
def update_preview(self):
    """Update the camera preview"""
    try:
        image = self.controller.camera.capture()
        if image is not None:
            self.display_image(image)
            self.current_image = image
    except Exception as e:
        print(f"Error updating preview: {e}")
        self.stop_preview()
```

```
def capture_image(self):
    """Capture an image and emit signal with position name"""
    if not hasattr(self.controller.camera, 'is_connected') or not self.controller.camera.is_connected:
        QMessageBox.warning(self, "Error", "Camera not connected")
        return
    position_name = self.position_name.text()
    if not position_name:
        QMessageBox.warning(self, "Error", "Please enter a position name")
        return
    try:
        image = self.controller.camera.capture()
        if image is not None:
            self.display_image(image)
            self.current_image = image
            self.image_captured.emit(image, position_name)
            self.position_name.clear()
        else:
            QMessageBox.warning(self, "Error", "Failed to capture image")
```

except Exception as e:

```
QMessageBox.warning(self, "Error", f"Capture error: {e}")
```

```
def display_image(self, image):
```

```
    """Display an image in the preview area with a crosshair in the center"""
```

```
    if image is None:
```

```
        return
```

```
    # Criar uma cópia da imagem para não modificar a original
```

```
    display_img = image.copy()
```

```
    # Desenhar a cruz vermelha no centro
```

```
    h, w, _ = display_img.shape
```

```
    center_x, center_y = w // 2, h // 2
```

```
    # Parâmetros da cruz
```

```
    color = (0, 0, 255) # Vermelho em BGR
```

```
    thickness = 2
```

```
    length = min(w, h) // 20 # 5% do tamanho da dimensão menor
```

```
    # Desenhar a cruz
```

```
    # Linha horizontal
```

```
    cv2.line(display_img,
```

```
              (center_x - length, center_y),
```

```
              (center_x + length, center_y),
```

```
              color, thickness)
```

```
    # Linha vertical
```

```
    cv2.line(display_img,
```

```
              (center_x, center_y - length),
```

```
              (center_x, center_y + length),
```

```
              color, thickness)
```

```
    # Converter a imagem OpenCV para QPixmap
```

```
    h, w, c = display_img.shape
```

```
    bytes_per_line = 3 * w
```

```
    q_img = QImage(display_img.data, w, h, bytes_per_line, QImage.Format.Format_RGB888).rgbSwapped()
```

```
    pixmap = QPixmap.fromImage(q_img)
```

```
    # Obter tamanho do widget de exibição
```

```
    label_width = self.image_label.width()
```

```
    label_height = self.image_label.height()
```

```
    # Redimensionar a imagem para caber no espaço disponível mantendo proporções
```

```
    pixmap = pixmap.scaled(label_width, label_height,
```

```
                           Qt.AspectRatioMode.KeepAspectRatio,
```

```
                           Qt.TransformationMode.SmoothTransformation)
```

```
    self.image_label.setPixmap(pixmap)
```

```
def resizeEvent(self, event):
```

```
    """Override do evento de redimensionamento para ajustar a imagem quando o widget for redimensionado"""
```

```
super().resizeEvent(event)
if self.current_image is not None:
    self.display_image(self.current_image)
```

```
class MovementControlWidget(QWidget):
    """Widget for controlling CNC movement (jog)"""
    def __init__(self, controller, cfg: AOIConfigManager, parent=None):
        super().__init__(parent)
        self.controller = controller
        self.cfg = cfg
        self.setup_ui()

    def setup_ui(self):
        layout = QVBoxLayout(self)

        # Group box for movement controls
        movement_group = QGroupBox("Movement Controls")
        # Vertical size fixed to its contents (no stretch)
        movement_group.setSizePolicy(QSizePolicy.Policy.Preferred, QSizePolicy.Policy.Fixed)
        movement_layout = QGridLayout()

        # Directional control buttons
        self.up_button = QPushButton("↑")
        self.down_button = QPushButton("↓")
        self.left_button = QPushButton("←")
        self.right_button = QPushButton("→")

        # Style the buttons
        for btn in [self.up_button, self.down_button, self.left_button, self.right_button]:
            btn.setMinimumSize(50, 50)
            font = QFont()
            font.setBold(True)
            font.setPointSize(16)
            btn.setFont(font)

        # Connect press/release events for continuous movement
        self.up_button.pressed.connect(lambda: self._on_direction_press("Y", -1))
        self.up_button.released.connect(self._on_direction_release)
        self.down_button.pressed.connect(lambda: self._on_direction_press("Y", 1))
        self.down_button.released.connect(self._on_direction_release)
        self.left_button.pressed.connect(lambda: self._on_direction_press("X", -1))
        self.left_button.released.connect(self._on_direction_release)
        self.right_button.pressed.connect(lambda: self._on_direction_press("X", 1))
        self.right_button.released.connect(self._on_direction_release)

        # ----- NOVOS BOTÕES Z -----
        self.z_up_button = QPushButton("Z+")
        self.z_down_button = QPushButton("Z-")
        for zbtn in (self.z_up_button, self.z_down_button):
            zbtn.setMinimumSize(50, 30)
        self.z_up_button.pressed.connect(lambda: self._on_direction_press("Z", -1))
```

```

self.z_up_button.released.connect(self._on_direction_release)
self.z_down_button.pressed.connect(lambda: self._on_direction_press("Z", 1))
self.z_down_button.released.connect(self._on_direction_release)

# Add buttons to grid
movement_layout.addWidget(self.up_button, 0, 1)
movement_layout.addWidget(self.left_button, 1, 0)
movement_layout.addWidget(self.right_button, 1, 2)
movement_layout.addWidget(self.down_button, 2, 1)

# Coloca Z+ acima de STOP e Z- abaixo
movement_layout.addWidget(self.z_up_button, 0, 3)
movement_layout.addWidget(self.z_down_button, 2, 3)

# Botão de Emergency Stop / Reset
self.emergency_stop_button = QPushButton("STOP")
self.emergency_stop_button.setCheckable(True) # Torna o botão toggle
self.emergency_stop_button.setMinimumSize(100, 40)
font_stop = QFont()
font_stop.setBold(True)
self.emergency_stop_button.setFont(font_stop)
# Estilo inicial (vermelho)
self.emergency_stop_button.setStyleSheet("background-color: red; color: white;")
self.emergency_stop_button.toggled.connect(self.on_emergency_stop_toggle) # Conecta ao handler

# Adiciona o botão ao layout, centralizado abaixo dos direcionais
movement_layout.addWidget(self.emergency_stop_button, 1, 1, Qt.AlignmentFlag.AlignCenter) # Coloca no centro (lin

# Step size and feed rate controls
step_layout = QHBoxLayout()
step_layout.addWidget(QLabel("Step Size:"))
# ----- STEP SIZE -----
default_step = self.cfg.get("movement", "step_size", default=10.0)
self.step_size = QLineEdit(f"{default_step}")
# ■ VALIDAÇÃO numérica (>=0)
self.step_size.setValidator(QDoubleValidator(0.0001, 100000.0, 4, self))
step_layout.addWidget(self.step_size)
step_layout.addWidget(QLabel("mm"))

feed_layout = QHBoxLayout()
feed_layout.addWidget(QLabel("Feed Rate:"))

# ----- FEED RATE -----
default_feed = self.cfg.get("movement", "feed_rate", default=1000.0)
self.feed_rate = QLineEdit(f"{default_feed}")
# ■ feed entre 1 e 30000 mm/min
self.feed_rate.setValidator(QDoubleValidator(1.0, 30000.0, 0, self))
feed_layout.addWidget(self.feed_rate)
feed_layout.addWidget(QLabel("mm/min"))

# Add step and feed rate controls

```

```

movement_layout.addLayout(step_layout, 3, 0, 1, 3)
movement_layout.addLayout(feed_layout, 4, 0, 1, 3)

# ---- grava no JSON quando o usuário termina de editar ----
self.step_size.editingFinished.connect(self._save_step_feed)
self.feed_rate.editingFinished.connect(self._save_step_feed)

self.go_to_zero_btn = QPushButton("Go to Zero")
self.go_to_zero_btn.clicked.connect(self.go_to_zero)
self.go_to_zero_btn.setMinimumHeight(40) # Altura mínima para facilitar o clique
font = QFont()
font.setBold(True)
self.go_to_zero_btn.setFont(font)
movement_layout.addWidget(self.go_to_zero_btn, 6, 0, 1, 3) # Posiciona abaixo dos controles existentes

# Botão para deslocar a head para posição de trabalho definida pelo usuário (WPos)
self.go_to_position_btn = QPushButton("Go to Position")
self.go_to_position_btn.setToolTip("Ir para posição de trabalho específica (WPos)")
self.go_to_position_btn.clicked.connect(self.show_go_to_dialog)
movement_layout.addWidget(self.go_to_position_btn, 7, 0, 1, 3)

# Checkbox "Enable Keyboard Control" agora na linha 8 (posição anterior do Test Motor Hold)
self.keyboard_control_checkbox = QCheckBox("Enable Keyboard Control")
self.keyboard_control_checkbox.setChecked(False)
movement_layout.addWidget(self.keyboard_control_checkbox, 8, 0, 1, 3)

# Movement mode (G90/G91)
mode_layout = QHBoxLayout()
self.mode_absolute = QPushButton("Passo")
self.mode_absolute.setCheckable(True)
self.mode_absolute.clicked.connect(lambda: self.set_motion_mode("G90"))

self.mode_relative = QPushButton("Contínuo")
self.mode_relative.setCheckable(True)
self.mode_relative.setChecked(True) # Default to relative mode
self.mode_relative.clicked.connect(lambda: self.set_motion_mode("G91"))

mode_layout.addWidget(self.mode_absolute)
mode_layout.addWidget(self.mode_relative)
movement_layout.addLayout(mode_layout, 5, 0, 1, 3)

movement_group.setLayout(movement_layout)
layout.addWidget(movement_group)
# Keep the groupbox at top without stretching
layout.setAlignment(movement_group, Qt.AlignmentFlag.AlignTop)

def _save_step_feed(self):
    try:
        step = float(self.step_size.text())
        feed = float(self.feed_rate.text())
        self.cfg.remember_step_feed(step, feed)

```

```
except ValueError:
    # silencioso – validação já existe
    return
```

```
def _on_direction_press(self, axis: str, direction: int):
```

```
    """
```

```
    Chamada quando o usuário pressiona um botão (ou tecla).
    Decide entre STEP ou JOG e delega ao GRBLCNCController.
```

```
    """
```

```
    if not self._precheck_connected():
```

```
        return
```

```
    feed = self._get_feed_rate()
```

```
    step = self._get_step_size()
```

```
    if feed is None or step is None:
```

```
        return
```

```
    # --- grava imediatamente no JSON ---
```

```
    self.cfg.remember_step_feed(step, feed)
```

```
    # “Passo-a-passo” = botão G90 selecionado → usa step_move
```

```
    if self.mode_absolute.isChecked():
```

```
        self.controller.cnc.step_move(axis, step * direction, feed)
```

```
    # “Contínuo” = G91 selecionado → jog
```

```
    else:
```

```
        self.controller.cnc.jog_start(axis, direction, feed)
```

```
def _on_direction_release(self):
```

```
    """Interrompe jog se estivermos em modo contínuo."""
```

```
    if not self._precheck_connected():
```

```
        return
```

```
    if self.mode_relative.isChecked():    # só há jog se G91
```

```
        self.controller.cnc.jog_stop()
```

```
def start_movement(self, axis: str, direction: int):
```

```
    """
```

```
    Mantido apenas para chamadas vindas de eventFilter (atalhos de
    teclado). Encaminha para _on_direction_press.
```

```
    """
```

```
    self._on_direction_press(axis, direction)
```

```
def _get_feed_rate(self) -> float | None:
```

```
    """Lê o feed-rate; devolve None e avisa em caso de erro."""
```

```
    try:
```

```
        val = float(self.feed_rate.text())
```

```
    if self.controller.cnc.is_connected:
```

```
        max_lim = max(self.controller.cnc.max_feed.values())
```

```
        if val > max_lim + 1e-3:
```

```
            resp = QMessageBox.question(
                self, "Feed acima do limite",
```

```

(f"O valor F={val:.0f} mm/min excede o limite atual "
 f"(<= {max_lim:.0f}).\n\n"
 "Deseja atualizar $110 e $111 para permitir essa "
 "velocidade em X e Y?"),
 QMessageBox.StandardButton.Yes | QMessageBox.StandardButton.No
)
if resp == QMessageBox.StandardButton.Yes:
    # novo limite com folga de 10 %
    new_lim = int(val * 1.1)          # 10 % de folga
    new_acc = int((new_lim / 60) * 5)  # ≈5 s para atingir Vmáx

    cmds = [
        f"$110={new_lim}", f"$111={new_lim}",
        f"$120={new_acc}", f"$121={new_acc}"
    ]
    for c in cmds:
        self.controller.cnc.send_command(c, priority=True)

    # actualiza cache interno
    self.controller.cnc.max_feed['x'] = new_lim
    self.controller.cnc.max_feed['y'] = new_lim
    self.controller.cnc.max_acc ['x'] = new_acc
    self.controller.cnc.max_acc ['y'] = new_acc
    # desabilita avisos futuros de clamp
    self.controller.cnc._feed_clamp_warned = True
    QMessageBox.information(
        self, "Limites actualizados",
        (f"Feed-máx X/Y = {new_lim} mm/min\n"
         f"Aceleração X/Y = {new_acc} mm/s²")
    )
else:
    # clampará ao valor máximo existente
    QMessageBox.information(self, "Feed ajustado",
                            f"A velocidade será limitada a {max_lim} mm/min.")
    val = max_lim
    return val
except ValueError:
    QMessageBox.warning(self, "Erro", "Feed-rate inválido")
    return None

def _get_step_size(self) -> float | None:
    """Lê o step-size; devolve None e avisa em caso de erro."""
    try:
        return float(self.step_size.text())
    except ValueError:
        QMessageBox.warning(self, "Erro", "Step size inválido")
        return None

def _precheck_connected(self) -> bool:
    if not self.controller.cnc.is_connected:
        QMessageBox.warning(self, "Erro", "CNC não conectada")

```

```
    return False
return True
```

```
def go_to_zero(self):
    """
    Dispara homing no CLP: envia pulso de 100 ms em cada coil de sensor:
    - M1350 → eixo X
    - M850 → eixo Y
    - M185 → eixo Z
    """
    if not self._precheck_connected():
        return
    # PLC backend → pulso de 100 ms em cada coil de homing
    if isinstance(self.controller.cnc, PLCAxisController):
        status = self.controller.cnc.machine_status
        if status in ("Run", "Jog", "Alarm"):
            QMessageBox.warning(self, "Aviso", f"Máquina ocupada ({status})")
            return
    # Endereços dos coils de homing
    homing_coils = {
        'X': 1350, # M1350_X
        'Y': 850, # M850_Y
        'Z': 1850 # M185_Z
    }
    for axis, coil in homing_coils.items():
        try:
            # sobe borda
            self.controller.cnc.client.write_coil(coil, True)
            time.sleep(0.1)
            # desce borda
            self.controller.cnc.client.write_coil(coil, False)
        except Exception as e:
            logger.error(f"Homing CLP: falha no pulso de {axis} (coil {coil}): {e}")
    # aguarda término de todos os eixos
    self.controller.cnc.wait_for_idle()
    # atualiza interface
    self.window().update_position_display()
    self.window().statusBar().showMessage("Homing CLP concluído")
    return
    # GRBL or other → fallback ao "go to zero" por movimento absoluto
    # evita travar se já em movimento/alarm
    status = self.controller.cnc.machine_status
    if status in ("Run", "Jog", "Alarm"):
        QMessageBox.warning(self, "Aviso", f"Máquina ocupada ({status})")
        return
    feed = self._get_feed_rate()
    if feed is None:
        return
    self._start_move_thread(x=0, y=0, z=0, feed=feed,
                           status_msg="Movendo para posição zero")
```

```

def get_current_feed_rate(self):
    """Método público para outras classes acessarem a velocidade configurada"""
    try:
        return float(self.feed_rate.text())
    except ValueError:
        return 1000.0 # fallback

def stop_movement(self):
    self._on_direction_release()

def show_go_to_dialog(self):
    """Exibe diálogo para coletar coordenadas de destino (WPos)."""
    from PyQt6.QtWidgets import QDialog, QLabel, QLineEdit, QHBoxLayout, QPushButton, QMessageBox
    from PyQt6.QtGui import QDoubleValidator

    dialog = QDialog(self)
    dialog.setWindowTitle("Go to Position")
    dialog.setModal(True)
    layout = QGridLayout(dialog)

    # Captura a posição de trabalho atual (WPos) para pré-preencher os campos
    try:
        current = self.controller.cnc.get_current_position()
    except Exception:
        current = {'x': 0.0, 'y': 0.0}

    layout.addWidget(QLabel("X (mm):"), 0, 0)
    x_input = QLineEdit()
    x_input.setValidator(QDoubleValidator(-10000.0, 10000.0, 4, x_input))
    x_input.setText(f"{current.get('x', 0.0):.3f}")
    layout.addWidget(x_input, 0, 1)

    layout.addWidget(QLabel("Y (mm):"), 1, 0)
    y_input = QLineEdit()
    y_input.setValidator(QDoubleValidator(-10000.0, 10000.0, 4, y_input))
    y_input.setText(f"{current.get('y', 0.0):.3f}")
    layout.addWidget(y_input, 1, 1)

    btn_layout = QHBoxLayout()
    ok_btn = QPushButton("OK")
    cancel_btn = QPushButton("Cancel")
    btn_layout.addWidget(ok_btn)
    btn_layout.addWidget(cancel_btn)
    layout.addLayout(btn_layout, 2, 0, 1, 2)

    ok_btn.clicked.connect(dialog.accept)
    cancel_btn.clicked.connect(dialog.reject)

    if dialog.exec() == QDialog.DialogCode.Accepted:
        try:
            x = float(x_input.text())

```

```

        y = float(y_input.text())
        self.go_to_position(x, y)
    except ValueError:
        QMessageBox.warning(self, "Erro", "Valores inválidos para X ou Y")

```

```

def go_to_position(self, x, y):
    """Realiza movimento para a posição absoluta de trabalho (WPos)."""
    from PyQt6.QtWidgets import QMessageBox
    from PyQt6.QtCore import QTimer

    # 1. Verifica se há conexão
    if not self.controller.cnc.is_connected:
        QMessageBox.warning(self, "Erro", "CNC não conectada")
        return

    # 2. Verifica se a máquina está livre
    status = self.controller.cnc.machine_status
    if status in ("Alarm", "Run", "Jog"):
        QMessageBox.warning(self, "Aviso", f"Máquina ocupada ({status})")
        return

    # 3. Guarda modo de distância atual (G90 ou G91)
    prev_mode = "G90" if self.mode_absolute.isChecked() else "G91"

    # 4. Obtém feed rate
    try:
        feed_rate = float(self.feed_rate.text())
    except:
        feed_rate = 1000

    # 5. Deslocamento assíncrono
    self._start_move_thread(x, y, feed_rate,
                           status_msg=f"Movendo para X:{x:.3f}, Y:{y:.3f}")

def _start_move_thread(self, x=None, y=None, z=None,
                      feed=1000, status_msg="Movendo..."):
    stbar = self.window().statusBar()
    stbar.showMessage(status_msg)

    self._move_thread = MoveTaskThread(
        self.controller.cnc, x, y, z, feed
    )

def _on_done(xx, yy, zz):
    stbar.showMessage(f"Head em X:{xx:.3f}, Y:{yy:.3f}, Z:{zz:.3f}")
    # força atualização UI
    QTimer.singleShot(50, self.window().update_position_display)

def _on_err(msg):
    QMessageBox.critical(self, "Erro", msg)
    stbar.showMessage("Falha no deslocamento")

```

```
self._move_thread.finished.connect(_on_done)
self._move_thread.error.connect(_on_err)
self._move_thread.start()
```

```
def _execute_resume_and_update(self):
```

```
    """Executa o resumo após parada e força atualização de posição em sequência"""
```

```
    try:
```

```
        # Primeiro envia o comando para retomar após hold
```

```
        self.controller.cnc.grbl.send_immediately("~")
```

```
        logger.debug("MOVIMENTO: Enviado comando de retomada (~)")
```

```
        # --- INÍCIO DA MODIFICAÇÃO ---
```

```
        # REMOVIDO: Chamada para _force_position_update. A atualização agora
```

```
        # dependerá do polling regular iniciado em connect_cnc.
```

```
        # QTimer.singleShot(150, lambda: self._force_position_update(1))
```

```
        logger.debug("MOVIMENTO: Atualização de posição dependerá do polling regular.")
```

```
        # --- FIM DA MODIFICAÇÃO ---
```

```
    except Exception as e:
```

```
        logger.error(f"MOVIMENTO: Erro ao executar sequência de retomada: {e}")
```

```
# A função _force_position_update pode ser mantida, mas não será mais chamada
```

```
# a partir de _execute_resume_and_update.
```

```
def _force_position_update(self, attempt=1):
```

```
    """Força múltiplas atualizações de posição para garantir precisão
```

```
    Args:
```

```
        attempt: Número da tentativa atual (para limitar tentativas)
```

```
    """
```

```
    if not hasattr(self.controller.cnc, 'grbl') or not self.controller.cnc.is_connected:
```

```
        return
```

```
    try:
```

```
        # Envia comando de status para obter posição atualizada
```

```
        # --- INÍCIO DA MODIFICAÇÃO ---
```

```
        # logger.debug(f"MOVIMENTO: Forçando atualização de posição - tentativa {attempt}") # REMOVIDO/COMENTADO
```

```
        # --- FIM DA MODIFICAÇÃO ---
```

```
        self.controller.cnc.grbl.send_immediately("?")
```

```
        # Se ainda estamos dentro do limite de tentativas, agenda outra verificação
```

```
        if attempt < 3:
```

```
            # Aumento progressivo do tempo entre tentativas (150ms, 200ms, 250ms)
```

```
            delay = 150 + (attempt * 50)
```

```
            # A chamada recursiva ainda existe, mas o log dentro dela foi removido
```

```
            QTimer.singleShot(delay, lambda: self._force_position_update(attempt + 1))
```

```
    except Exception as e:
```

```
        # Manter o log de erro
```

```
        logger.error(f"MOVIMENTO: Erro ao forçar atualização de posição: {e}")
```

```

def set_motion_mode(self, mode):
    """Set the motion mode (G90/G91)"""
    # Se for PLC, não existe GRBL: apenas atualiza UI e guarda modo internamente

    if isinstance(self.controller.cnc, PLCAxisController):
        if mode == "G90":
            self.mode_absolute.setChecked(True)
            self.mode_relative.setChecked(False)
        else:
            self.mode_absolute.setChecked(False)
            self.mode_relative.setChecked(True)
        # opcional: armazenar no controller para referência futura
        self.controller.cnc.current_motion_mode = mode
        logger.info(f"MOVIMENTO (PLC): modo de movimento definido para {mode}")
        return

    # Se não estiver conectado (nem PLC, nem GRBL), só atualiza UI e volta
    if not self.controller.cnc.is_connected:
        if mode == "G90":
            self.mode_absolute.setChecked(True)
            self.mode_relative.setChecked(False)
        else:
            self.mode_absolute.setChecked(False)
            self.mode_relative.setChecked(True)
        logger.warning(f"MOVIMENTO: CNC não conectada, modo {mode} definido apenas na UI.")
        return

    # Atualiza a UI (GRBL)
    if mode == "G90":
        self.mode_absolute.setChecked(True)
        self.mode_relative.setChecked(False)
    else: # G91
        self.mode_absolute.setChecked(False)
        self.mode_relative.setChecked(True)
    # Envia o comando G90/G91 para o GRBL
    try:
        logger.debug(f"MOVIMENTO: Definindo modo de movimento para {mode}")
        self.controller.cnc.grbl.send_immediately(mode)
    except Exception as e:
        logger.error(f"MOVIMENTO: Erro ao definir modo de movimento: {e}")
        QMessageBox.warning(self, "Error", f"Error setting motion mode: {e}")

def on_emergency_stop_toggle(self, checked):
    """
    Manipula o clique no botão Emergency Stop/Reset.
    """
    if not hasattr(self.controller.cnc, 'is_connected') or not self.controller.cnc.is_connected:
        QMessageBox.warning(self, "Erro", "CNC não conectada")
        self.emergency_stop_button.setChecked(not checked)
    return

```

if checked:

Botão foi pressionado para entrar no modo RESET

logger.info("EMERGENCY STOP: Botão pressionado. Enviando Soft Reset.")

if self.controller.cnc.send_soft_reset():

Configura visual do botão para Reset

self.emergency_stop_button.setText("Reset")

self.emergency_stop_button.setStyleSheet("background-color: orange; color: black;")

Atualiza status

main_window = self.window()

if hasattr(main_window, 'statusBar'):

main_window.statusBar().showMessage("Máquina parada. Clique em Reset para desbloquear.")

else:

Soft reset falhou, reverte o botão

logger.error("EMERGENCY STOP: Falha ao enviar Soft Reset")

QMessageBox.critical(self, "Erro", "Falha ao enviar comando de parada")

self.emergency_stop_button.setChecked(False)

else:

Botão foi pressionado para sair do modo RESET

logger.info("RESET: Botão pressionado para desbloquear. Enviando \$X.")

if self.controller.cnc.unlock():

Configura visual do botão para STOP

self.emergency_stop_button.setText("STOP")

self.emergency_stop_button.setStyleSheet("background-color: red; color: white;")

Atualiza status

main_window = self.window()

if hasattr(main_window, 'statusBar'):

main_window.statusBar().showMessage("Máquina desbloqueada e pronta.")

Solicita atualização de status para verificar nova condição

QTimer.singleShot(200, lambda: self.controller.cnc.grbl.send_immediately("?"))

else:

Desbloqueio falhou, reverte o botão

logger.error("RESET: Falha ao enviar comando de desbloqueio")

QMessageBox.critical(self, "Erro", "Falha ao desbloquear a máquina")

self.emergency_stop_button.setChecked(True)

def _auto_unlock_after_reset(self):

"""Executa sequência automática de desbloqueio após reset de emergência"""

logger.info("AUTO UNLOCK: Iniciando sequência de desbloqueio automático após reset")

try:

1. Envia comando de desbloqueio

self.controller.cnc.grbl.send_immediately("\$X")

logger.info("AUTO UNLOCK: Comando \$X enviado")

2. Pequena pausa

QTimer.singleShot(200, lambda: self._check_if_unlocked())

except Exception as e:

logger.error(f"AUTO UNLOCK: Erro ao enviar comando de desbloqueio: {e}")

```

main_window = self.window()
if hasattr(main_window, 'statusBar'):
    main_window.statusBar().showMessage(f"Erro no desbloqueio automático: {e}")

```

```

def _check_if_unlocked(self):

```

```

    """Verifica se o desbloqueio foi bem-sucedido e restaura configurações se necessário"""

```

```

    try:

```

```

        # Solicita status para conferir se saiu do alarme
        self.controller.cnc.grbl.send_immediately("?")

```

```

        # Exibe mensagem de sucesso

```

```

        main_window = self.window()

```

```

        if hasattr(main_window, 'statusBar'):

```

```

            main_window.statusBar().showMessage("Sistema parado e desbloqueado automaticamente.")

```

```

        logger.info("AUTO UNLOCK: Sequência de desbloqueio automático concluída")

```

```

    except Exception as e:

```

```

        logger.error(f"AUTO UNLOCK: Erro ao verificar status após desbloqueio: {e}")

```

```

@dataclass

```

```

class MapParams:

```

```

    origin: dict

```

```

    end: dict

```

```

    step_x: float

```

```

    step_y: float

```

```

    folder: str

```

```

    program_name: str

```

```

class _PreviewSuspend:

```

```

    """

```

```

    Context-manager que pausa o preview da câmera e garante reativação
    mesmo em caso de exceções.

```

```

    """

```

```

    def __init__(self, preview_widget):

```

```

        self.preview_widget = preview_widget

```

```

        self.was_running = preview_widget and preview_widget.preview_timer.isActive()

```

```

    def __enter__(self):

```

```

        if self.was_running:

```

```

            self.preview_widget.stop_preview()

```

```

    def __exit__(self, exc_type, exc_val, exc_tb):

```

```

        if self.was_running:

```

```

            self.preview_widget.start_preview()

```

```

class AOIControllerApp(QMainWindow):

```

```

    def __init__(self):

```

```

        super().__init__()

```

```

        self.setWindowTitle("Controle de Inspeção Óptica Automatizada")

```

```

        self.setGeometry(100, 100, 1200, 800)

```

```

# Carrega configurações do usuário
self.config = AOIConfigManager()

# Inicializa o controlador AOI usando CLP (Modbus TCP),
# mas não trava a aplicação se a conexão falhar
plc_host = self.config.get("connections", "plc_host", default="192.168.0.5")
plc_port = self.config.get("connections", "plc_port", default=502)
# Inicializa o controlador com PLC mas sem conectar automaticamente
logger.debug("Inicializando CNCAOIController com PLCAxisController (sem conexão automática)")
self.controller = CNCAOIController(
    use_plc=True,
    plc_host=plc_host,
    plc_port=plc_port
)
logger.debug("CNCAOIController(use_plc=True) inicializado; backend = %s",
    type(self.controller.cnc).__name__)
self.current_sequence = None
self.is_running_sequence = False

# Variável para armazenar o último valor de posição (para comparação)
self.last_logged_position = None

# Armazenar o offset do sistema de coordenadas de trabalho (WCS) ativo (ex: G54)
self.current_wcs_offset = {'x': 0.0, 'y': 0.0, 'z': 0.0} # Inicializa o offset WCS padrão (G54)

# Armazenar a última Posição da Máquina (MPos) conhecida
self.current_mpos = {'x': 0.0, 'y': 0.0, 'z': 0.0}
# Armazenar o Sistema de Coordenadas de Trabalho (WCS) ativo (ex: "G54")
self.active_wcs = "G54" # Assume G54 como padrão inicial
# Mapeamento de WCS para número P do G10
self.wcs_to_p = {"G54": 1, "G55": 2, "G56": 3, "G57": 4, "G58": 5, "G59": 6}

# Configuração da interface
self.setup_ui()
# Configuração do menu
self.setup_menu()

# ----- Painel de conexão inicialmente oculto -----
self.connection_group.setVisible(False)

# ----- Auto-connect se preferido -----

logger.debug(
    "Verificando conexão PLC no arranque; backend=%s, conectado=%s",
    type(self.controller.cnc).__name__,
    getattr(self.controller.cnc, 'is_connected', False)
)
if isinstance(self.controller.cnc, PLCAxisController):
    # PLC não conectado - aguarda conexão manual
    self.connect_cnc_btn.setText("Conectar PLC")

```

```

self.cnc_status.setText("Desconectado")
self.statusBar().showMessage("PLC desconectado - clique em 'Conectar PLC' no menu Conexões")
logger.info("PLC não conectado. Aguardando conexão manual pelo usuário.")

# Auto-connect apenas após a interface estar pronta
QTimer.singleShot(500, self._attempt_auto_connect)

# Timer para atualizar a posição – agora conectamos a um método que loga a ação
self.update_timer = QTimer(self)
self.update_timer.timeout.connect(self.on_update_timer)
self.update_timer.start(1000) # Atualiza a cada 1000ms

# Capturar eventos de teclado para movimentação de qualquer widget:
# instala o filter globalmente apenas UMA vez
QApplication.instance().installEventFilter(self)

# chama cleanup se o Qt encerrar por outros caminhos
QApplication.instance().aboutToQuit.connect(self._cleanup_resources)

# Auto-connect com base no JSON de prefs
def _attempt_auto_connect(self):
    # CNC
    if self.config.get("connections", "auto_connect_cnc", default=False):
        port = self.config.get("connections", "last_cnc_port", default="")
        if port:
            idx = self.cnc_port_combo.findText(port)
            if idx >= 0:
                self.cnc_port_combo.setCurrentIndex(idx)
                QTimer.singleShot(100, self.connect_cnc)
    # Câmera
    if self.config.get("connections", "auto_connect_camera", default=False):
        cam_id = int(self.config.get("connections", "last_camera_id", default=0))
        idx = self.camera_id_combo.findText(str(cam_id))
        if idx >= 0:
            self.camera_id_combo.setCurrentIndex(idx)
            QTimer.singleShot(200, self.connect_camera)

def eventFilter(self, source, event):
    """
    Intercepta eventos de teclado e dispara start/stop de movimento
    se a opção estiver habilitada.
    """
    # KeyPress
    if event.type() == QEvent.Type.KeyPress and self.movement_widget.keyboard_control_checkbox.isChecked():
        if hasattr(event, 'isAutoRepeat') and event.isAutoRepeat():
            return True
        key = event.key()
        if key == Qt.Key.Key_Up:
            self.movement_widget.start_movement("Y", -1)
            return True

```

```

elif key == Qt.Key.Key_Down:
    self.movement_widget.start_movement("Y", 1)
    return True
elif key == Qt.Key.Key_Left:
    self.movement_widget.start_movement("X", -1)
    return True
elif key == Qt.Key.Key_Right:
    self.movement_widget.start_movement("X", 1)
    return True
elif key == Qt.Key.Key_PageUp:
    self.movement_widget.start_movement("Z", -1)
    return True
elif key == Qt.Key.Key_PageDown:
    self.movement_widget.start_movement("Z", 1)
    return True
# KeyRelease
elif event.type() == QEvent.Type.KeyRelease and self.movement_widget.keyboard_control_checkbox.isChecked():
    if hasattr(event, 'isAutoRepeat') and event.isAutoRepeat():
        return True
    key = event.key()
    if key in (
        Qt.Key.Key_Up,
        Qt.Key.Key_Down,
        Qt.Key.Key_Left,
        Qt.Key.Key_Right,
        Qt.Key.Key_PageUp,
        Qt.Key.Key_PageDown
    ):
        self.movement_widget.stop_movement()
        return True
return super().eventFilter(source, event)

def on_update_timer(self):
    #logger.debug("Timer fired: atualizando tela de posição da head.")
    self.update_position_display()

def setup_ui(self):
    # Widget central
    central_widget = QWidget()
    self.setCentralWidget(central_widget)

    # Layout principal
    main_layout = QVBoxLayout(central_widget)

    # Grupo de conexão (oculto por padrão; mostrado via menu)
    self.connection_group = QGroupBox("Conexão")
    connection_layout = QGridLayout()

    # CNC Connection
    connection_layout.addWidget(QLabel("Porta CNC:"), 0, 0)
    self.cnc_port_combo = QComboBox()

```

```

self.refresh_ports()
connection_layout.addWidget(self.cnc_port_combo, 0, 1)

self.connect_cnc_btn = QPushButton("Conectar CNC")
self.connect_cnc_btn.clicked.connect(self.connect_cnc)
connection_layout.addWidget(self.connect_cnc_btn, 0, 2)

# Camera connection
connection_layout.addWidget(QLabel("Câmera ID:"), 1, 0)
self.camera_id_combo = QComboBox()
self.camera_id_combo.addItems(["0", "1", "2", "3"])
connection_layout.addWidget(self.camera_id_combo, 1, 1)

self.connect_camera_btn = QPushButton("Conectar Câmera")
self.connect_camera_btn.clicked.connect(self.connect_camera)
connection_layout.addWidget(self.connect_camera_btn, 1, 2)

# Refresh ports button
self.refresh_ports_btn = QPushButton("Atualizar Portas")
self.refresh_ports_btn.clicked.connect(self.refresh_ports)
connection_layout.addWidget(self.refresh_ports_btn, 0, 3)

# Test camera button
self.test_camera_btn = QPushButton("Testar Câmera")
self.test_camera_btn.clicked.connect(self.test_camera)
connection_layout.addWidget(self.test_camera_btn, 1, 3)

self.connection_group.setLayout(connection_layout)
main_layout.addWidget(self.connection_group)

# Grupo de Calibração de Movimento -----
calibration_group = QGroupBox("Calibração de Movimento")
calibration_layout = QHBoxLayout()

# Campos permanecem criados porque são usados pela
# lógica de calibração, porém o grupo ficará oculto.
self.pulses_input = QLineEdit(str(
    self.config.get("calibration", "pulses_per_rev", default=400)
))
self.fuso_input = QLineEdit(str(
    self.config.get("calibration", "fuso_pitch", default=5)
))
self.apply_calibration_btn = QPushButton("Aplicar Calibração")
self.apply_calibration_btn.clicked.connect(self.apply_calibration)

# (os widgets não são adicionados ao layout visual)
calibration_group.setLayout(calibration_layout)
calibration_group.setVisible(False)    # ← esconde
main_layout.addWidget(calibration_group) # mantém no DOM para uso interno

# Splitter para dividir a interface em painéis

```

```
splitter = QSplitter(Qt.Orientation.Horizontal)
```

```
# Painel esquerdo: controles e lista de posições
```

```
left_panel = QWidget()
```

```
left_layout = QVBoxLayout(left_panel)
```

```
# Informações de posição
```

```
position_group = QGroupBox("Posição Atual")
```

```
position_layout = QGridLayout()
```

```
# Exibe os valores X, Y, Z e status
```

```
position_layout.addWidget(QLabel("X:"), 0, 0)
```

```
self.x_position = QLabel("0.000 mm")
```

```
position_layout.addWidget(self.x_position, 0, 1)
```

```
position_layout.addWidget(QLabel("Y:"), 1, 0)
```

```
self.y_position = QLabel("0.000 mm")
```

```
position_layout.addWidget(self.y_position, 1, 1)
```

```
position_layout.addWidget(QLabel("Z:"), 2, 0)
```

```
self.z_position = QLabel("0.000 mm")
```

```
position_layout.addWidget(self.z_position, 2, 1)
```

```
position_layout.addWidget(QLabel("Status:"), 3, 0)
```

```
self.cnc_status = QLabel("Desconectado")
```

```
position_layout.addWidget(self.cnc_status, 3, 1)
```

```
position_group.setLayout(position_layout)
```

```
left_layout.addWidget(position_group)
```

```
# Widget de lista de posições
```

```
self.position_list_widget = PositionListWidget()
```

```
self.position_list_widget.add_position_btn.clicked.connect(self.add_current_position)
```

```
self.position_list_widget.remove_position_btn.clicked.connect(self.remove_position)
```

```
self.position_list_widget.position_selected.connect(self.on_position_selected)
```

```
left_layout.addWidget(self.position_list_widget)
```

```
# Controle de sequência
```

```
self.sequence_widget = SequenceControlWidget()
```

```
self.sequence_widget.create_sequence_btn.clicked.connect(self.create_sequence)
```

```
self.sequence_widget.run_sequence_btn.clicked.connect(self.run_sequence)
```

```
self.sequence_widget.stop_sequence_btn.clicked.connect(self.stop_sequence)
```

```
self.sequence_widget.save_btn.clicked.connect(self.save_program)
```

```
self.sequence_widget.load_btn.clicked.connect(self.load_program)
```

```
self.sequence_widget.save_gcode_btn.clicked.connect(self.save_gcode)
```

```
self.sequence_widget.load_gcode_btn.clicked.connect(self.load_gcode)
```

```
left_layout.addWidget(self.sequence_widget)
```

```
# Table to display each step/result of the executed sequence
```

```
self.results_table = QTableWidgetItem(0, 3)
```

```
self.results_table.setHorizontalHeaderLabels(["Posição", "Horário", "Status"])
```

```
# Make columns stretch to fill available space
```

```

self.results_table.horizontalHeader().setSectionResizeMode(
    QHeaderView.ResizeMode.Stretch
)
left_layout.addWidget(self.results_table)

# Painel direito: aba para Câmera & Movimento e Visualização de Tensão
right_panel = QTabWidget()
# guardamos para habilitar/desabilitar abas depois
self.right_panel = right_panel

# MOVER PARA AQUI: Adicionar a aba de Câmera & Movimento (após definir right_panel)
# Tab para Camera & Movement
camera_movement_tab = QWidget()
camera_movement_layout = QHBoxLayout(camera_movement_tab)

# Left side: controls and position registry
cm_left_panel = QWidget()
cm_left_layout = QVBoxLayout(cm_left_panel)

# Movement controls
self.movement_widget = MovementControlWidget(self.controller, self.config)
cm_left_layout.addWidget(self.movement_widget)

# Right side: camera preview
self.camera_preview = CameraPreviewWidget(self.controller, self.config)
self.camera_preview.image_captured.connect(self.on_image_captured)

# Inverter colunas: preview à esquerda (mais espaço) e controles à direita
camera_movement_layout.addWidget(self.camera_preview, 3)    # Proporção 3 (preview)
camera_movement_layout.addWidget(cm_left_panel, 1)          # Proporção 1 (controles)

# Agora é seguro adicionar a nova aba ao right_panel que já foi definido
right_panel.addTab(camera_movement_tab, "Câmera & Movimento")

# Aba de Visualização de Tensão
self.tension_visualization = TensionVisualizationWidget()
right_panel.addTab(self.tension_visualization, "Visualização de Tensão")
# desabilita abas até o CLP conectar
self.right_panel.setTabEnabled(0, False)
self.right_panel.setTabEnabled(1, False)

# Adiciona painéis ao splitter
splitter.addWidget(left_panel)
splitter.addWidget(self.right_panel)
splitter.setSizes([400, 800])

main_layout.addWidget(splitter)

# Barra de status
self.statusBar().showMessage("Pronto para conectar")

```

```

def setup_menu(self):
    """Configura o menu da aplicação"""
    menubar = self.menuBar()

    # Menu de Arquivo
    file_menu = menubar.addMenu('&Arquivo')

    exit_action = QAction('Sair', self)
    exit_action.setShortcut('Ctrl+Q')
    exit_action.triggered.connect(self.close)
    file_menu.addAction(exit_action)

    # Menu de Ferramentas
    tools_menu = menubar.addMenu('&Ferramentas')

    # ação para definir mapa
    definir_mapa_action = QAction('Definir Mapa', self)
    definir_mapa_action.triggered.connect(self.show_definir_mapa_dialog)
    tools_menu.addAction(definir_mapa_action)

    calibration_action = QAction('Calibração CNC', self)
    calibration_action.triggered.connect(self.show_calibration_dialog)
    tools_menu.addAction(calibration_action)

    # Preferências
    pref_action = QAction('Preferências', self)
    pref_action.setShortcut('Ctrl+,')
    pref_action.triggered.connect(self.show_settings_dialog)
    tools_menu.addAction(pref_action)

    # -----
    # Item de menu "Tensão do Stencil" – abre o diálogo de medição
    # -----
    tension_action = QAction('Tensão do Stencil', self)
    tension_action.triggered.connect(self.open_stencil_tension_dialog)
    menubar.addAction(tension_action)

    # ----- painel de conexões -----
    conn_panel = QAction('Conexões...', self)
    conn_panel.setCheckable(True)
    conn_panel.setChecked(False)
    conn_panel.triggered.connect(
        lambda checked: self.connection_group.setVisible(checked)
    )
    tools_menu.addAction(conn_panel)

    # Menu de Ajuda
    help_menu = menubar.addMenu('&Ajuda')

    about_action = QAction('Sobre', self)

```

```

about_action.triggered.connect(self.show_about_dialog)
help_menu.addAction(about_action)

# abre o diálogo de medição de tensão
def open_stencil_tension_dialog(self):
    if not self.controller.cnc.is_connected:
        QMessageBox.warning(self, "Aviso", "Conecte a CNC antes de medir a tensão do stencil.")
        return
    dlg = StencilTensionDialog(self, self.controller.cnc)
    dlg.exec()

def show_settings_dialog(self):
    dlg = SettingsDialog(self.config, self)
    if dlg.exec():
        # Se o usuário modificou algo, re-aplica (se a CNC já estiver conectada)
        if self.controller.cnc.is_connected:
            self.config.apply_to_cnc(self.controller.cnc)
            self.statusBar().showMessage("Preferências salvas")

def show_calibration_dialog(self):
    """Mostra um diálogo para configuração de calibração"""
    dialog = QDialog(self)
    dialog.setWindowTitle("Calibração do Sistema CNC")
    dialog.setMinimumWidth(500)

    layout = QVBoxLayout(dialog)

    # Grupo de parâmetros físicos
    param_group = QGroupBox("Parâmetros da Máquina")
    param_layout = QGridLayout(param_group)

    param_layout.addWidget(QLabel("Pulsos por Revolução:"), 0, 0)
    pulses_input = QLineEdit(self.pulses_input.text())
    param_layout.addWidget(pulses_input, 0, 1)

    param_layout.addWidget(QLabel("Passo do Fuso (mm):"), 1, 0)
    fuso_input = QLineEdit(self.fuso_input.text())
    param_layout.addWidget(fuso_input, 1, 1)

    param_layout.addWidget(QLabel("Steps/mm calculado:"), 2, 0)
    steps_mm_result = QLabel("Calculando...")
    param_layout.addWidget(steps_mm_result, 2, 1)

    # Atualiza o cálculo quando os valores mudam
    def update_calculation():
        try:
            pulses = float(pulses_input.text())
            fuso = float(fuso_input.text())
            steps_mm = pulses / fuso
            steps_mm_result.setText(f"{steps_mm:.3f} steps/mm")
        except:

```

```

        steps_mm_result.setText("Erro no cálculo")

    pulses_input.textChanged.connect(update_calculation)
    fuso_input.textChanged.connect(update_calculation)
    update_calculation() # Executa o cálculo inicial

    layout.addWidget(param_group)

# Botões de ação
buttons_layout = QHBoxLayout()

apply_btn = QPushButton("Aplicar Parâmetros")
def apply_and_close():
    self.pulses_input.setText(pulses_input.text())
    self.fuso_input.setText(fuso_input.text())
    dialog.accept()
    self.apply_calibration()
apply_btn.clicked.connect(apply_and_close)

test_btn = QPushButton("Testar Calibração")
test_btn.clicked.connect(lambda: [dialog.accept(), self.show_calibration_test_dialog()])

cancel_btn = QPushButton("Cancelar")
cancel_btn.clicked.connect(dialog.reject)

buttons_layout.addWidget(apply_btn)
buttons_layout.addWidget(test_btn)
buttons_layout.addWidget(cancel_btn)

layout.addLayout(buttons_layout)

dialog.exec()

def show_about_dialog(self):
    """Mostra informações sobre o aplicativo"""
    QMessageBox.about(self, "Sobre HesaiVision",
        "HesaiVision v1.0\n\n"
        "Sistema de Inspeção Óptica Automatizada\n\n"
        "Desenvolvido para controle de CNC com GRBL\n\n"
        "© 2025 HesaiVision")

def show_definir_mapa_dialog(self):
    """Abre diálogo para definir cantos e gerar mapa (modeless, sempre no topo)."""
    # 1) Cria sem flags inválidas
    dialog = QDialog(self)
    dialog.setWindowTitle("Definir Mapa")

    # 2) Non-modal: permite operar a janela principal
    dialog.setWindowModality(Qt.WindowModality.NonModal)

    # 3) Sempre no topo, com título e botão de fechar

```

```

dialog.setWindowFlags(
    dialog.windowFlags()
    | Qt.WindowType.WindowTitleHint
    | Qt.WindowType.WindowCloseButtonHint
    | Qt.WindowType.WindowStaysOnTopHint
)

layout = QVBoxLayout(dialog)

# Nome do programa
h1 = QHBoxLayout()
h1.addWidget(QLabel("Nome do Programa:"))
self.map_program_name_edit = QLineEdit()
h1.addWidget(self.map_program_name_edit)
layout.addLayout(h1)

# Pasta de salvamento
h2 = QHBoxLayout()
h2.addWidget(QLabel("Pasta de Salvamento:"))
self.map_folder_edit = QLineEdit()
h2.addWidget(self.map_folder_edit)
btn_browse = QPushButton("Buscar...")
btn_browse.clicked.connect(self._select_map_folder)
h2.addWidget(btn_browse)
layout.addLayout(h2)

# Passos X/Y
h3 = QHBoxLayout()
h3.addWidget(QLabel("Passo X (mm):"))
self.map_step_x_edit = QLineEdit("10")
h3.addWidget(self.map_step_x_edit)
h3.addWidget(QLabel("Passo Y (mm):"))
self.map_step_y_edit = QLineEdit("10")
h3.addWidget(self.map_step_y_edit)
layout.addLayout(h3)

# Botões de definição de canto
btn_origin = QPushButton("Definir canto inferior esquerdo")
btn_origin.clicked.connect(lambda: self._define_map_corner('origin'))
layout.addWidget(btn_origin)

btn_end = QPushButton("Definir canto superior direito")
btn_end.clicked.connect(lambda: self._define_map_corner('end'))
layout.addWidget(btn_end)

# Botão gerar mapa
btn_generate = QPushButton("Gerar Mapa")
btn_generate.clicked.connect(lambda: self._on_generate_map(dialog))
layout.addWidget(btn_generate)

dialog.show() # modeless, não bloqueia a janela principal

```

```

def _select_map_folder(self):
    folder = QFileDialog.getExistingDirectory(self, "Selecione pasta para salvar imagens")
    if folder:
        self.map_folder_edit.setText(folder)

def _define_map_corner(self, which):
    pos = self.controller.cnc.get_current_position()
    if which == 'origin':
        self.map_origin = {'x': pos['x'], 'y': pos['y']}
        QMessageBox.information(self, "Origem", f"Canto inferior esquerdo: X={pos['x']:.3f}, Y={pos['y']:.3f}")
    else:
        self.map_end = {'x': pos['x'], 'y': pos['y']}
        QMessageBox.information(self, "Limite", f"Canto superior direito: X={pos['x']:.3f}, Y={pos['y']:.3f}")

def _on_generate_map(self, dialog):

    # Passo 1 – coletar e validar parâmetros -----
    params = self._collect_map_params()
    if params is None:    # validação falhou ⇒ aborta
        return

    # Passo 2 – iniciar thread de geração -----
    self._start_map_thread(params, dialog)

def _collect_map_params(self) -> MapParams | None:
    """
    Valida inputs da UI e devolve objeto MapParams ou None em caso de erro.
    Toda mensagem ao usuário é tratada aqui.
    """
    origin = getattr(self, 'map_origin', None)
    end = getattr(self, 'map_end', None)
    if not origin or not end:
        QMessageBox.warning(self, "Erro", "Defina ambos os cantos antes de gerar o mapa.")
        return None

    try:
        step_x = float(self.map_step_x_edit.text())
        step_y = float(self.map_step_y_edit.text())
    except ValueError:
        QMessageBox.warning(self, "Erro", "Passos X/Y inválidos.")
        return None

    dx, dy = end['x'] - origin['x'], end['y'] - origin['y']
    if step_x <= 0 or step_y <= 0:
        QMessageBox.warning(self, "Erro", "Os passos devem ser maiores que zero.")
        return None

    # Ajuste opcional se o passo superar dimensão
    if step_x > dx or step_y > dy:
        if QMessageBox.question(

```

```

        self,
        "Passo maior que dimensão",
        ("Algum passo é maior que a dimensão da placa. "
        "Deseja ajustar automaticamente?"),
        QMessageBox.StandardButton.Yes | QMessageBox.StandardButton.No
    ) == QMessageBox.StandardButton.No:
        return None
    step_x = min(step_x, dx)
    step_y = min(step_y, dy)
    self.map_step_x_edit.setText(f"{step_x:.3f}")
    self.map_step_y_edit.setText(f"{step_y:.3f}")

    folder = self.map_folder_edit.text().strip()
    prog = self.map_program_name_edit.text().strip()
    if not folder or not prog:
        QMessageBox.warning(self, "Erro", "Informe o nome do programa e a pasta de salvamento.")
        return None

    return MapParams(origin, end, step_x, step_y, folder, prog)

def _start_map_thread(self, p: MapParams, dialog):
    """
    Separa a configuração da thread e da UI/ProgressBar.
    """
    # Context manager garante preview restaurado
    with _PreviewSuspend(self.camera_preview):
        self.map_thread = MapGeneratorThread(
            self.controller, p.origin, p.end,
            p.step_x, p.step_y, p.folder, p.program_name,
            self._get_current_feed_rate() # Passa velocidade configurada
        )

    # Progress dialog simples
    self.map_progress = QProgressDialog("Gerando mapa...", "Cancelar", 0, 0, self)
    self.map_progress.setWindowTitle("Progresso do Mapa")
    self.map_progress.setWindowModality(Qt.WindowModality.NonModal)
    self.map_progress.show()

    # Conexões de sinal ■ slots
    self.map_thread.progress.connect(self._on_map_progress)
    self.map_thread.image_captured.connect(self.camera_preview.display_image)
    self.map_thread.finished.connect(lambda: self._on_map_finished(dialog))
    self.map_thread.error.connect(self._on_map_error)

    self.map_progress.canceled.connect(self.map_thread.requestInterruption)
    self.map_thread.start()

def _get_current_feed_rate(self):
    """Obtém a velocidade de movimentação configurada na interface"""
    try:
        return float(self.movement_widget.feed_rate.text())
    except (ValueError, AttributeError):

```

```

        return 1000.0 # fallback

# ----- slots da geração de mapa -----

def _on_map_progress(self, done: int, total: int):
    self.map_progress.setMaximum(total)
    self.map_progress.setValue(done)
    pct = int(done / total * 100) if total else 0
    self.map_progress.setLabelText(f"Capturadas {done}/{total} imagens ({pct}%)")

def _on_map_finished(self, dialog):
    self.map_progress.close()
    QMessageBox.information(self, "Concluído", "Mapa gerado com sucesso.")
    dialog.accept()

def _on_map_error(self, msg: str):
    self.map_progress.close()
    QMessageBox.critical(self, "Erro", msg)

def save_gcode(self):
    """Salva a sequência atual como arquivo G-CODE"""
    if not self.current_sequence:
        QMessageBox.warning(self, "Aviso", "Crie uma sequência primeiro")
        return

    from aoi_lib.gcode_manager import GCodeManager
    gcode_manager = GCodeManager()

    filename, _ = QFileDialog.getSaveFileName(
        self, "Salvar como G-CODE", "", "Arquivos G-CODE (*.gcode *.nc *.ngc)"
    )

    if filename:
        if not filename.lower().endswith((''.gcode', '.nc', '.ngc')):
            filename += '.gcode'

        if gcode_manager.save_gcode_to_file(self.current_sequence, filename):
            self.statusBar().showMessage(f"G-CODE salvo em {filename}")
        else:
            QMessageBox.critical(self, "Erro", "Falha ao salvar o arquivo G-CODE")

def apply_calibration(self):
    """
    Aplica a calibração de movimento com base nos valores informados pelo usuário.
    Envia comandos diretos para o GRBL para configurar os steps/mm.
    """
    if not self.controller.cnc.is_connected:
        QMessageBox.warning(self, "Erro", "CNC não conectada. Conecte primeiro.")
        return

    # Se for PLC, atualiza apenas o fator de conversão interno

```

```

if isinstance(self.controller.cnc, PLCAxisController):
    try:
        pulses = float(self.pulses_input.text())
        fuso_pass = float(self.fuso_input.text())

        # Atualiza o fator de conversão pulsos/mm do PLC
        self.controller.cnc.pulses_per_mm = pulses / fuso_pass

        # Salva no JSON para persistir a configuração
        self.config.remember_calibration(pulses, fuso_pass)

        self.statusBar().showMessage(f"Calibração PLC aplicada: {pulses / fuso_pass:.3f} pulsos/mm")
        QMessageBox.information(
            self, "Calibração PLC",
            f"Fator de conversão atualizado:\n{pulses / fuso_pass:.3f} pulsos/mm"
        )
    except ValueError:
        QMessageBox.warning(self, "Erro", "Valores de calibração inválidos.")
    return

# Se for GRBL, verifica se tem o atributo grbl
if not hasattr(self.controller.cnc, 'grbl') or not self.controller.cnc.grbl:
    QMessageBox.warning(self, "Erro", "Controlador GRBL não disponível.")
    return

try:
    pulses = float(self.pulses_input.text())
    fuso_pass = float(self.fuso_input.text())

    # Calcula steps/mm: (pulsos por revolução) / (passo do fuso em mm)
    steps_per_mm = pulses / fuso_pass

    # Armazena o valor calculado
    self.controller.cnc.steps_to_mm_factor = fuso_pass / pulses

    # Envia comandos para configurar o GRBL
    logger.info(f"CALIBRAÇÃO: Configurando steps/mm para {steps_per_mm}")

    # Verifica se o usuário deseja realmente enviar estes valores
    reply = QMessageBox.question(
        self,
        "Confirmar Calibração",
        f"Deseja enviar os seguintes parâmetros para o GRBL?\n\n"
        f"Steps/mm eixo X: {steps_per_mm:.3f}\n"
        f"Steps/mm eixo Y: {steps_per_mm:.3f}\n\n"
        f"Issó irá alterar a configuração do controlador.",
        QMessageBox.StandardButton.Yes | QMessageBox.StandardButton.No
    )

    if reply == QMessageBox.StandardButton.Yes:
        # Comandos para configurar os eixos X e Y

```

```

self.controller.cnc.grbl.send_immediately(f"$100={steps_per_mm:.3f}")
QTimer.singleShot(100, lambda: self.controller.cnc.grbl.send_immediately(f"$101={steps_per_mm:.3f}"))

# Solicita ao usuário que faça um teste de calibração
QTimer.singleShot(500, self.show_calibration_test_dialog)

self.statusBar().showMessage(f"Calibração aplicada: {steps_per_mm:.3f} steps/mm")

# ----- salva no JSON -----
self.config.remember_calibration(pulses, fuso_pass)

else:
    self.statusBar().showMessage("Calibração cancelada pelo usuário")

except ValueError:
    QMessageBox.warning(self, "Erro", "Valores de calibração inválidos.")

def show_calibration_test_dialog(self):
    """
    Exibe um diálogo para testar a calibração atual.
    """
    dialog = QDialog(self)
    dialog.setWindowTitle("Teste de Calibração")
    dialog.setMinimumWidth(400)

    layout = QVBoxLayout(dialog)

    # Instruções
    instructions = QLabel(
        "Para testar a calibração:\n\n"
        "1. Coloque um papel milimetrado ou uma régua sob a cabeça da máquina\n"
        "2. Escolha uma distância de teste\n"
        "3. Clique em 'Mover X' ou 'Mover Y' para testar cada eixo\n"
        "4. Verifique se o deslocamento físico corresponde ao valor escolhido\n"
        "5. Se necessário, ajuste os valores de calibração e aplique novamente"
    )
    instructions.setWordWrap(True)
    layout.addWidget(instructions)

    # Distância de teste
    test_layout = QHBoxLayout()
    test_layout.addWidget(QLabel("Distância de teste:"))
    distance_input = QLineEdit("10")
    test_layout.addWidget(distance_input)
    test_layout.addWidget(QLabel("mm"))
    layout.addLayout(test_layout)

    # Botões de teste
    buttons_layout = QHBoxLayout()

    move_x_btn = QPushButton("Mover X")

```

```
move_x_btn.clicked.connect(lambda: self.test_calibration_move(0, float(distance_input.text())))
```

```
move_y_btn = QPushButton("Mover Y")
```

```
move_y_btn.clicked.connect(lambda: self.test_calibration_move(1, float(distance_input.text())))
```

```
reset_position_btn = QPushButton("Zerar Posição")
```

```
reset_position_btn.clicked.connect(self.set_zero_position)
```

```
buttons_layout.addWidget(move_x_btn)
```

```
buttons_layout.addWidget(move_y_btn)
```

```
buttons_layout.addWidget(reset_position_btn)
```

```
layout.addLayout(buttons_layout)
```

```
# Resultados
```

```
result_group = QGroupBox("Resultados")
```

```
result_layout = QVBoxLayout(result_group)
```

```
self.calibration_test_result = QLabel("Execute um teste para ver os resultados")
```

```
result_layout.addWidget(self.calibration_test_result)
```

```
layout.addWidget(result_group)
```

```
# Botões de controle
```

```
control_layout = QHBoxLayout()
```

```
close_btn = QPushButton("Concluir")
```

```
close_btn.clicked.connect(dialog.accept)
```

```
control_layout.addWidget(close_btn)
```

```
layout.addLayout(control_layout)
```

```
dialog.exec()
```

```
def test_calibration_move(self, axis, distance):
```

```
    """
```

```
    Realiza um movimento de teste para calibração.
```

```
    Args:
```

```
        axis: 0 para X, 1 para Y
```

```
        distance: Distância em mm para mover
```

```
    """
```

```
    if not self.controller.cnc.is_connected:
```

```
        QMessageBox.warning(self, "Erro", "CNC não conectada")
```

```
        return
```

```
    try:
```

```
        # Captura posição inicial
```

```
        initial_position = self.controller.cnc.get_current_position()
```

```
        # Prepara o comando
```

```
        axis_name = "X" if axis == 0 else "Y"
```

```

# Define o modo absoluto para garantir precisão
self.controller.cnc.grbl.send_immediately("G90")

# Calcula a posição absoluta a ser atingida
target_position = {}
target_position['x'] = initial_position['x'] + distance if axis == 0 else initial_position['x']
target_position['y'] = initial_position['y'] + distance if axis == 1 else initial_position['y']

# Envia o movimento como coordenada absoluta
if axis == 0:
    self.controller.cnc.move_to_absolute_position(target_position['x'], None, 500)
else:
    self.controller.cnc.move_to_absolute_position(None, target_position['y'], 500)
self.controller.cnc.wait_for_idle()

# Aguarda um pouco para o movimento ser concluído
QTimer.singleShot(1500, lambda: self.verify_calibration_result(axis, initial_position, distance))

except Exception as e:
    logger.error(f"CALIBRAÇÃO: Erro no teste de calibração: {e}")
    QMessageBox.warning(self, "Erro", f"Erro no teste: {str(e)}")

def _show_calibration_result(self, axis, initial_position, expected_distance):
    """
    Calcula deslocamento real, erro e atualiza o QLabel de resultados.
    Executado de forma assíncrona pelo QTimer.
    """
    try:
        current_position = self.controller.cnc.get_current_position()

        axis_name = "x" if axis == 0 else "y"
        actual_distance = current_position[axis_name] - initial_position[axis_name]

        error = actual_distance - expected_distance
        error_percent = (error / expected_distance * 100) if expected_distance else 0

        result_text = (
            f"Eixo: {axis_name.upper()}\n"
            f"Movimento comandado: {expected_distance:.3f} mm\n"
            f"Movimento real: {actual_distance:.3f} mm\n"
            f"Erro: {error:.3f} mm ({error_percent:.2f}%) \n\n"
        )

        if abs(error_percent) < 1:
            result_text += "■ Calibração excelente (erro < 1%)"
        elif abs(error_percent) < 5:
            result_text += "✓ Calibração aceitável (erro < 5%)"
        else:
            result_text += "■ Calibração insatisfatória – ajuste os parâmetros"
    
```

```
if hasattr(self, "calibration_test_result"):
    self.calibration_test_result.setText(result_text)
```

```
logger.info(
    "CALIBRAÇÃO: Resultado – %s",
    result_text.replace("\n", " | ")
)
```

```
except Exception as e:
```

```
    logger.error(f"CALIBRAÇÃO: Erro ao calcular resultado: {e}")
```

```
def verify_calibration_result(self, axis, initial_position, expected_distance):
```

```
    """
```

```
    Verifica o resultado do teste de calibração.
```

```
    Args:
```

```
        axis: 0 para X, 1 para Y
```

```
        initial_position: Posição antes do movimento
```

```
        expected_distance: Distância esperada do movimento
```

```
    """
```

```
    try:
```

```
        # 1) força a atualização de status
```

```
        self.controller.cnc.grbl.send_immediately("?")
```

```
        # 2) Agenda o cálculo daqui a 250 ms para NÃO travar a GUI
```

```
        QTimer.singleShot(
```

```
            250,
```

```
            lambda: self._show_calibration_result(
```

```
                axis, initial_position, expected_distance
```

```
            )
```

```
        )
```

```
    return
```

```
except Exception as e:
```

```
    logger.error(f"CALIBRAÇÃO: Erro ao verificar resultado: {e}")
```

```
def on_image_captured(self, image, position_name):
```

```
    """Handle captured image and register position"""
```

```
    # Display in image viewer
```

```
    self.image_viewer.display_image(image, f"Image: {position_name}")
```

```
    # Get current position
```

```
    if not self.controller.cnc.is_connected:
```

```
        QMessageBox.warning(self, "Error", "CNC not connected")
```

```
    return
```

```
    current_pos = self.controller.cnc.get_current_position()
```

```
    # Register position with image
```

```
    self.position_registry.add_position(
```

```
        position_name,  
        current_pos['x'],  
        current_pos['y'],  
        current_pos['z'],  
        image  
    )
```

```
self.statusBar().showMessage(f'Position '{position_name}' registered at X:{current_pos['x']:.3f}, Y:{current_pos['y']:.3f}, Z:{current_pos['z']:.3f}')
```

```
def create_sequence_from_registry(self):
```

```
    """Create a sequence from registered positions"""
```

```
    if not self.position_registry.positions:
```

```
        QMessageBox.warning(self, "Warning", "No positions registered")
```

```
    return
```

```
    # Get sequence name
```

```
    sequence_name = self.sequence_widget.sequence_name.text()
```

```
    if not sequence_name:
```

```
        QMessageBox.warning(self, "Warning", "Please enter a sequence name")
```

```
    return
```

```
    # Clear existing positions in the position list widget
```

```
    self.position_list_widget.clear_positions()
```

```
    # Create positions for the sequence
```

```
    positions = []
```

```
    for pos in self.position_registry.positions:
```

```
        # Create position object with camera parameters
```

```
        camera_params = {"has_image": pos['image'] is not None}
```

```
        inspection_pos = InspectionPosition(  
            pos['name'],
```

```
            pos['x'],
```

```
            pos['y'],
```

```
            pos.get('z', 0.0),
```

```
            camera_params
```

```
        )
```

```
        positions.append(inspection_pos)
```

```
    # Also add to the position list widget
```

```
    self.position_list_widget.add_position(inspection_pos)
```

```
    # Create the sequence
```

```
    self.current_sequence = self.controller.create_sequence(sequence_name, positions)
```

```
    # Update UI
```

```
    self.sequence_widget.sequence_status.setText(f"Created: {len(positions)} positions")
```

```
    self.statusBar().showMessage(f'Sequence '{sequence_name}' created with {len(positions)} positions')
```

```
    # Ask if user wants to save as G-CODE
```

```
    reply = QMessageBox.question(  
        self, "Save as G-CODE", "Do you want to save the sequence as G-CODE?",  
        QMessageBox.Yes | QMessageBox.No, QMessageBox.No)
```

```

self,
"Save G-CODE",
"Do you want to save this sequence as G-CODE?",
QMessageBox.StandardButton.Yes | QMessageBox.StandardButton.No
)

```

```

if reply == QMessageBox.StandardButton.Yes:
    self.save_gcode()

```

```

def load_gcode(self):
    """Carrega uma sequência a partir de um arquivo G-CODE"""
    from aoi_lib.gcode_manager import GCodeManager
    gcode_manager = GCodeManager()

    filename, _ = QFileDialog.getOpenFileName(
        self, "Abrir G-CODE", "", "Arquivos G-CODE (*.gcode *.nc *.ngc)"
    )

    if filename:
        # Carrega o G-CODE
        result = gcode_manager.read_gcode_file(filename)

        if result:
            # Limpa posições atuais
            self.position_list_widget.clear_positions()

            # Cria sequência a partir dos dados do G-CODE
            positions = []
            for pos_data in result['positions']:
                # Cria objeto de posição
                position = InspectionPosition(
                    pos_data['name'],
                    pos_data['x'],
                    pos_data['y'],
                    pos_data['camera_params']
                )

                # Adiciona à lista visual
                self.position_list_widget.add_position(position)

                # Adiciona à lista interna
                positions.append(position)

            # Cria a sequência
            self.current_sequence = self.controller.create_sequence(
                result['sequence_name'], positions
            )

            # Atualiza interface
            self.sequence_widget.sequence_name.setText(result['sequence_name'])
            self.sequence_widget.sequence_status.setText(

```

```

        f"Carregada do G-CODE: {len(positions)} posições"
    )

    self.statusBar().showMessage(f"G-CODE carregado de {filename}")
else:
    QMessageBox.critical(self, "Erro", "Falha ao carregar o arquivo G-CODE")

def refresh_ports(self):
    """Atualiza a lista de portas seriais disponíveis"""
    import serial.tools.list_ports

    self.cnc_port_combo.clear()
    ports = [port.device for port in serial.tools.list_ports.comports()]

    if ports:
        self.cnc_port_combo.addItems(ports)

        # Verificar se COM9 está na lista e selecionar
        com9_index = self.cnc_port_combo.findText("COM9")
        if com9_index >= 0:
            self.cnc_port_combo.setCurrentIndex(com9_index)
            self.statusBar().showMessage("Porta COM9 detectada")
        else:
            self.statusBar().showMessage("Nenhuma porta serial encontrada")

def connect_cnc(self):
    """Conecta à máquina CNC usando a biblioteca grbl-streamer"""
    # Se for PLCAxisController, alterna Modbus connect/disconnect
    if isinstance(self.controller.cnc, PLCAxisController):
        plc = self.controller.cnc
        if plc.is_connected:
            # desconectar
            plc.close()
            self.connect_cnc_btn.setText("Conectar PLC")
            self.cnc_status.setText("Desconectado")
            self.statusBar().showMessage("PLC desconectado")
            logger.info("PLC desconectado pelo usuário")
            # ao desconectar PLC, volta a desabilitar as abas
            self.right_panel.setTabEnabled(0, False)
            self.right_panel.setTabEnabled(1, False)
        else:
            # recria e conecta via construtor
            logger.info(f"Tentando conectar ao PLC em {plc.host}:{plc.port}")
            try:
                new_plc = PLCAxisController(host=plc.host, port=plc.port)
            except Exception as e:
                QMessageBox.warning(
                    self,
                    "Erro de Conexão",
                    f"Falha ao conectar ao PLC em {plc.host}:{plc.port}:\n{e}"
                )
    )

```

```

        logger.error(f"Falha ao conectar ao PLC em {plc.host}:{plc.port}: {e}")
    else:
        # substitui o controller e reaplica calibração
        self.controller.cnc = new_plc
        ppr = float(self.config.get("calibration", "pulses_per_rev", default=1.0))
        pitch = float(self.config.get("calibration", "fuso_pitch", default=1.0))
        new_plc.pulses_per_mm = ppr / pitch if pitch != 0 else 1.0

        self.connect_cnc_btn.setText("Desconectar PLC")
        self.cnc_status.setText("Conectado")
        self.statusBar().showMessage(f"PLC conectado em {new_plc.host}:{new_plc.port}")
        logger.info(f"PLC conectado com sucesso em {new_plc.host}:{new_plc.port}")
        # ao conectar PLC com sucesso, habilita as abas
        self.right_panel.setTabEnabled(0, True)
        self.right_panel.setTabEnabled(1, True)
    return

# Senão, cai no fluxo original GRBL...
if hasattr(self.controller.cnc, 'grbl') and self.controller.cnc.grbl:
    self.controller.cnc.grbl.poll_stop()
    self.controller.cnc.grbl.disconnect()
    self.controller.cnc.grbl = None
    self.controller.cnc.is_connected = False
    self.connect_cnc_btn.setText("Conectar CNC")
    self.cnc_status.setText("Desconectado")
    self.statusBar().showMessage("CNC desconectada")

else:
    # Conectar
    port = self.cnc_port_combo.currentText()
    if not port:
        QMessageBox.warning(self, "Erro", "Selecione uma porta serial")
        return

    self.statusBar().showMessage(f"Conectando à CNC na porta {port}...")

    try:
        # Define função de callback para eventos do GrblStreamer
        def grbl_callback(eventstring, *data):
            logger.debug(f"CALLBACK: Evento '{eventstring}' recebido com data: {data}")

        # Capturar offsets do sistema de coordenadas (G54, G55, etc.)
        if eventstring == "on_hash_stateupdate":
            if data and isinstance(data[0], dict):
                hash_state = data[0]

                # Processar G54 APENAS se o WCS ativo for G54
                # Isso evita que a resposta tardia do $$ sobrescreva um offset
                # que foi definido manualmente via G10 L20 para G54.
                if self.active_wcs == "G54":
                    g54_offset_data = hash_state.get('G54')
                    if isinstance(g54_offset_data, (list, tuple)) and len(g54_offset_data) >= 2:

```

```

try:
    new_offset_x_phys = float(g54_offset_data[0])
    new_offset_y_phys = float(g54_offset_data[1])
    new_offset_z_phys = float(g54_offset_data[2]) if len(g54_offset_data) > 2 else 0.0

    # Converte Y físico → lógico (depende de invert_y)
    if self.controller.cnc.invert_y:
        new_offset_y_log = -new_offset_y_phys
    else:
        new_offset_y_log = new_offset_y_phys

    self.current_wcs_offset = {
        # X e Z permanecem iguais
        'x': new_offset_x_phys,
        # guardamos FÍSICO para operar com G10 L20.
        'y': new_offset_y_phys,
        'z': new_offset_z_phys
    }

    logger.info(
        "CALLBACK: Offset G54 atualizado "
        f"(físico): {{x:{new_offset_x_phys:.3f}, y:{new_offset_y_phys:.3f}, z:{new_offset_z_phys:.3f}}}; "
        f"(lógico Y={new_offset_y_log:.3f})"
    )
except (ValueError, TypeError):
    logger.error(f"CALLBACK: Erro ao converter offset G54 de $#: {g54_offset_data}")
else:
    logger.warning(f"CALLBACK: Offset G54 não encontrado ou inválido nos dados hash para WCS ativo")
else:
    logger.debug(f"CALLBACK: Ignorando atualização de offset G54 de $# porque WCS ativo é {self.active_wcs}")
# TODO: Se precisar suportar outros WCS (G55-G59), adicionar lógica similar aqui

# Capturar estado do parser para saber o WCS ativo
elif eventstring == "on_gcode_parser_stateupdate":
    # ... (código existente para atualizar self.active_wcs) ...
    if data and isinstance(data[0], list) and len(data[0]) > 1:
        parser_state = data[0]
        # O índice 1 contém o WCS ativo (ex: "54", "55", etc.)
        # O índice 0 contém o modo de movimento (ex: "1" para G1)
        # O índice 4 contém o modo de distância (ex: "90" para G90)
        new_active_wcs = f"G{parser_state[1]}"
        new_motion_mode = f"G{parser_state[0]}"
        new_distance_mode = f"G{parser_state[4]}"

    if new_active_wcs != self.active_wcs:
        logger.info(f"CALLBACK: WCS Ativo mudou de {self.active_wcs} para {new_active_wcs}")
        self.active_wcs = new_active_wcs
        # Ao mudar o WCS, seria ideal buscar o offset correspondente via $#
        # ou ter todos os offsets armazenados. Por enquanto, apenas logamos.
        # self.controller.cnc.grbl.send_immediately("#") # Cuidado com loops

```

```

# Atualiza estado interno da aplicação sobre modos G90/G91
# Isso garante que a UI e a lógica de movimento estejam sincronizadas
# com o estado real do GRBL reportado por $G.
if hasattr(self, 'movement_widget'): # Verifica se o widget existe
    if new_distance_mode == "G90":
        if not self.movement_widget.mode_absolute.isChecked():
            logger.info("CALLBACK ($G): Sincronizando UI para G90 (Absoluto)")
            self.movement_widget.mode_absolute.setChecked(True)
            self.movement_widget.mode_relative.setChecked(False)
        elif new_distance_mode == "G91":
            if not self.movement_widget.mode_relative.isChecked():
                logger.info("CALLBACK ($G): Sincronizando UI para G91 (Relativo)")
                self.movement_widget.mode_absolute.setChecked(False)
                self.movement_widget.mode_relative.setChecked(True)

elif eventstring == "on_stateupdate":
    logger.info(f"CALLBACK: Processando 'on_stateupdate'. Dados brutos: {data}")

    if len(data) >= 3:
        state = data[0]
        mpos_tuple = data[1] # Posição da Máquina (MPos)
        # wpos_tuple = data[2] # Posição de Trabalho (WPos) - Ignorando pois está vindo zerado

    logger.debug(f"CALLBACK DETALHADO: state={state}, mpos={mpos_tuple}") # Removido wpos do log d

    # Atualiza estado da máquina
    old_state = self.controller.cnc.machine_status if hasattr(self.controller.cnc, 'machine_status') else None
    self.controller.cnc.machine_status = state

    if old_state != state:
        logger.debug(f"CALLBACK: Estado da máquina mudou de '{old_state}' para '{state}'")

    # Calcular WPOS a partir de MPOS e do offset armazenado
    if isinstance(mpos_tuple, (list, tuple)) and len(mpos_tuple) >= 2:
        try:
            # 1) valores FÍSICOS reportados pelo GRBL
            mpos_x_phys = float(mpos_tuple[0])
            mpos_y_phys = float(mpos_tuple[1])
            mpos_z_phys = float(mpos_tuple[2]) if len(mpos_tuple) > 2 else 0.0

            # 2) guarda MPos física para rotinas G10
            self.current_mpos = {
                'x': mpos_x_phys,
                'y': mpos_y_phys,
                'z': mpos_z_phys
            }

            # 3) converte considerando o modo de cinemática
            if self.controller.cnc.kinematics_mode == "corexy":
                # No modo CoreXY: mpos_x_phys = motor A, mpos_y_phys = motor B
                # Converte A,B para coordenadas cartesianas X,Y

```

```

x_cart, y_cart = self.controller.cnc._convert_ab_to_xy(mpos_x_phys, mpos_y_phys)

# CORREÇÃO: Converte o offset de coordenadas de motores para cartesianas
off_a = self.current_wcs_offset['x'] # offset motor A
off_b = self.current_wcs_offset['y'] # offset motor B
off_x_cart, off_y_cart = self.controller.cnc._convert_ab_to_xy(off_a, off_b)
off_z_cart = self.current_wcs_offset['z']

# Aplica inversão lógica se configurada
y_cart_log = -y_cart if self.controller.cnc.invert_y else y_cart
z_cart_log = -mpos_z_phys if self.controller.cnc.invert_z else mpos_z_phys

# Aplica inversão lógica também ao offset para consistência
off_y_cart_log = -off_y_cart if self.controller.cnc.invert_y else off_y_cart
off_z_cart_log = -off_z_cart if self.controller.cnc.invert_z else off_z_cart

# Agora calcula WPos usando coordenadas cartesianas para ambos
calculated_wpos_x = x_cart - off_x_cart
calculated_wpos_y = y_cart_log - off_y_cart_log
calculated_wpos_z = z_cart_log - off_z_cart_log

else:
    # Modo cartesiano (comportamento original)
    mpos_y_log = -mpos_y_phys if self.controller.cnc.invert_y else mpos_y_phys
    mpos_z_log = -mpos_z_phys if self.controller.cnc.invert_z else mpos_z_phys
    off_y_log = (-self.current_wcs_offset['y']
                 if self.controller.cnc.invert_y
                 else self.current_wcs_offset['y'])
    off_z_log = (-self.current_wcs_offset['z']
                 if self.controller.cnc.invert_z
                 else self.current_wcs_offset['z'])
    calculated_wpos_x = mpos_x_phys - self.current_wcs_offset['x']
    calculated_wpos_y = mpos_y_log - off_y_log
    calculated_wpos_z = mpos_z_log - off_z_log

new_position = {
    'x': calculated_wpos_x,
    'y': calculated_wpos_y,
    'z': calculated_wpos_z
}

old_position = None
if hasattr(self.controller.cnc, 'current_position'):
    old_position = self.controller.cnc.current_position.copy()
    # logger.debug(f"CALLBACK: Posição interna ANTES da atualização: {old_position}") # Log opcional

logger.debug(f"CALLBACK: MPos={mpos_tuple}, Offset={self.current_wcs_offset}, WPos Calculada=
logger.debug(f"CALLBACK: Tentando atualizar posição interna (usando WPOS CALCULADA) para:

# Compara new_position (WPos calculada) com old_position
position_changed = (old_position is None) or \

```

```

        (abs(old_position['x'] - new_position['x']) > 1e-4) or \
        (abs(old_position['y'] - new_position['y']) > 1e-4) or \
        (abs(old_position.get('z', 0.0) - new_position.get('z', 0.0)) > 1e-4)

    if position_changed:
        logger.info(f"CALLBACK: POSIÇÃO INTERNA ATUALIZADA (usando WPOS CALCULADA): {old_position}")
        # Atualiza a posição no controlador com a WPos calculada
        self.controller.cnc.current_position = new_position

    except (ValueError, TypeError, IndexError) as e:
        logger.error(f"CALLBACK: Erro ao processar MPOS ou calcular WPOS: {e}, mpos={mpos_tuple}")
    else:
        logger.error(f"CALLBACK: Formato inválido para MPOS: {type(mpos_tuple)}, valor: {mpos_tuple}")

else:
    logger.error(f"CALLBACK: 'on_stateupdate' recebido com dados insuficientes (len={len(data)}). Dados: {data}")

# ... (restante do código do callback para outros eventos: on_write, on_read, etc.) ...
elif eventstring == "on_write":
    logger.debug(f"CALLBACK: Comando enviado para GRBL: {data[0] if data else 'vazio'}")
# ... (etc.) ...

# --- INÍCIO DA MODIFICAÇÃO ---
# Inicializa o GrblStreamer APENAS com o callback
self.controller.cnc.grbl = GrblStreamer(grbl_callback)
# --- FIM DA MODIFICAÇÃO ---

logger.debug(f"CONEXÃO: Tentando conectar à porta {port} com baudrate 115200")
# Conecta usando o método cnect()
self.controller.cnc.grbl.cnect(port, 115200)
time.sleep(2.0)

if not self.controller.cnc.grbl.connected:
    logger.error("CONEXÃO: Falha ao estabelecer conexão serial (grbl.connected é False).")
    raise ConnectionError("Falha ao conectar à porta serial após inicialização.")

logger.debug("CONEXÃO: Enviando comando de desbloqueio $X")
self.controller.cnc.grbl.send_immediately("$X")
time.sleep(0.1)

logger.debug("CONEXÃO: Configurando $10=3 para relatório completo de posição")
self.controller.cnc.grbl.send_immediately("$10=3") # Mantém $10=3 para receber MPos
time.sleep(0.1)

# --- INÍCIO DA MODIFICAÇÃO ---
# Solicitar estado hash logo após conectar para obter offsets
logger.debug("CONEXÃO: Solicitando estado hash ($#) para obter offsets")
self.controller.cnc.grbl.send_immediately("#")
time.sleep(0.1)
# --- FIM DA MODIFICAÇÃO ---

```

```
logger.debug("CONEXÃO: Configurando modo relativo G91")
self.controller.cnc.grbl.send_immediately("G91")
time.sleep(0.1)
```

```
logger.debug("CONEXÃO: Iniciando polling de status")
self.controller.cnc.grbl.poll_start()
```

```
self.controller.cnc.is_connected = True
self.config.apply_to_cnc(self.controller.cnc)
self.config.remember_cnc_port(port)
self.controller.cnc.machine_status = "Idle"
self.connect_cnc_btn.setText("Desconectar CNC")
self.cnc_status.setText("Conectado")
self.statusBar().showMessage(f"CNC conectada na porta {port}")
```

```
except Exception as e:
```

```
# ... (código de tratamento de erro de conexão) ...
logger.error(f"CONEXÃO: Falha ao conectar ou configurar: {str(e)}", exc_info=True)
QMessageBox.critical(self, "Erro", f"Falha ao conectar a CNC: {str(e)}")
if hasattr(self.controller.cnc, 'grbl') and self.controller.cnc.grbl:
    try:
        self.controller.cnc.grbl.disconnect()
    except: pass
self.controller.cnc.grbl = None
self.controller.cnc.is_connected = False
self.controller.cnc.machine_status = "Erro Conexão"
self.connect_cnc_btn.setText("Conectar CNC")
self.cnc_status.setText("Erro Conexão")
```

```
def connect_camera(self):
```

```
    """Conecta à câmera"""
```

```
    if hasattr(self.controller.camera, 'is_connected') and self.controller.camera.is_connected:
```

```
        # Interrompe preview antes de liberar a câmera
```

```
        self.camera_preview.stop_preview()
```

```
    # Desconectar
```

```
    self.controller.camera.disconnect()
```

```
    self.connect_camera_btn.setText("Conectar Câmera")
```

```
    self.statusBar().showMessage("Câmera desconectada")
```

```
else:
```

```
    # Conectar
```

```
    try:
```

```
        camera_id = int(self.camera_id_combo.currentText())
```

```
    self.statusBar().showMessage(f"Conectando à câmera ID {camera_id}...")
```

```
    if self.controller.connect_camera(camera_id):
```

```
        self.connect_camera_btn.setText("Desconectar Câmera")
```

```
        self.statusBar().showMessage(f"Câmera ID {camera_id} conectada")
```

```
        self.config.remember_camera_id(camera_id)
```

```
    else:
```

```
        QMessageBox.critical(self, "Erro", f"Falha ao conectar à câmera: {self.controller.camera.last_error}")
    except ValueError:
```

```
        QMessageBox.warning(self, "Erro", "ID de câmera inválido")
```

```
def test_camera(self):
```

```
    """Testa a captura de imagem da câmera"""
```

```
    if not hasattr(self.controller.camera, 'is_connected') or not self.controller.camera.is_connected:
```

```
        QMessageBox.warning(self, "Aviso", "Câmera não conectada")
```

```
    return
```

```
    image = self.controller.camera.capture()
```

```
    if image is not None:
```

```
        self.image_viewer.display_image(image, "Imagem de teste da câmera")
```

```
        self.statusBar().showMessage("Imagem capturada com sucesso")
```

```
    else:
```

```
        QMessageBox.warning(self, "Erro", f"Falha ao capturar imagem: {self.controller.camera.last_error}")
```

```
def update_position_display(self):
```

```
    """Atualiza a exibição da posição atual (agora exibindo WPos calculada)"""
```

```
    if not self.controller.cnc.is_connected:
```

```
        # logger.debug("update_position_display: CNC não conectada.") # Log já existente
```

```
    return
```

```
    try:
```

```
        # get_current_position agora retorna a WPos calculada
```

```
        position = self.controller.cnc.get_current_position()
```

```
        logger.debug("update_position_display: posição (WPos calculada) obtida do CNC: %s", position)
```

```
    except Exception as e:
```

```
        logger.error("update_position_display: erro ao obter posição: %s", e)
```

```
    return
```

```
# Comparação para log (opcional, pode ser removido se poluir muito)
```

```
if self.last_logged_position is not None:
```

```
    if position == self.last_logged_position:
```

```
        pass
```

```
        # logger.warning("update_position_display: posição (WPos calculada) inalterada: %s", position)
```

```
    else:
```

```
        logger.debug("update_position_display: posição (WPos calculada) mudou de %s para %s",
                      self.last_logged_position, position)
```

```
    else:
```

```
        logger.debug("update_position_display: nenhuma posição (WPos calculada) anterior registrada.")
```

```
self.last_logged_position = position.copy()
```

```
# Atualiza os labels da interface com a WPos calculada
```

```
self.x_position.setText(f'{position["x"]:.3f} mm')
```

```
self.y_position.setText(f'{position["y"]:.3f} mm')
```

```
# ----- NOVO: eixo Z -----
```

```
if hasattr(self, "z_position"):
```

```
    self.z_position.setText(f'{position["z"]:.3f} mm')
```

```
self.cnc_status.setText(self.controller.cnc.machine_status)
```

```

def add_current_position(self):
    """Adiciona a posição atual à lista"""
    if not self.controller.cnc.is_connected:
        QMessageBox.warning(self, "Aviso", "CNC não conectada")
        return

    position_name, ok = QInputDialog.getText(self, "Nova Posição", "Nome da posição:")

    if ok and position_name:
        position = self.controller.add_current_position(position_name)
        self.position_list_widget.add_position(position)
        self.statusBar().showMessage(f"Posição '{position_name}' adicionada")

def remove_position(self):
    """Remove a posição selecionada"""
    position = self.position_list_widget.remove_selected_position()

    if position:
        self.controller.position_manager.remove_position(position.name)
        self.statusBar().showMessage(f"Posição '{position.name}' removida")

def on_position_selected(self, position):
    """Manipula a seleção de uma posição"""
    # Poderia mover para esta posição, mostrar detalhes, etc.
    self.statusBar().showMessage(f"Posição selecionada: {position.name} ({position.x:.3f}, {position.y:.3f})")

def create_sequence(self):
    """Cria uma nova sequência com as posições atuais"""
    positions = self.position_list_widget.get_all_positions()

    if not positions:
        QMessageBox.warning(self, "Aviso", "Adicione pelo menos uma posição")
        return

    sequence_name = self.sequence_widget.sequence_name.text()
    if not sequence_name:
        QMessageBox.warning(self, "Aviso", "Digite um nome para a sequência")
        return

    self.current_sequence = self.controller.create_sequence(sequence_name, positions)
    self.statusBar().showMessage(f"Sequência '{sequence_name}' criada com {len(positions)} posições")
    self.sequence_widget.sequence_status.setText(f"Criada: {len(positions)} posições")

def run_sequence(self):
    """Executes the current sequence"""
    if not self.current_sequence:
        QMessageBox.warning(self, "Warning", "Create a sequence first")
        return

    if not self.controller.cnc.is_connected:

```

```

    QMessageBox.warning(self, "Warning", "CNC not connected")
    return

if not hasattr(self.controller.camera, 'is_connected') or not self.controller.camera.is_connected:
    QMessageBox.warning(self, "Warning", "Camera not connected")
    return

# Clear previous results
self.results_table.setRowCount(0)

# Configure UI for execution
self.is_running_sequence = True
self.sequence_widget.run_sequence_btn.setEnabled(False)
self.sequence_widget.stop_sequence_btn.setEnabled(True)
self.sequence_widget.sequence_status.setText("Executing...")

# Start execution
try:
    self.statusBar().showMessage(f"Executing sequence '{self.current_sequence.name}'...")
    # Configura velocidade no controlador baseada na interface
    self.controller.set_feed_rate(self.movement_widget.get_current_feed_rate())

    # Use a thread to run the sequence
    self.run_thread = SequenceRunnerThread(self.controller, self.current_sequence.name)
    self.run_thread.image_captured.connect(self.on_sequence_image_captured)
    self.run_thread.sequence_completed.connect(self.on_sequence_completed)
    self.run_thread.sequence_error.connect(self.on_sequence_error)
    self.run_thread.start()

except Exception as e:
    self.statusBar().showMessage(f"Error executing sequence: {str(e)}")
    self.sequence_widget.sequence_status.setText("Error")
    self.on_sequence_completed()

def on_sequence_image_captured(self, result):
    """Called when an image is captured during sequence execution"""
    position = result["position"]
    image = result["image"]
    timestamp = result["timestamp"]

    # Display the image on the existing camera_preview widget
    self.camera_preview.display_image(image)
    # Atualiza a status bar com o nome/posição
    self.statusBar().showMessage(
        f"Position captured: {position.name} ({position.x:.3f}, {position.y:.3f})"
    )

# Add to results table
row = self.results_table.rowCount()
self.results_table.insertRow(row)
self.results_table.setItem(row, 0, QTableWidgetItem(position.name))

```

```

self.results_table.setItem(row, 1, QTableWidgetItem(time.strftime("%H:%M:%S", time.localtime(timestamp))))
self.results_table.setItem(row, 2, QTableWidgetItem("Captured"))

def stop_sequence(self):
    """Para a execução da sequência atual"""
    if self.is_running_sequence:
        self.controller.stop_sequence()
        self.statusBar().showMessage("Execução de sequência interrompida")
        self.on_sequence_completed()

def on_sequence_completed(self):
    """Chamado quando a execução da sequência é concluída"""
    self.is_running_sequence = False
    self.sequence_widget.run_sequence_btn.setEnabled(True)
    self.sequence_widget.stop_sequence_btn.setEnabled(False)
    self.sequence_widget.sequence_status.setText("Concluída")
    self.statusBar().showMessage("Execução de sequência concluída")

def on_sequence_error(self, error_message):
    """Chamado quando ocorre um erro durante a execução da sequência"""
    QMessageBox.critical(self, "Erro na Sequência", error_message)
    self.on_sequence_completed()

def save_program(self):
    """Salva o programa de inspeção atual"""
    if not self.current_sequence:
        QMessageBox.warning(self, "Aviso", "Crie uma sequência primeiro")
        return

    filename, _ = QFileDialog.getSaveFileName(self, "Salvar Programa", "", "Arquivos JSON (*.json)")

    if filename:
        if not filename.endswith('.json'):
            filename += '.json'

        if self.controller.save_positions(filename):
            self.statusBar().showMessage(f"Programa salvo em {filename}")
        else:
            QMessageBox.critical(self, "Erro", "Falha ao salvar o programa")

def load_program(self):
    """Carrega um programa de inspeção salvo"""
    filename, _ = QFileDialog.getOpenFileName(self, "Carregar Programa", "", "Arquivos JSON (*.json)")

    if filename:
        if self.controller.load_positions(filename):
            # Atualiza a interface
            self.position_list_widget.clear_positions()

            # Adiciona posições carregadas à lista
            for name, position in self.controller.position_manager.positions.items():

```

```
self.position_list_widget.add_position(position)
```

```
# Se há sequências, usa a primeira como atual
```

```
if self.controller.position_manager.sequences:
```

```
    sequence_name = next(iter(self.controller.position_manager.sequences))
```

```
    self.current_sequence = self.controller.position_manager.sequences[sequence_name]
```

```
    self.sequence_widget.sequence_name.setText(sequence_name)
```

```
    self.sequence_widget.sequence_status.setText(f"Carregada: {len(self.current_sequence.positions)} posições")
```

```
    self.statusBar().showMessage(f"Programa carregado de {filename}")
```

```
else:
```

```
    QMessageBox.critical(self, "Erro", "Falha ao carregar o programa")
```

```
# LIMPEZA GERAL (Threads, Timers, Dispositivos)
```

```
def _cleanup_resources(self):
```

```
    """Para tudo que possa manter o Qt vivo após o fechamento."""
```

```
    if getattr(self, "_already_clean", False):
```

```
        return # evita executar 2x
```

```
    self._already_clean = True
```

```
# 1) Sequências em execução
```

```
if getattr(self, "is_running_sequence", False):
```

```
    self.controller.stop_sequence()
```

```
# 2) Timers -----
```

```
for tm_name in ("update_timer",):
```

```
    tm = getattr(self, tm_name, None)
```

```
    if tm and tm.isActive():
```

```
        tm.stop()
```

```
if getattr(self, "camera_preview", None):
```

```
    self.camera_preview.stop_preview()
```

```
# 3) QThreads -----
```

```
for th_name in ("run_thread", "map_thread", "_move_thread"):
```

```
    th = getattr(self, th_name, None)
```

```
    if th and th.isRunning():
```

```
        th.requestInterruption()
```

```
        th.quit()
```

```
        th.wait(2000) # aguarda até 2 s
```

```
# 4) Thread de status do GRBL dentro do controlador CNC
```

```
if getattr(self.controller.cnc, "running", False):
```

```
    self.controller.cnc.running = False
```

```
if getattr(self.controller.cnc, "status_thread", None):
```

```
    self.controller.cnc.status_thread.join(timeout=2)
```

```
# 5) grbl-streamer (poll thread) -----
```

```
if getattr(self.controller.cnc, "grbl", None):
```

```
    try:
```

```
        self.controller.cnc.grbl.poll_stop()
```

```
        self.controller.cnc.grbl.disconnect() # fecha serial + join
```

```
except Exception:
    pass
```

```
# 6) Dispositivos -----
```

```
if getattr(self.controller.camera, "is_connected", False):
    self.controller.camera.disconnect()
if getattr(self.controller.cnc, "is_connected", False):
    self.controller.cnc.disconnect()
```

```
# closeEvent agora só dispara a limpeza
def closeEvent(self, event):
    self._cleanup_resources()
    event.accept()
```

```
class SequenceRunnerThread(QThread):
    """Thread para executar uma sequência de inspeção"""
    image_captured = pyqtSignal(dict) # Emite resultados da captura
    sequence_completed = pyqtSignal() # Emite quando a sequência é concluída
    sequence_error = pyqtSignal(str) # Emite quando ocorre um erro

    def __init__(self, controller, sequence_name):
        super().__init__()
        self.controller = controller
        self.sequence_name = sequence_name

    def run(self):
        try:
            # Execute a sequência e processe os resultados
            def process_result(result):
                self.image_captured.emit(result)

            self.controller.run_sequence(self.sequence_name, process_result)
            self.sequence_completed.emit()

        except Exception as e:
            self.sequence_error.emit(str(e))
```

```
class MapGeneratorThread(QThread):
    """
    Thread responsável por percorrer a grade, movimentar a CNC e capturar
    as imagens sem travar a GUI.
    """
    progress = pyqtSignal(int, int) # imagens_capturadas, total
    image_captured = pyqtSignal(object) # cv2 image (opcional para preview)
    finished = pyqtSignal()
    error = pyqtSignal(str)

    def __init__(self, controller, origin, end, sx, sy, folder, prog_name, feed_rate=1000):
        super().__init__()
        self.ctrl = controller
        self.origin = origin
```

```
self.end = end
self.sx = sx
self.sy = sy
self.folder = folder
self.prog_name = prog_name
self.feed_rate = feed_rate
```

```
def run(self):
    log = logging.getLogger("MapGeneratorThread")
    try:
        points = list(self.ctrl._grid_points(self.origin, self.end,
                                             self.sx, self.sy))
        total = len(points)
        os.makedirs(self.folder, exist_ok=True)

        # -- Vai para a origem (somente se não estivermos nela) -----
        cur = self.ctrl.cnc.get_current_position()
        if (abs(cur['x'] - self.origin['x']) > 1e-3 or
            abs(cur['y'] - self.origin['y']) > 1e-3):
            self.ctrl.cnc.move_to_absolute_position(self.origin['x'],
                                                    self.origin['y'], feed_rate=self.feed_rate)
            self.ctrl.cnc.wait_for_idle()
        else:
            log.debug("MapGeneratorThread: Já estamos na origem; iniciando varredura sem espera extra.")

        captured = 0
        for r, col, x, y in points:
            if self.isInterruptRequested():
                log.warning("Mapa cancelado pelo usuário")
                self.error.emit("Operação cancelada")
                return
            self.ctrl.cnc.move_to_absolute_position(x, y, feed_rate=self.feed_rate)
            self.ctrl.cnc.wait_for_idle()

            img = self.ctrl.camera.capture()
            if img is not None:
                fname = f'{self.prog_name}_r{r:03d}_c{col:03d}.png'
                cv2.imwrite(os.path.join(self.folder, fname), img)
                self.image_captured.emit(img)
            captured += 1
            self.progress.emit(captured, total)
            time.sleep(0.05)

        self.finished.emit()
    except Exception as exc:
        log.exception("Erro na geração do mapa")
        self.error.emit(str(exc))
```

```
if __name__ == "__main__":
    app = QApplication(sys.argv)
```

```
window = AOIControllerApp()  
window.show()  
sys.exit(app.exec())
```